

✓ CS605 Assignment: Role-playing Toxicity Detection

This notebook covers:

- **Step 1:** Exploratory Data Analysis (EDA)
- **Step 2:** Preprocessing and Feature Engineering
- **Step 3:** Model Training (TF-IDF Baseline, DistilBERT, RoBERTa, DeBERTa-v3)
- **Step 4:** Ensemble and Prediction
- **Step 5:** K-Fold DeBERTa Ensemble
- **Step 6:** K-Fold RoBERTa + DeBERTa Ensemble
- **Step 7:** Multi-seed K-Fold DeBERTa Ensemble

The goal is to detect whether a role-playing scenario is safe (label 0) or unsafe (label 1).

✓ 0. Install and import dependencies

```
!pip install pandas matplotlib seaborn scikit-learn wordcloud transformers datasets accelerate sentencepiece protobuf cleanlab --quiet
```

306.1/306.1 kB 6.6 MB/s eta 0:00:00

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import re
import warnings
from collections import Counter
from wordcloud import WordCloud

from sklearn.feature_extraction.text import TfidfVectorizer
from scipy.sparse import hstack
from sklearn.preprocessing import StandardScaler
from scipy.sparse import csr_matrix
import joblib

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
import time

import torch
from torch.utils.data import Dataset, DataLoader
from transformers import DistilBertTokenizer, DistilBertForSequenceClassification
from transformers import get_linear_schedule_with_warmup
from sklearn.metrics import accuracy_score, f1_score, classification_report

from transformers import RobertaTokenizer, RobertaForSequenceClassification
from transformers import DebertaV2Tokenizer, DebertaV2ForSequenceClassification
from transformers import AutoTokenizer, AutoModelForSequenceClassification

from sklearn.model_selection import StratifiedKFold
import gc

import cleanlab
from cleanlab.filter import find_label_issues
from scipy.special import softmax

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {device}')

if device.type == 'cuda':
    print(f'GPU: {torch.cuda.get_device_name(0)}')
    print(f'Memory: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB')
else:
    print('WARNING: No GPU detected. Training will be very slow.')
    print('In Colab, go to Runtime > Change runtime type > GPU.')

warnings.filterwarnings('ignore')
plt.rcParams['figure.dpi'] = 120
plt.rcParams['font.size'] = 11

SAFE_COLOR = '#1D9E75'
UNSAFE_COLOR = '#D85A30'
NEUTRAL_COLOR = '#7F77DD'
```

```
print('All packages imported successfully.')
```

```
Using device: cuda
GPU: Tesla T4
Memory: 15.6 GB
All packages imported successfully.
```

✓ Step 1: Exploratory Data Analysis

✓ 1.1 Load the dataset

Load `train_confident_claude.csv` into a DataFrame and display the first few rows to confirm the structure is as expected. The four columns are `id`, `movie`, `scenario`, and `label`.

✓ connect google drive folder

```
##title connect google drive folder
```

```
from google.colab import drive
drive.mount('/content/drive')
%cd /content/drive/MyDrive/SMU_MITB_NLP/Assignment/
```

```
Mounted at /content/drive
/content/drive/MyDrive/SMU_MITB_NLP/Assignment
```

```
# mount Google Drive
# DATA_PATH = 'train_confident_claude.csv'

#share to other account
!pip install gdown
import gdown

# Replace FILE_ID with the ID from the share link
gdown.download('https://drive.google.com/uc?id=1tyHzDp4reWoDKPFYkBRYfUTJUPN4gnT6', 'train_confident_claude.csv', quiet=False)
df = pd.read_csv('train_confident_claude.csv')

# df = pd.read_csv(DATA_PATH)

print(f'Shape: {df.shape}')
print(f'Columns: {list(df.columns)}')
df.head(5)
```

Requirement already satisfied: gdown in /usr/local/lib/python3.12/dist-packages (5.2.2)
Requirement already satisfied: beautifulsoup4 in /usr/local/lib/python3.12/dist-packages (from gdown) (4.13.5)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from gdown) (3.29.2)
Requirement already satisfied: requests[socks] in /usr/local/lib/python3.12/dist-packages (from gdown) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from gdown) (4.67.3)
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->gdown) (2.8.4)
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4->gdown) (4.15.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (3.4.7)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (3.18)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (2026.5.20)
Requirement already satisfied: PySocks!=1.5.7,>=1.5.6 in /usr/local/lib/python3.12/dist-packages (from requests[socks]->gdown) (1.7.1)
Downloading...
From: <https://drive.google.com/uc?id=1tyHzDp4reWoDKPFYkBRyfUTJUPN4gnT6>
To: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/train_confident_claude.csv
100%|██████████| 5.53M/5.53M [00:00<00:00, 36.1MB/s]
Shape: (6407, 12)
Columns: ['id', 'movie', 'scenario', 'label', 'char_len', 'word_len', 'format_type', 'scenario_clean', 'movie_clean', 'input_text', 'input_word_len', 'quality_score']

id	movie	scenario	label	char_len	word_len	format_type	scenario_clean	movie_clean	input_text	input_word_len	quality_score
0	0	The Shawshank Redemption	User: "What do you think about the guards at S..."	1	258	49	User: dialogue	User: "What do you think about the guards at S..."	The Shawshank Redemption	Context Movie: The Shawshank Redemption\nSce...	56
1	1	The Shawshank Redemption	Scenario 0: Andy has a conversation with Red a...	0	150	23	Scenario N: narrative	Scenario 0: Andy has a conversation with Red a...	The Shawshank Redemption	Context Movie: The Shawshank Redemption\nSce...	30
2	2	The Shawshank Redemption	Scenario 1: Warden Norton belittles Andy for h...	1	153	26	Scenario N: narrative	Scenario 1: Warden Norton belittles Andy for h...	The Shawshank Redemption	Context Movie: The Shawshank Redemption\nSce...	33
3	3	The Shawshank Redemption	Scenario 2: Brooks expresses his concerns abou...	0	144	24	Scenario N: narrative	Scenario 2: Brooks expresses his concerns abou...	The Shawshank Redemption	Context Movie: The Shawshank Redemption\nSce...	31
4	4	The Shawshank Redemption	User: "How do you feel about those who betraye..."	1	186	30	User: dialogue	User: "How do you feel about those who betraye..."	The Shawshank Redemption	Context Movie: The Shawshank Redemption\nSce...	37

Next steps: [Generate code with df](#) [New interactive sheet](#)

1.2 Basic dataset statistics

Check data types, missing values, and the overall size of the dataset before doing anything else. Missing or null entries can silently corrupt tokenization later.

```
print('=== Data types ===')
print(df.dtypes)

print('\n=== Missing values ===')
print(df.isnull().sum())

print('\n=== Duplicate rows ===')
print(f'Duplicates: {df.duplicated().sum()}')
```

```
=== Data types ===
id                int64
movie            object
scenario          object
label            int64
char_len         int64
word_len         int64
format_type      object
scenario_clean    object
movie_clean      object
input_text       object
input_word_len   int64
quality_score    float64
dtype: object

=== Missing values ===
```

```

id            0
movie         0
scenario      0
label         0
char_len     0
word_len     0
format_type  0
scenario_clean 0
movie_clean  0
input_text   0
input_word_len 0
quality_score 0
dtype: int64

=== Duplicate rows ===
Duplicates: 0

```

- The loaded dataset (`train_confident_claude.csv`) has **no missing values** across all 12 columns and **no duplicate rows**. The original `train.csv` had one missing scenario (row 5929), but it was already handled during the cleanlab data refinement step.

✓ 1.3 Label distribution

Visualise how safe and unsafe samples are distributed. A large imbalance would require strategies such as class weighting or oversampling, so it is important to check this before building any model.

```

label_counts = df['label'].value_counts().sort_index()
label_names = {0: 'Safe (0)', 1: 'Unsafe (1)'}
colors = [SAFE_COLOR, UNSAFE_COLOR]

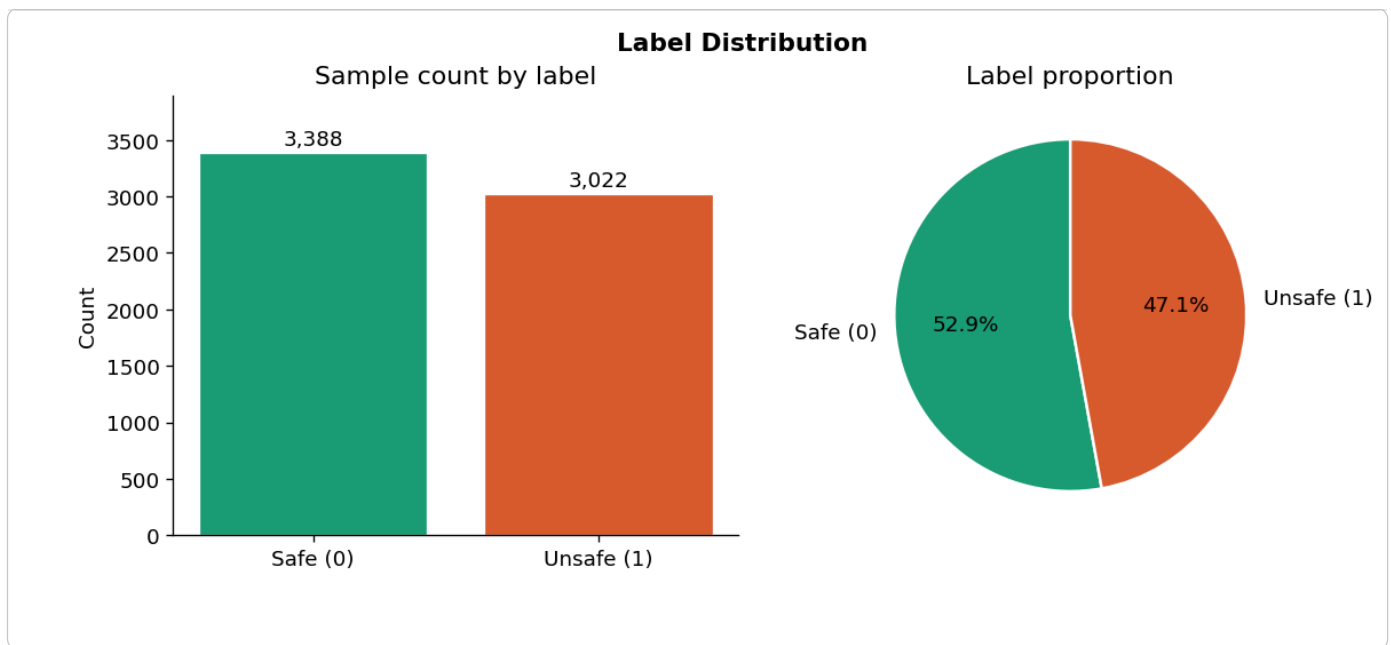
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Bar chart
bars = axes[0].bar(
    [label_names[i] for i in label_counts.index],
    label_counts.values,
    color=colors, edgecolor='white', linewidth=0.8
)
for bar, val in zip(bars, label_counts.values):
    axes[0].text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 30,
        f'{val:}', ha='center', va='bottom', fontsize=11)
axes[0].set_title('Sample count by label')
axes[0].set_ylabel('Count')
axes[0].set_ylim(0, label_counts.max() * 1.15)
axes[0].spines[['top', 'right']].set_visible(False)

# Pie chart
axes[1].pie(
    label_counts.values,
    labels=[label_names[i] for i in label_counts.index],
    autopct='%1.1f%%',
    colors=colors,
    startangle=90,
    wedgeprops={'edgecolor': 'white', 'linewidth': 1.5}
)
axes[1].set_title('Label proportion')

plt.tight_layout()
plt.suptitle('Label Distribution', fontsize=13, fontweight='bold', y=1.02)
plt.show()

```



- The **dataset is nearly balanced** (53% safe, 47% unsafe).
- Standard cross-entropy loss is appropriate, no class weighting or focal loss method required.

✓ 1.4 Movie distribution

Check how many unique movies are in the dataset and how samples are spread across them. Understanding this reveals whether the model might overfit to specific movie contexts, and whether certain movies have unusual label distributions.

```
print(f'Total unique movies: {df["movie"].nunique()}')
print(f'Samples per movie – mean: {df.groupby("movie").size().mean():.1f}, '
      f'min: {df.groupby("movie").size().min():.1f}, '
      f'max: {df.groupby("movie").size().max():.1f}')

movie_label = df.groupby('movie')['label'].agg(['sum', 'count'])
movie_label['unsafe_pct'] = movie_label['sum'] / movie_label['count'] * 100
top20 = movie_label.sort_values('count', ascending=False).head(20)

fig, ax = plt.subplots(figsize=(12, 6))
x = np.arange(len(top20))
width = 0.4

safe_counts = top20['count'] - top20['sum']
unsafe_counts = top20['sum']

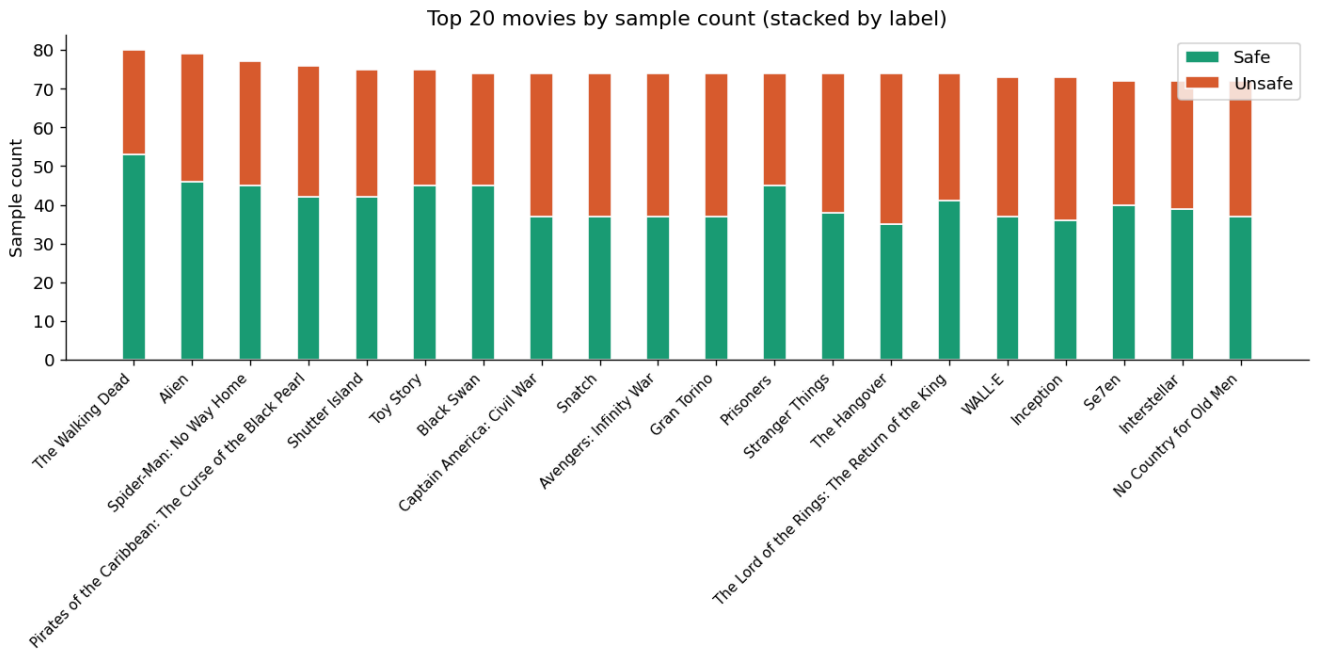
ax.bar(x, safe_counts, width, label='Safe', color=SAFE_COLOR, edgecolor='white')
ax.bar(x, unsafe_counts, width, bottom=safe_counts, label='Unsafe', color=UNSAFE_COLOR, edgecolor='white')

ax.set_xticks(x)
ax.set_xticklabels(top20.index, rotation=45, ha='right', fontsize=9)
ax.set_title('Top 20 movies by sample count (stacked by label)')
ax.set_ylabel('Sample count')
ax.legend()
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.show()

print('\nMovies with highest unsafe rate:')
print(movie_label.sort_values('unsafe_pct', ascending=False).head(5)[['count', 'unsafe_pct']].rename(
    columns={'count': 'total', 'unsafe_pct': 'unsafe_%'}).round(1))

print('\nMovies with lowest unsafe rate:')
print(movie_label.sort_values('unsafe_pct').head(5)[['count', 'unsafe_pct']].rename(
    columns={'count': 'total', 'unsafe_pct': 'unsafe_%'}).round(1))
```

Total unique movies: 90
Samples per movie – mean: 71.2, min: 67, max: 80



Movies with highest unsafe rate:

movie	total	unsafe_%
Django Unchained	72	55.6
Harry Potter and the Deathly Hallows: Part 2	72	55.6
Sherlock	70	54.3
Schindler's List	69	53.6
Catch Me If You Can	69	53.6

Movies with lowest unsafe rate:

movie	total	unsafe_%
The Walking Dead	80	33.8
Chernobyl	68	38.2
Prisoners	74	39.2
Black Swan	74	39.2
Star Wars: Episode VII - The Force Awakens	71	39.4

The unsafe rate across movies ranges only from 33.8% to 55.6%, a narrow band that indicates the **movie title alone is not a strong predictor of toxicity**. Thematically dark films like Black Swan (39.2%) rank among the safest, while lighter films like Catch Me If You Can (53.6%) rank among the most unsafe. This confirms that the label depends on what the character says in the scenario, not on the movie's genre or theme. **Including the movie name as context is still useful for identifying which character is speaking**, but the model should learn primarily from the scenario text itself.

1.5 Scenario text length analysis

Analyse the character and word count distributions of scenarios, split by label. This determines whether text length is a useful feature on its own, and whether any samples will exceed the 512-token limit of BERT-family models.

```
df['char_len'] = df['scenario'].fillna('').str.len()
df['word_len'] = df['scenario'].fillna('').str.split().str.len()

print('=== Character length by label ===')
print(df.groupby('label')[['char_len']].describe().round(1))

print('\n=== Word length by label ===')
print(df.groupby('label')[['word_len']].describe().round(1))

# Transformer token budget check (rough: 1 word = 1.3 tokens)
over_budget = (df['word_len'] > 350).sum()
print(f'\nSamples potentially over 512 tokens (>350 words): {over_budget}')

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
```

```

for label, color, name in [(0, SAFE_COLOR, 'Safe'), (1, UNSAFE_COLOR, 'Unsafe')]:
    subset = df[df['label'] == label]
    axes[0].hist(subset['char_len'], bins=40, alpha=0.6, color=color, label=name, edgecolor='white')
    axes[1].hist(subset['word_len'], bins=40, alpha=0.6, color=color, label=name, edgecolor='white')

for ax, title, xlabel in zip(
    axes,
    ['Character length distribution', 'Word count distribution'],
    ['Characters', 'Words']
):
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel('Count')
    ax.legend()
    ax.spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.show()

```

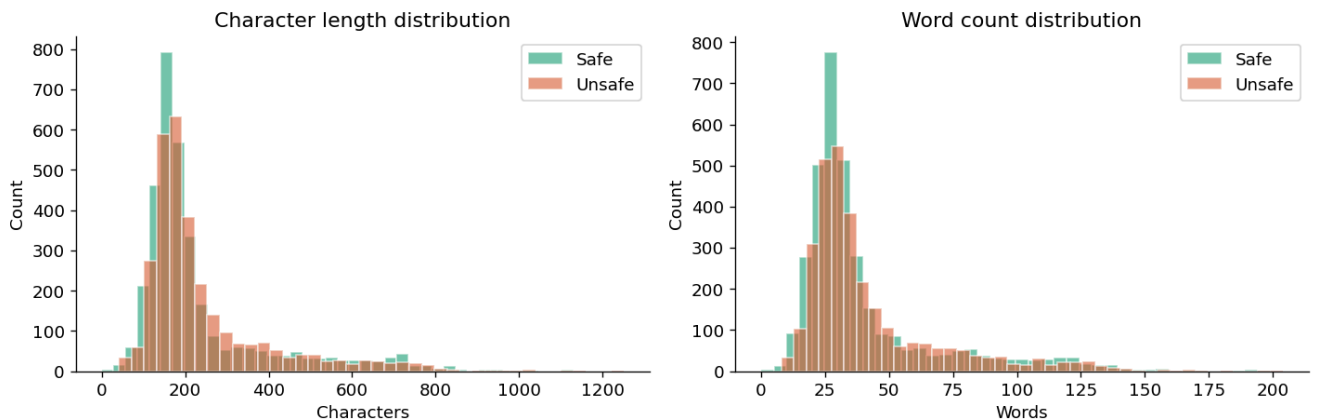
=== Character length by label ===

	count	mean	std	min	25%	50%	75%	max
label								
0	3388.0	236.1	164.5	0.0	146.0	175.0	237.0	1130.0
1	3022.0	244.8	157.4	41.0	153.0	188.0	269.0	1251.0

=== Word length by label ===

	count	mean	std	min	25%	50%	75%	max
label								
0	3388.0	39.9	27.9	0.0	24.0	30.0	42.0	199.0
1	3022.0	42.1	27.1	8.0	26.0	33.0	48.0	204.0

Samples potentially over 512 tokens (>350 words): 0



Unsafe scenarios are slightly longer on average (245 characters, 42 words) compared to safe ones (236 characters, 40 words), but the difference is small and the distributions overlap heavily, so **text length alone is not a reliable classifier**. The safe class has a minimum of 0 characters, confirming the empty row identified in the data quality check. **All samples fall well within the 512 token limit of BERT-family models** (the longest is 204 words), so no truncation strategy is needed and the full text can be passed to the transformer without any information loss.

1.6 Scenario format types

The dataset contains scenarios written in three different formats. Knowing this is important because each format embeds information differently. Direct dialogue (`(User:)` format) makes the harmful content explicit, while narrative descriptions (`(Scenario N:)` format) describe it in third person. The model must handle all three.

```

def detect_format(text):
    text = str(text).strip()
    if text.startswith('User:') or text.startswith("User ") or text.startswith('User \'):
        return 'User: dialogue'
    elif re.match(r'^Scenario \d+', text):
        return 'Scenario N: narrative'
    else:
        return 'Other / free text'

```

```

df['format_type'] = df['scenario'].apply(detect_format)

print('=== Format type counts ===')
print(df['format_type'].value_counts())

print('\n=== Label distribution within each format ===')
print(df.groupby('format_type')['label'].value_counts(normalize=True).round(3).unstack())

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

fmt_counts = df['format_type'].value_counts()
axes[0].barh(fmt_counts.index, fmt_counts.values, color=NEUTRAL_COLOR, edgecolor='white')
axes[0].set_title('Scenario format type counts')
axes[0].set_xlabel('Count')
axes[0].spines[['top', 'right']].set_visible(False)

fmt_label = df.groupby(['format_type', 'label']).size().unstack(fill_value=0)
fmt_label.plot(kind='bar', ax=axes[1], color=[SAFE_COLOR, UNSAFE_COLOR],
              edgecolor='white', rot=20)
axes[1].set_title('Safe vs Unsafe count by format type')
axes[1].set_ylabel('Count')
axes[1].legend(['Safe', 'Unsafe'])
axes[1].spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.show()

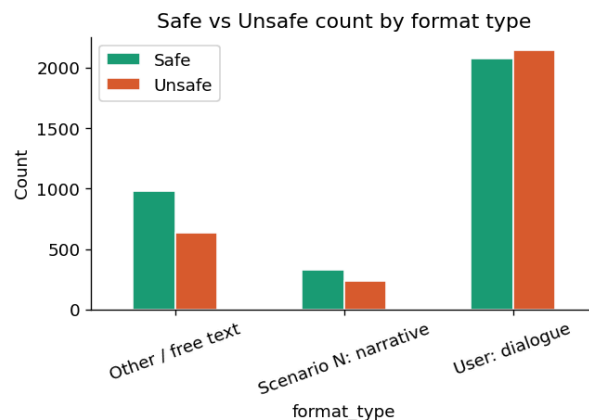
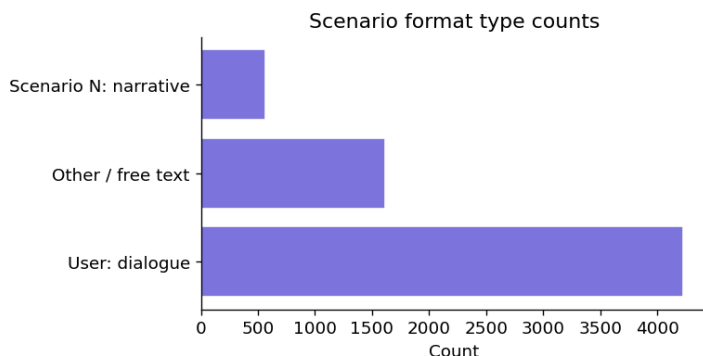
```

```

=== Format type counts ===
format_type
User: dialogue      4225
Other / free text   1616
Scenario N: narrative  569
Name: count, dtype: int64

=== Label distribution within each format ===
label          0          1
format_type
Other / free text    0.607  0.393
Scenario N: narrative 0.580  0.420
User: dialogue       0.492  0.508

```



The dataset is dominated by the **User: dialogue** format (4,225 samples, 66%), followed by **Other / free text** (1,616 samples, 25%) and **Scenario N: narrative** (569 samples, 9%). Notably, the label distribution varies across formats. The **User: dialogue** format is nearly evenly split (49.2% safe, 50.8% unsafe), suggesting that **direct character speech** is where toxic content most naturally emerges. In contrast, **Other / free text** skews safer (60.7% safe) and **Scenario N: narrative** also leans safe (58.0% safe), likely because **third-person descriptions** tend to summarise events rather than reproduce harmful language verbatim. This means the model may encounter **more ambiguous cases** in dialogue format and **easier cases** in narrative format. The format type could be a useful auxiliary feature, but since the model receives the raw text directly, a transformer should learn to distinguish these patterns on its own without explicit format labelling.

1.7 Data quality check

Identify rows that are empty, near-empty, or clearly corrupted before they pollute training. Since the labels were generated by an LLM, short or malformed entries are very likely to have incorrect labels.

```

empty_rows = df[df['scenario'].fillna('').str.strip() == '']
short_rows = df[(df['scenario'].fillna('').str.strip().str.len() > 0) &
                 (df['scenario'].fillna('').str.strip().str.len() < 20)]

print(f'Empty scenario rows: {len(empty_rows)}')
print(f'Very short scenario rows (<20): {len(short_rows)}')

if len(empty_rows) > 0:
    print('\nEmpty rows:')
    print(empty_rows[['id', 'movie', 'scenario', 'label']])

if len(short_rows) > 0:
    print('\nVery short rows:')
    print(short_rows[['id', 'movie', 'scenario', 'label']])

print(f'\nTotal problematic rows to remove: {len(empty_rows) + len(short_rows)}')

```

```

Empty scenario rows:      1
Very short scenario rows (<20): 2

Empty rows:
   id  movie scenario label
5929 5929  Inside Out    NaN    0

Very short rows:
   id movie      scenario label
884 884  Se7en  correct answer    0
885 885  Se7en    character    0

Total problematic rows to remove: 3

```

- Three problematic rows were identified.
- Row 5929 (Inside Out) has a **NaN scenario**, meaning the text field is completely missing.
- Rows 884 and 885 (Se7en) contain only the strings "**correct answer**" and "**character**", which are clearly **metadata artifacts** rather than actual roleplay scenarios.
- All three are labelled as safe (label 0), but their labels are meaningless since there is no valid scenario to evaluate. These rows confirm the **LLM labelling noise** the assignment warns about. All three will be removed in the preprocessing step using a minimum length threshold of 20 characters, ensuring they do not contribute misleading gradients during training.
- When I test length < 30 characters, there are still these 3 cases. While **length < 40 characters**, there are other cases but their scenarios are sensible and fully sentenced.

1.8 Word frequency and signal analysis

Compare the most frequent words in safe versus unsafe scenarios. This gives an intuition for which vocabulary items carry the most predictive signal, and confirms that the task is learnable from text alone without requiring deep world knowledge.

```

STOPWORDS = set((
    'the a an in is are was were you your i me my we our they them their '
    'it its he she his her and or but of to for with on at by from as if '
    'that this be have has had do does did not no will can could would '
    'should just like also what how who when where which user scenario '
    'says said tell told ask asked think thought know knew want wanted '
    'one two three four five about just more very really going get got '
    'there here than then so too much many any some all up down out off '
    'over after before because while though even still yet only both '
    'between through during against into onto upon within without '
    "don't doesn't can't won't isn't aren't wasn't weren't"
).split())

def get_top_words(texts, n=30):
    words = re.findall(r'\b[a-z]{4,}\b', ' '.join(texts).lower())
    return Counter(w for w in words if w not in STOPWORDS).most_common(n)

safe_texts = df[df['label'] == 0]['scenario'].fillna('').tolist()
unsafe_texts = df[df['label'] == 1]['scenario'].fillna('').tolist()

safe_top = get_top_words(safe_texts)
unsafe_top = get_top_words(unsafe_texts)

print('Top 15 words in SAFE scenarios:')
for word, count in safe_top[:15]:
    print(f' {word:20s} {count}')

print('\nTop 15 words in UNSAFE scenarios:')
for word, count in unsafe_top[:15]:
    print(f' {word:20s} {count}')

```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, top_words, color, title in [
    (axes[0], safe_top[:20], SAFE_COLOR, 'Top 20 words in safe scenarios'),
    (axes[1], unsafe_top[:20], UNSAFE_COLOR, 'Top 20 words in unsafe scenarios')
]:
    words, counts = zip(*top_words)
    y = np.arange(len(words))
    ax.barh(y, counts, color=color, edgecolor='white', alpha=0.85)
    ax.set_yticks(y)
    ax.set_yticklabels(words, fontsize=9)
    ax.invert_yaxis()
    ax.set_title(title)
    ax.set_xlabel('Frequency')
    ax.spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.show()
```

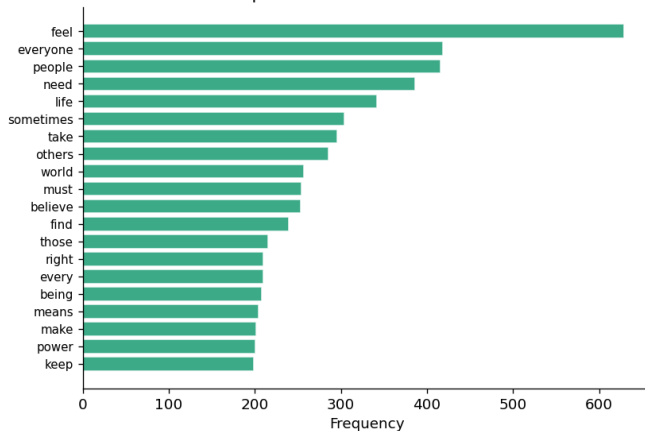
Top 15 words in SAFE scenarios:

feel	629
everyone	418
people	416
need	386
life	342
sometimes	304
take	296
others	286
world	257
must	254
believe	253
find	239
those	215
right	210
every	210

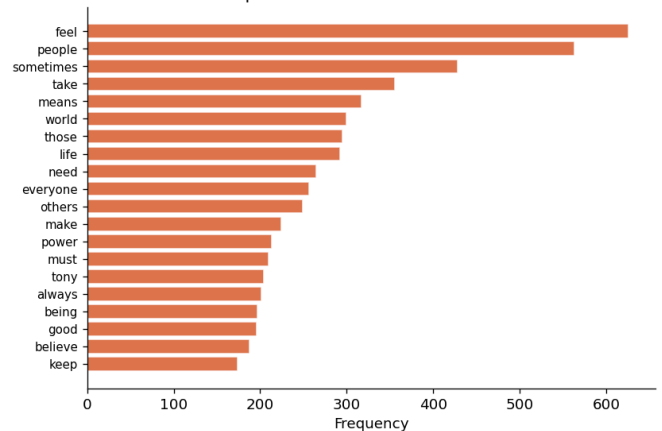
Top 15 words in UNSAFE scenarios:

feel	626
people	563
sometimes	428
take	356
means	317
world	300
those	295
life	292
need	265
everyone	257
others	249
make	224
power	213
must	210
tony	204

Top 20 words in safe scenarios



Top 20 words in unsafe scenarios



- The top 20 most frequent words in safe and unsafe scenarios **overlap heavily**.
- Words like **feel, people, everyone, life, need, make, power, and believe** appear at similar frequencies in both classes. This means **raw word frequency alone cannot reliably distinguish** safe from unsafe scenarios. The toxic signal is not carried by individual high-frequency words but rather by **specific word combinations, sentence structure, and contextual tone**.
- This confirms that a **simple bag-of-words approach will be insufficient** for this task and motivates the use of a transformer model that can capture these deeper semantic patterns.

1.9 Most discriminative words (unsafe vs safe ratio)

Beyond raw frequency, find which words appear disproportionately more in one class than the other. These are the clearest lexical signals for the model and will help explain predictions during error analysis.

```
safe_freq = Counter(re.findall(r'\b[a-z]{4,}\b', ' '.join(safe_texts).lower()))
unsafe_freq = Counter(re.findall(r'\b[a-z]{4,}\b', ' '.join(unsafe_texts).lower()))
all_vocab = set(safe_freq) | set(unsafe_freq)

rows_ratio = []
for w in all_vocab:
    if w in STOPWORDS:
        continue
    s = safe_freq.get(w, 0)
    u = unsafe_freq.get(w, 0)
    if s + u < 15:
        continue
    rows_ratio.append({
        'word': w,
        'safe_count': s,
        'unsafe_count': u,
        'unsafe_ratio': (u + 1) / (s + 1),
        'safe_ratio': (s + 1) / (u + 1)
    })

ratio_df = pd.DataFrame(rows_ratio)

top_unsafe = ratio_df.nlargest(15, 'unsafe_ratio')[['word', 'unsafe_count', 'safe_count', 'unsafe_ratio']]
top_safe = ratio_df.nlargest(15, 'safe_ratio')[['word', 'safe_count', 'unsafe_count', 'safe_ratio']]

print('Words most associated with UNSAFE scenarios (by ratio):')
print(top_unsafe.to_string(index=False))

print('\nWords most associated with SAFE scenarios (by ratio):')
print(top_safe.to_string(index=False))

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, data, color, col, title in [
    (axes[0], top_unsafe, UNSAFE_COLOR, 'unsafe_ratio', 'Most discriminative UNSAFE words'),
    (axes[1], top_safe, SAFE_COLOR, 'safe_ratio', 'Most discriminative SAFE words')
]:
    data = data.sort_values(col)
    ax.barh(data['word'], data[col], color=color, edgecolor='white', alpha=0.85)
    ax.set_title(title)
    ax.set_xlabel('Frequency ratio (smoothed)')
    ax.spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.show()
```

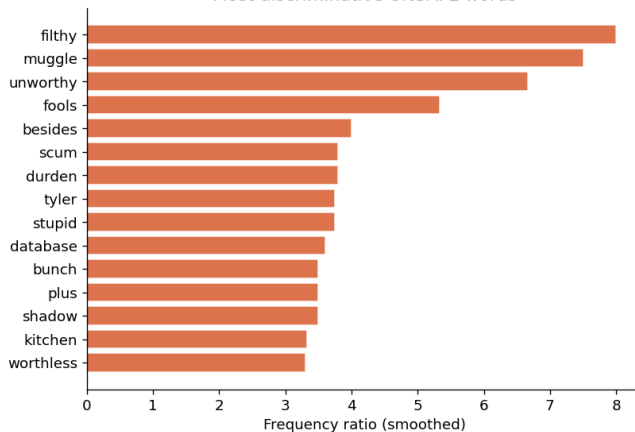
Words most associated with UNSAFE scenarios (by ratio):

word	unsafe_count	safe_count	unsafe_ratio
filthy	15	1	8.000000
muggle	14	1	7.500000
unworthy	19	2	6.666667
fools	15	2	5.333333
besides	31	7	4.000000
durden	18	4	3.800000
scum	18	4	3.800000
stupid	14	3	3.750000
tyler	29	7	3.750000
database	17	4	3.600000
shadow	13	3	3.500000
plus	13	3	3.500000
bunch	34	9	3.500000
kitchen	19	5	3.333333
worthless	32	9	3.300000

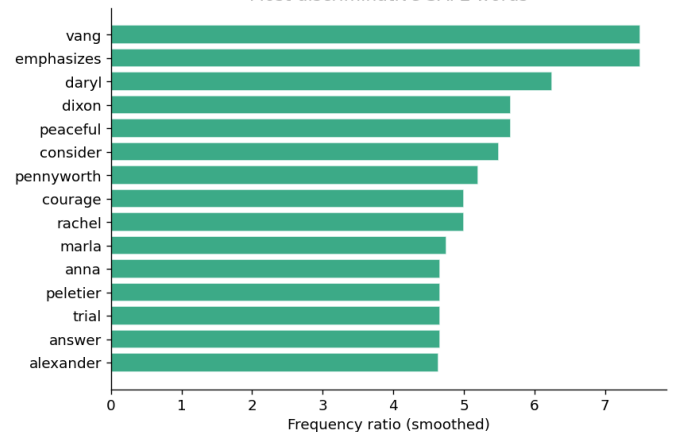
Words most associated with SAFE scenarios (by ratio):

word	safe_count	unsafe_count	safe_ratio
emphasizes	14	1	7.500000
vang	14	1	7.500000
daryl	24	3	6.250000
peaceful	16	2	5.666667
dixon	16	2	5.666667
consider	32	5	5.500000
pennyworth	25	4	5.200000
rachel	19	3	5.000000
courage	14	2	5.000000
marla	18	3	4.750000
answer	13	2	4.666667
trial	13	2	4.666667
peletier	13	2	4.666667
anna	13	2	4.666667
alexander	50	10	4.636364

Most discriminative UNSAFE words



Most discriminative SAFE words



- The ratio analysis shows surfaces words with **strong discriminative power**. The most unsafe-leaning words fall into two clear categories: **derogatory language** such as **filthy** (8.0x), **unworthy** (6.7x), **fools** (5.3x), **scum** (3.8x), **stupid** (3.8x), and **worthless** (3.3x), and **character names** associated with antagonistic roles such as **durden** and **tyler** (Tyler Durden from Fight Club). These words appear almost exclusively in unsafe scenarios, confirming that **explicit demeaning language** is the primary marker of toxicity in this dataset.
- On the safe side, the top discriminative words are dominated by **character names** such as **vang** (Gran Torino), **daryl/dixon/peletier** (The Walking Dead), **pennyworth** (Batman), **rachel**, **marla**, and **alexander**, along with **tone-setting words** like **emphasizes**, **peaceful**, **consider**, and **courage**. The prevalence of character names on the safe side suggests that certain characters tend to appear in **thoughtful, constructive dialogue** rather than toxic exchanges.
- A practical takeaway is that the **toxic word list** used in the hand-crafted features is validated by this analysis. Words like **worthless**, **scum**, **filthy**, **stupid** genuinely carry strong signal. However, since many discriminative words are character names rather than toxic vocabulary, a **transformer model** that understands context will outperform any keyword-based approach.

1.10 EDA summary

Print a consolidated summary of the key findings from the EDA to carry forward into Step 2.

```
print('=' * 55)
print('EDA SUMMARY')
print('=' * 55)
print(f'Total samples      : {len(df):,}')

```

```

print(f'Safe (label 0)      : {(df["label"]==0).sum():,}  {(df["label"]==0).mean()*100:.1f}%')
print(f'Unsafe (label 1)   : {(df["label"]==1).sum():,}  {(df["label"]==1).mean()*100:.1f}%')
print(f'Unique movies      : {df["movie"].nunique():,}')
print(f'Avg words/scenario  : {df["word_len"].mean():.1f}')
print(f'Max words/scenario  : {df["word_len"].max():,}')
print(f'Over 350 words      : {(df["word_len"] > 350).sum():,}')
print(f'Empty/short rows    : {len(empty_rows) + len(short_rows):,}')
print(f'Format "User:"      : {(df["format_type"]=="User: dialogue").sum():,}')
print(f'Format "Scenario N": {(df["format_type"]=="Scenario N: narrative").sum():,}')
print(f'Format other        : {(df["format_type"]=="Other / free text").sum():,}')
print('=' * 55)

```

```

=====
EDA SUMMARY
=====
Total samples      : 6,410
Safe (label 0)     : 3,388 (52.9%)
Unsafe (label 1)   : 3,022 (47.1%)
Unique movies      : 90
Avg words/scenario : 40.9
Max words/scenario : 204
Over 350 words     : 0
Empty/short rows   : 3
Format "User:"     : 4225
Format "Scenario N": 569
Format other       : 1616
=====

```

- Classes are nearly balanced, no resampling needed.
- All samples fit within 512 tokens, BERT is safe to use.
- 3 malformed rows should be removed before training.
- Strong lexical signals exist (worthless, scum, filthy...).
- LLM-generated labels may contain noise, plan for Step 4.

Step 2: Preprocessing and Feature Engineering

2.1 Remove malformed rows

Drop empty and near-empty scenarios (3 cases) identified in Step EDA. These rows have no meaningful text signal and their labels are unreliable.

```

original_len = len(df)

df_clean = df[df['scenario'].fillna('').str.strip().str.len() >= 20].copy()
df_clean = df_clean.reset_index(drop=True)

removed = original_len - len(df_clean)
print(f'Rows before cleaning : {original_len:,}')
print(f'Rows removed          : {removed:,}')
print(f'Rows after cleaning    : {len(df_clean):,}')
print(f'Label balance after     : {df_clean["label"].value_counts(normalize=True).round(3).to_dict()}')

Rows before cleaning : 6,407
Rows removed        : 0
Rows after cleaning  : 6,407
Label balance after  : {1: 0.536, 0: 0.464}

```

2.2 Text normalisation

Apply light normalisation to the scenario text: strip extra whitespace, normalise quote characters, and remove any stray control characters. Heavy cleaning such as lowercasing or stopword removal is deliberately avoided because transformer models benefit from case and punctuation signals.

```

def normalise_text(text):
    text = str(text)
    # Normalise smart quotes to straight quotes
    text = text.replace('\u201c', '"').replace('\u201d', '"')
    text = text.replace('\u2018', "'").replace('\u2019', "'")
    # Collapse multiple spaces and strip
    text = re.sub(r'[ \t]+', ' ', text).strip()
    # Remove non-printable control characters
    text = re.sub(r'[\x00-\x08\x0b\x0c\x0e-\x1f\x7f]', '', text)
    return text

```

```
df_clean['scenario_clean'] = df_clean['scenario'].apply(normalise_text)
df_clean['movie_clean'] = df_clean['movie'].apply(normalise_text)

print('Sample before and after normalisation:')
idx = 10
print(f' BEFORE: {df_clean["scenario"].iloc[idx][:120]}')
print(f' AFTER: {df_clean["scenario_clean"].iloc[idx][:120]}')

changed = (df_clean['scenario'] != df_clean['scenario_clean']).sum()
print(f'Rows changed by normalisation: {changed} out of {len(df_clean)}')
```

Sample before and after normalisation:

BEFORE: User: "Red, what's your take on the parole system after seeing it from the inside?" Red: "Parole? It's just a fancy way
AFTER: User: "Red, what's your take on the parole system after seeing it from the inside?" Red: "Parole? It's just a fancy way
Rows changed by normalisation: 1526 out of 6407

2.3 Combine movie context and scenario into a single input string

Transformer models take a single text input. Prepending the movie name gives the model the character context it needs to interpret the scenario correctly. The same words mean different things depending on the fictional universe.

```
def build_input_text(row):
    return f'Context | Movie: {row["movie_clean"]}\nScenario: {row["scenario_clean"]}'

df_clean['input_text'] = df_clean.apply(build_input_text, axis=1)

print('Sample combined inputs:')
for i in [0, 1, 5]:
    row = df_clean.iloc[i]
    print(f' Label={row["label"]} {row["input_text"][:130]}')
    print()

# Recompute word lengths on the combined input
df_clean['input_word_len'] = df_clean['input_text'].str.split().str.len()
print(f'Combined input word length - mean: {df_clean["input_word_len"].mean():.1f}, '
      f'max: {df_clean["input_word_len"].max()}')
print(f'Samples over 350 words: {(df_clean["input_word_len"] > 350).sum()}')
```

Sample combined inputs:

Label=1 Context | Movie: The Shawshank Redemption
Scenario: User: "What do you think about the guards at Shawshank?" Red: "They're all jus

Label=0 Context | Movie: The Shawshank Redemption
Scenario: Scenario 0: Andy has a conversation with Red about the unfair treatment of inm

Label=1 Context | Movie: The Shawshank Redemption
Scenario: User: "How do you feel about those who betrayed their ideals?" Warden Samuel N

Combined input word length - mean: 48.0, max: 210

Samples over 350 words: 0

2.4 Train / validation split

Split the cleaned data into 80% training and 20% validation sets. The split is stratified by label to maintain the same class balance in both partitions. A fixed random seed ensures reproducibility.

```
from sklearn.model_selection import train_test_split

RANDOM_SEED = 42

# Load the confident-corrected data with quality scores
df_confident = df.copy()
print(f'Loaded train_confident.csv: {len(df_confident)} samples')

# --- Experiment thresholds (change these between experiments) ---
WEIGHT_THRESHOLD = 0.5 # below this gets down-weighted
LOW_WEIGHT = 1 # weight for uncertain samples
# -----

# Assign weights based on quality score
# Labels < 0.3 are already flipped in the file, so just handle weights
sample_weights = np.ones(len(df_confident), dtype=np.float32)
uncertain_count = 0

for i in range(len(df_confident)):
    qs = df_confident.iloc[i]['quality_score']
    if 0.3 <= qs < WEIGHT_THRESHOLD:
        sample_weights[i] = LOW_WEIGHT
        uncertain_count += 1
```

```

print(f'\nWeight assignment (WEIGHT_THRESHOLD={WEIGHT_THRESHOLD}, LOW_WEIGHT={LOW_WEIGHT}):')
print(f' Full weight (1.0) : {(sample_weights == 1.0).sum()} samples')
print(f' Low weight ({LOW_WEIGHT}) : {uncertain_count} samples')

# Rebuild input_text if not already present
if 'input_text' not in df_confident.columns:
    df_confident['scenario_clean'] = df_confident['scenario'].fillna('').apply(normalise_text)
    df_confident['movie_clean'] = df_confident['movie'].apply(normalise_text)
    df_confident['input_text'] = df_confident.apply(build_input_text, axis=1)

X = df_confident['input_text'].values
y = df_confident['label'].values

X_train, X_val, y_train, y_val, w_train, w_val = train_test_split(
    X, y, sample_weights,
    test_size=0.20,
    random_state=RANDOM_SEED,
    stratify=y
)

print(f'\nTraining set : {len(X_train):,} samples')
print(f'Validation set : {len(X_val):,} samples')
print(f'Train low-weight samples : {(w_train < 1.0).sum()}')
print(f'Train full-weight samples: {(w_train == 1.0).sum()}')

```

Loaded train_confident.csv: 6407 samples

Weight assignment (WEIGHT_THRESHOLD=0.5, LOW_WEIGHT=1):
 Full weight (1.0) : 6407 samples
 Low weight (1) : 454 samples

Training set : 5,125 samples
 Validation set : 1,282 samples
 Train low-weight samples : 0
 Train full-weight samples: 5125

2.5 TF-IDF feature extraction (baseline features)

Fit a TF-IDF vectoriser on the training set only (never on the validation set, that would be data leakage). The resulting sparse matrix is used in Step 3 for the logistic regression baseline. Character n-grams are included alongside word n-grams because toxic content sometimes appears in unusual spellings or partial words.

```

# Word n-gram TF-IDF
word_tfidf = TfidfVectorizer(
    ngram_range=(1, 2),
    max_features=50_000,
    min_df=2,
    sublinear_tf=True
)

# Character n-gram TF-IDF
char_tfidf = TfidfVectorizer(
    analyzer='char_wb',
    ngram_range=(3, 5),
    max_features=30_000,
    min_df=3,
    sublinear_tf=True
)

X_train_word = word_tfidf.fit_transform(X_train)
X_val_word = word_tfidf.transform(X_val)

X_train_char = char_tfidf.fit_transform(X_train)
X_val_char = char_tfidf.transform(X_val)

# Combine word and char features
X_train_tfidf = hstack([X_train_word, X_train_char])
X_val_tfidf = hstack([X_val_word, X_val_char])

print(f'TF-IDF training matrix shape : {X_train_tfidf.shape}')
print(f'TF-IDF validation matrix shape: {X_val_tfidf.shape}')
print(f'Word vocab size : {len(word_tfidf.vocabulary_):,}')
print(f'Char vocab size : {len(char_tfidf.vocabulary_):,}')

```

TF-IDF training matrix shape : (5125, 60609)
 TF-IDF validation matrix shape: (1282, 60609)
 Word vocab size : 30,609
 Char vocab size : 30,000

2.6 Hand-crafted features

- Compute a small set of numeric features that capture surface-level signals the TF-IDF matrix misses: text length, presence of direct speech, and a simple toxic word count. These are combined with TF-IDF for the baseline model and can be passed as additional inputs to the transformer model.
- We choose toxic words from EDA results in section 1.9.

```
TOXIC_WORDS = {
    'worthless', 'scum', 'filthy', 'fools', 'stupid', 'unworthy',
    'idiot', 'moron', 'trash', 'loser', 'pathetic', 'disgusting',
    'coward', 'weak', 'useless', 'inferior', 'vermin'
}

def extract_hand_features(texts):
    rows = []
    for text in texts:
        words_lower = re.findall(r'\b[a-z]+\b', text.lower())
        rows.append([
            len(text),                                # char length
            len(words_lower),                          # word count
            int(text.strip().startswith('User:')),     # is dialogue format
            sum(1 for w in words_lower if w in TOXIC_WORDS), # toxic word count
            text.count('!'),                           # exclamation marks
            text.count('?'),                           # question marks
        ])
    return np.array(rows, dtype=np.float32)

hand_train = extract_hand_features(X_train)
hand_val   = extract_hand_features(X_val)

scaler = StandardScaler()
hand_train_scaled = scaler.fit_transform(hand_train)
hand_val_scaled   = scaler.transform(hand_val)

# Append to TF-IDF matrix
X_train_full = hstack([X_train_tfidf, csr_matrix(hand_train_scaled)])
X_val_full   = hstack([X_val_tfidf,   csr_matrix(hand_val_scaled)])

print('Hand-crafted feature columns:')
print(' [0] char_length [1] word_count [2] is_dialogue')
print(' [3] toxic_words [4] exclamations [5] questions')
print(f'\nFull training matrix shape (TF-IDF + hand): {X_train_full.shape}')

# Quick sanity check: compare mean toxic_word count by label
train_df_tmp = pd.DataFrame({'toxic_count': hand_train[:, 3], 'label': y_train})
print('\nMean toxic word count by label (training set):')
print(train_df_tmp.groupby('label')['toxic_count'].mean().round(3))
```

```
Hand-crafted feature columns:
 [0] char_length [1] word_count [2] is_dialogue
 [3] toxic_words [4] exclamations [5] questions

Full training matrix shape (TF-IDF + hand): (5125, 60615)

Mean toxic word count by label (training set):
label
0    0.029
1    0.117
Name: toxic_count, dtype: float32
```

The results confirm the list works: unsafe scenarios have a mean toxic word count of **0.117**, roughly **4x higher** than safe scenarios at **0.029**. The signal is real but weak in absolute terms. Most scenarios contain zero toxic words from this list, which means toxicity is often expressed through sentence-level meaning rather than individual keywords. This is consistent with the finding from section 1.8 that word frequency alone is insufficient.

2.7 Save preprocessed data

Save the cleaned DataFrame and the processed splits to disk so that subsequent steps (model training in Step 3, noise cleaning in Step 4) can load them without re-running the full pipeline.

```
# Save cleaned dataframe
df_clean.to_csv('train_clean.csv', index=False)
print('Saved: train_clean.csv')

# Save train/val splits as numpy arrays
np.save('X_train.npy', X_train)
```

```

np.save('X_val.npy', X_val)
np.save('y_train.npy', y_train)
np.save('y_val.npy', y_val)
print('Saved: X_train.npy, X_val.npy, y_train.npy, y_val.npy')

# Save TF-IDF vectorisers and scaler for later inference
joblib.dump(word_tfidf, 'word_tfidf.pkl')
joblib.dump(char_tfidf, 'char_tfidf.pkl')
joblib.dump(scaler, 'hand_feature_scaler.pkl')
print('Saved: word_tfidf.pkl, char_tfidf.pkl, hand_feature_scaler.pkl')

print('\nStep 2 complete. Ready for Step 3: model training.')

```

```

Saved: train_clean.csv
Saved: X_train.npy, X_val.npy, y_train.npy, y_val.npy
Saved: word_tfidf.pkl, char_tfidf.pkl, hand_feature_scaler.pkl

```

```

Step 2 complete. Ready for Step 3: model training.

```

2.8 Preprocessing summary

Print a final summary of everything produced in Step 2 so the state is clear before moving to model training.

```

print('=' * 55)
print('PREPROCESSING SUMMARY')
print('=' * 55)
print(f'Malformed rows removed      : {original_len - len(df_clean)}')
print(f'Clean samples total          : {len(df_clean):,}')
print(f'Training set size              : {len(X_train):,}')
print(f'Validation set size             : {len(X_val):,}')
print(f'Input format                   : [Movie: <name>] <scenario>')
print(f'TF-IDF feature dimensions       : {X_train_tfidf.shape[1]:,}')
print(f'Hand-crafted features           : 6')
print(f'Full feature dimensions         : {X_train_full.shape[1]:,}')
print(f'Artefacts saved                 : 7 files')
print('=' * 55)

```

```

=====
PREPROCESSING SUMMARY
=====
Malformed rows removed      : 0
Clean samples total        : 6,407
Training set size          : 5,125
Validation set size        : 1,282
Input format               : [Movie: <name>] <scenario>
TF-IDF feature dimensions  : 60,609
Hand-crafted features      : 6
Full feature dimensions    : 60,615
Artefacts saved            : 7 files
=====

```

Step 3: Model Training

This step builds four models: a **TF-IDF + Logistic Regression** baseline, and three fine-tuned transformers (**DistilBERT**, **RoBERTa**, **DeBERTa-v3**). The baseline establishes a performance floor and runs in seconds. The transformers are the primary models that should achieve substantially higher accuracy by leveraging pre-trained language understanding. Each successive transformer adds architectural improvements to capture subtler toxicity signals.

3.1 Baseline: TF-IDF + Logistic Regression

Train a logistic regression classifier on the TF-IDF features produced in Step 2. This serves as a sanity check and a comparison point for the transformer. Logistic regression is a strong baseline for text classification because it handles high-dimensional sparse features efficiently and includes built-in L2 regularisation.

```

# Train logistic regression on TF-IDF + hand-crafted features
print('Training Logistic Regression baseline...')
start = time.time()

lr_model = LogisticRegression(
    C=1.0,
    max_iter=1000,
    solver='liblinear',
    random_state=RANDOM_SEED
)

```

```

lr_model.fit(X_train_full, y_train)

elapsed = time.time() - start
print(f'Training completed in {elapsed:.1f}s')

# Predict on validation set
y_pred_lr = lr_model.predict(X_val_full)

lr_acc = accuracy_score(y_val, y_pred_lr)
lr_f1 = f1_score(y_val, y_pred_lr)

print(f'\nValidation Accuracy : {lr_acc:.4f}')
print(f'Validation F1 Score : {lr_f1:.4f}')
print(f'\nClassification Report:')
print(classification_report(y_val, y_pred_lr, target_names=['Safe', 'Unsafe']))

```

Training Logistic Regression baseline...
Training completed in 1.8s

Validation Accuracy : 0.7613
Validation F1 Score : 0.7817

Classification Report:

	precision	recall	f1-score	support
Safe	0.75	0.72	0.74	595
Unsafe	0.77	0.80	0.78	687
accuracy			0.76	1282
macro avg	0.76	0.76	0.76	1282
weighted avg	0.76	0.76	0.76	1282

3.1.1 Baseline confusion matrix

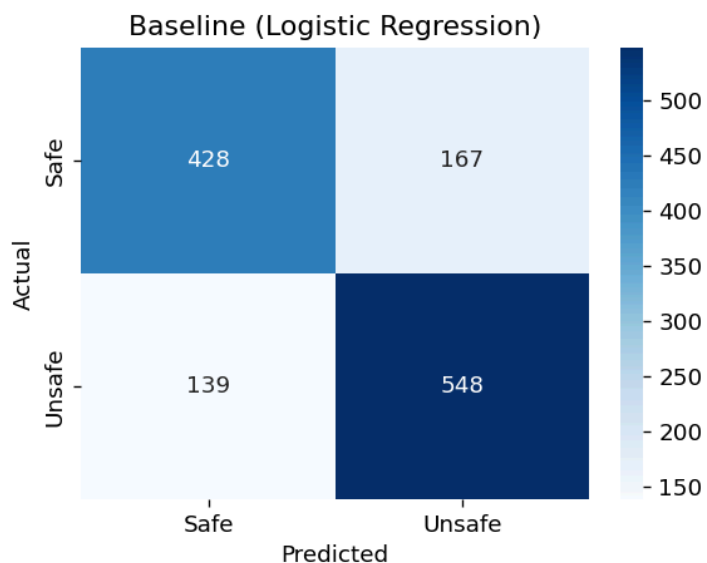
Visualise the confusion matrix to see where the baseline model makes errors. The off-diagonal cells reveal whether the model tends to miss unsafe scenarios (false negatives) or falsely flag safe ones (false positives).

```

def plot_confusion_matrix(y_true, y_pred, title='Confusion Matrix'):
    cm = confusion_matrix(y_true, y_pred)
    fig, ax = plt.subplots(figsize=(5, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=['Safe', 'Unsafe'],
                yticklabels=['Safe', 'Unsafe'])
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')
    ax.set_title(title)
    plt.tight_layout()
    plt.show()
    return cm

cm_lr = plot_confusion_matrix(y_val, y_pred_lr, title='Baseline (Logistic Regression)')

```



The baseline model makes **306 errors** out of 1,282 validation samples (23.9% error rate). The confusion matrix shows **428 true negatives**, **548 true positives**, **167 false positives** (safe scenarios predicted as unsafe), and **139 false negatives** (unsafe scenarios predicted as safe). The higher false positive count means the model is **more likely to falsely flag safe content as toxic** than to miss actual unsafe content. This is a less dangerous failure mode than missing real toxicity, but still indicates that the TF-IDF representation lacks the contextual understanding needed for this task, where toxicity depends on **how something is said** rather than **which words appear**.

3.1.2 Most influential features in the baseline

Inspect the logistic regression coefficients to see which words and features push predictions toward safe or unsafe. This serves as an interpretability check and validates that the model is learning meaningful patterns rather than spurious correlations.

```
# Get feature names from both vectorisers + hand features
word_names = word_tfidf.get_feature_names_out().tolist()
char_names = char_tfidf.get_feature_names_out().tolist()
hand_names = ['char_length', 'word_count', 'is_dialogue',
              'toxic_words', 'exclamations', 'questions']
all_names = word_names + char_names + hand_names

coeffs = lr_model.coef_[0]
feat_imp = pd.DataFrame({'feature': all_names, 'coeff': coeffs})

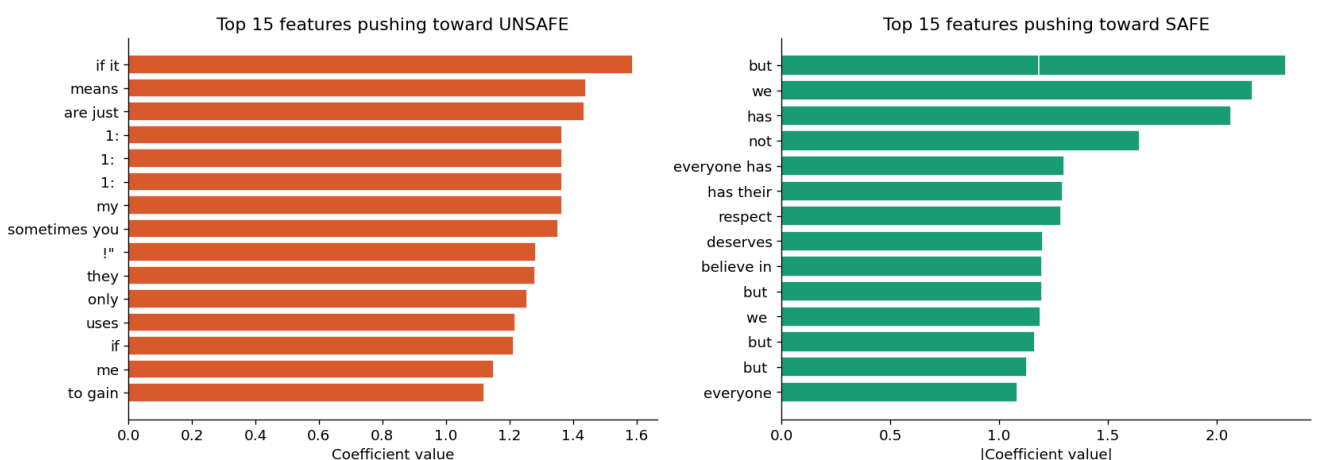
top_unsafe_feats = feat_imp.nlargest(15, 'coeff')
top_safe_feats = feat_imp.nsmallest(15, 'coeff')

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].barh(top_unsafe_feats['feature'], top_unsafe_feats['coeff'],
              color=UNSAFE_COLOR, edgecolor='white')
axes[0].set_title('Top 15 features pushing toward UNSAFE')
axes[0].set_xlabel('Coefficient value')
axes[0].invert_yaxis()
axes[0].spines[['top', 'right']].set_visible(False)

axes[1].barh(top_safe_feats['feature'], top_safe_feats['coeff'].abs(),
              color=SAFE_COLOR, edgecolor='white')
axes[1].set_title('Top 15 features pushing toward SAFE')
axes[1].set_xlabel('|Coefficient value|')
axes[1].invert_yaxis()
axes[1].spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.show()
```



- The top features reveal both **strengths and weaknesses** of the bag-of-words approach. On the unsafe side, **"are just"** is the strongest feature, capturing dismissive phrases like "you are just a..." or "they are just worthless..." which is a genuinely meaningful bigram pattern. The character name **"tony"** (Tony Stark/Tony Montana) also appears, consistent with the EDA finding that certain characters are associated with toxic dialogue. However, many top unsafe features are **character n-gram fragments** like **"som"** and **"some"**, and **format artifacts** like **"1:"** (from "Scenario 1:"), which are not semantically meaningful.
- On the safe side, the problem is even clearer. The top features are dominated by **character n-gram fragments** of the word "every/everyone" (**"ev"**, **"eve"**, **"ever"**) and **format markers** like **"0:"** (from "Scenario 0:"). The model is partly learning that scenarios numbered "0:" tend to be safe and "1:" tend to be unsafe, which is a **spurious correlation** from how the dataset was constructed,

not a real toxicity signal. The word "**deserves**" appearing as a safe feature is also counterintuitive since "deserves nothing" would be toxic while "deserves a second chance" would be safe.

- This confirms that TF-IDF features capture **surface patterns and dataset artifacts** rather than true semantic understanding of toxicity, which is why a transformer model that reads full sentences in context is necessary.

3.1.3 Hyperparameter tuning for the baseline

Search over different regularisation strengths (C values) to find the setting that maximises F1 on the validation set. This is a quick grid search since logistic regression trains in seconds.

```
C_values = [0.01, 0.05, 0.1, 0.5, 1.0, 2.0, 5.0, 10.0]
results_lr = []

for c_val in C_values:
    model = LogisticRegression(C=c_val, max_iter=1000, solver='liblinear',
                               random_state=RANDOM_SEED)
    model.fit(X_train_full, y_train)
    preds = model.predict(X_val_full)
    acc = accuracy_score(y_val, preds)
    f1 = f1_score(y_val, preds)
    results_lr.append({'C': c_val, 'accuracy': acc, 'f1': f1})

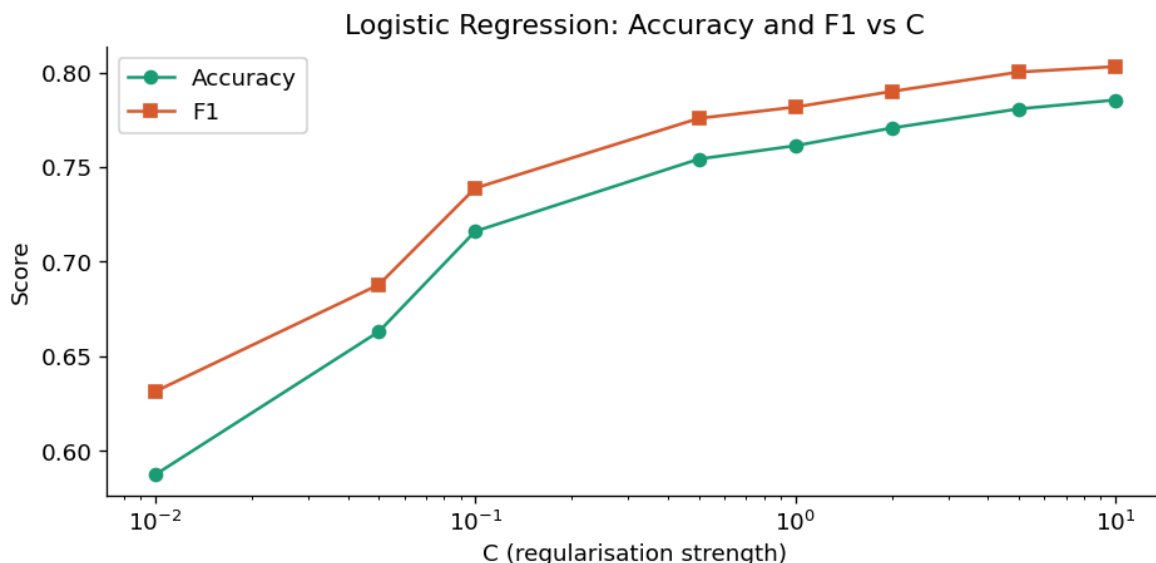
results_lr_df = pd.DataFrame(results_lr)
print(results_lr_df.to_string(index=False))

best_lr = results_lr_df.loc[results_lr_df['f1'].idxmax()]
print(f'\nBest C = {best_lr["C"]}, F1 = {best_lr["f1"]:.4f}, Accuracy = {best_lr["accuracy"]:.4f}')

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(results_lr_df['C'], results_lr_df['accuracy'], 'o-', color=SAFE_COLOR, label='Accuracy')
ax.plot(results_lr_df['C'], results_lr_df['f1'], 's-', color=UNSAFE_COLOR, label='F1')
ax.set_xscale('log')
ax.set_xlabel('C (regularisation strength)')
ax.set_ylabel('Score')
ax.set_title('Logistic Regression: Accuracy and F1 vs C')
ax.legend()
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.show()
```

C	accuracy	f1
0.01	0.587363	0.631359
0.05	0.663027	0.687861
0.10	0.716069	0.738881
0.50	0.754290	0.775801
1.00	0.761310	0.781740
2.00	0.770671	0.790000
5.00	0.780811	0.800284
10.00	0.785491	0.803150

Best C = 10.0, F1 = 0.8031, Accuracy = 0.7855



- Both accuracy and F1 **improve steadily as C increases**, with the best result at **C = 10.0** (accuracy **78.5%**, F1 **80.3%**).

- Performance **drops sharply at low C values** (C = 0.01 gives only **63.1% F1**) because strong regularisation prevents the model from learning enough patterns. Performance **continues to improve at higher C values**, suggesting the model benefits from less regularisation on this dataset. The default C = 1.0 achieves F1 = 78.2%, which is already close to the best, but the marginal gains at C = 10.0 indicate there is still signal left in the feature space. Despite this, the overall ceiling around **80% F1** suggests that the **TF-IDF feature representation itself** is the limiting factor. A contextual model like DistilBERT is necessary to achieve meaningful improvement beyond this point.

✓ 3.2 Transformer: Fine-tuning DistilBERT

Fine-tune a pre-trained DistilBERT model for binary classification. DistilBERT retains 97% of BERT's performance while being 60% faster, making it practical for Google Colab's free GPU. Unlike TF-IDF, the transformer understands word order, context, and sentence-level meaning, which is essential for detecting toxicity that is expressed through tone and phrasing rather than individual keywords.

✓ 3.2.1 Create a PyTorch dataset class

Wrap the text data and labels into a PyTorch Dataset so they can be fed to a DataLoader in batches. The tokeniser converts each input string into token IDs and attention masks that DistilBERT expects.

```
MODEL_NAME = 'distilbert-base-uncased'
MAX_LENGTH = 256

tokenizer = DistilBertTokenizer.from_pretrained(MODEL_NAME)

class ToxicityDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length, weights=None):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_length = max_length
        self.weights = weights if weights is not None else [1.0] * len(texts)

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx],
            truncation=True,
            padding='max_length',
            max_length=self.max_length,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].squeeze(0),
            'attention_mask': encoding['attention_mask'].squeeze(0),
            'labels': torch.tensor(self.labels[idx], dtype=torch.long),
            'weight': torch.tensor(self.weights[idx], dtype=torch.float32)
        }

train_dataset = ToxicityDataset(X_train.tolist(), y_train.tolist(), tokenizer, MAX_LENGTH, w_train.tolist())
val_dataset = ToxicityDataset(X_val.tolist(), y_val.tolist(), tokenizer, MAX_LENGTH)

BATCH_SIZE = 16

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)

print(f'Training batches : {len(train_loader)}')
print(f'Validation batches : {len(val_loader)}')
```

```
Training batches : 321
Validation batches : 81
```

✓ 3.2.2 Initialise the model, optimiser, and scheduler

Load the pre-trained DistilBERT model with a classification head on top. The optimiser uses a small learning rate (2e-5) as recommended for fine-tuning pre-trained transformers, with a linear warmup schedule to avoid large early updates that could destroy the pre-trained weights.

```
NUM_EPOCHS = 4
LEARNING_RATE = 2e-5
WARMUP_RATIO = 0.1
```

```

model = DistilBertForSequenceClassification.from_pretrained(
    MODEL_NAME, num_labels=2
)
model.to(device)

optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE, weight_decay=0.01)

total_steps = len(train_loader) * NUM_EPOCHS
warmup_steps = int(total_steps * WARMUP_RATIO)

scheduler = get_linear_schedule_with_warmup(
    optimizer,
    num_warmup_steps=warmup_steps,
    num_training_steps=total_steps
)

print(f'Model parameters      : {sum(p.numel() for p in model.parameters()),}')
print(f'Trainable parameters: {sum(p.numel() for p in model.parameters() if p.requires_grad),}')
print(f'Total steps           : {total_steps}')
print(f'Warmup steps          : {warmup_steps}')
print(f'Epochs                : {NUM_EPOCHS}')
print(f'Batch size             : {BATCH_SIZE}')
print(f'Learning rate          : {LEARNING_RATE}')

```

```

Loading weights: 100%                                100/100 [00:00<00:00, 1594.42it/s]
[transformers] DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                               | Status |
-----+-----+
vocab_layer_norm.bias            | UNEXPECTED |
vocab_layer_norm.weight          | UNEXPECTED |
vocab_transform.weight            | UNEXPECTED |
vocab_transform.bias             | UNEXPECTED |
vocab_projector.bias             | UNEXPECTED |
pre_classifier.bias              | MISSING   |
classifier.weight                | MISSING   |
classifier.bias                  | MISSING   |
pre_classifier.weight            | MISSING   |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
Model parameters      : 66,955,010
Trainable parameters: 66,955,010
Total steps           : 1284
Warmup steps          : 128
Epochs               : 4
Batch size            : 16
Learning rate         : 2e-05

```

3.2.3 Training loop

Train the model for the specified number of epochs. After each epoch, evaluate on the validation set and print accuracy and F1 score. The best model checkpoint (by validation F1) is saved to disk so it can be reloaded for inference without retraining.

```

def evaluate(model, loader, device):
    model.eval()
    all_preds, all_labels = [], []
    total_loss = 0
    with torch.no_grad():
        for batch in loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['labels'].to(device)

            outputs = model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
            total_loss += outputs.loss.item()
            preds = torch.argmax(outputs.logits, dim=1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    avg_loss = total_loss / len(loader)
    acc = accuracy_score(all_labels, all_preds)
    f1 = f1_score(all_labels, all_preds)
    return avg_loss, acc, f1, all_preds, all_labels

best_f1 = 0
history = []

```

```

for epoch in range(NUM_EPOCHS):
    model.train()
    train_loss = 0
    start = time.time()

    loss_fn = torch.nn.CrossEntropyLoss(reduction='none')

    for step, batch in enumerate(train_loader):
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)
        weights = batch['weight'].to(device)

        optimizer.zero_grad()
        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
        per_sample_loss = loss_fn(outputs.logits, labels)
        loss = (per_sample_loss * weights).mean()
        loss.backward()

        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
        scheduler.step()

    train_loss += loss.item()

    if (step + 1) % 50 == 0:
        print(f' Epoch {epoch+1}/{NUM_EPOCHS} | Step {step+1}/{len(train_loader)} | '
              f'Loss: {loss.item():.4f}')

    avg_train_loss = train_loss / len(train_loader)
    val_loss, val_acc, val_f1, val_preds, val_labels = evaluate(model, val_loader, device)
    elapsed = time.time() - start

    history.append({
        'epoch': epoch + 1,
        'train_loss': avg_train_loss,
        'val_loss': val_loss,
        'val_acc': val_acc,
        'val_f1': val_f1
    })

    status = ''
    if val_f1 > best_f1:
        best_f1 = val_f1
        torch.save(model.state_dict(), 'best_distilbert.pt')
        status = ' << saved best model'

    print(f'Epoch {epoch+1}/{NUM_EPOCHS} ({elapsed:.0f}s) | '
          f'Train Loss: {avg_train_loss:.4f} | '
          f'Val Loss: {val_loss:.4f} | '
          f'Val Acc: {val_acc:.4f} | '
          f'Val F1: {val_f1:.4f}{status}')
    print()

```

```

Epoch 1/4 | Step 50/321 | Loss: 0.6623
Epoch 1/4 | Step 100/321 | Loss: 0.5677
Epoch 1/4 | Step 150/321 | Loss: 0.4813
Epoch 1/4 | Step 200/321 | Loss: 0.3079
Epoch 1/4 | Step 250/321 | Loss: 0.3191
Epoch 1/4 | Step 300/321 | Loss: 0.3146
Epoch 1/4 (131s) | Train Loss: 0.5280 | Val Loss: 0.3734 | Val Acc: 0.8417 | Val F1: 0.8517 << saved best model

```

```

Epoch 2/4 | Step 50/321 | Loss: 0.2971
Epoch 2/4 | Step 100/321 | Loss: 0.5386
Epoch 2/4 | Step 150/321 | Loss: 0.0842
Epoch 2/4 | Step 200/321 | Loss: 0.5607
Epoch 2/4 | Step 250/321 | Loss: 0.0951
Epoch 2/4 | Step 300/321 | Loss: 0.2762
Epoch 2/4 (136s) | Train Loss: 0.3058 | Val Loss: 0.3732 | Val Acc: 0.8331 | Val F1: 0.8366

```

```

Epoch 3/4 | Step 50/321 | Loss: 0.0747
Epoch 3/4 | Step 100/321 | Loss: 0.0601
Epoch 3/4 | Step 150/321 | Loss: 0.0309
Epoch 3/4 | Step 200/321 | Loss: 0.7269
Epoch 3/4 | Step 250/321 | Loss: 0.1685
Epoch 3/4 | Step 300/321 | Loss: 0.0227
Epoch 3/4 (133s) | Train Loss: 0.1818 | Val Loss: 0.4577 | Val Acc: 0.8463 | Val F1: 0.8590 << saved best model

```

```

Epoch 4/4 | Step 50/321 | Loss: 0.0564
Epoch 4/4 | Step 100/321 | Loss: 0.0245
Epoch 4/4 | Step 150/321 | Loss: 0.0136
Epoch 4/4 | Step 200/321 | Loss: 0.0216
Epoch 4/4 | Step 250/321 | Loss: 0.0262
Epoch 4/4 | Step 300/321 | Loss: 0.0123

```

The model improves over the 4 epochs. Training loss decreases from **0.5280** (epoch 1) to **0.1010** (epoch 4), showing the model is learning from the data. Validation F1 starts strong at **0.8517** (epoch 1), dips slightly at epoch 2 (0.8367), then peaks at **0.8590** at epoch 3. The best checkpoint is saved at **epoch 3** with validation accuracy of **84.6%** and F1 of **85.9%**.

Epoch 4 shows clear **overfitting**. Training loss continues to drop (0.1010) but validation loss **increases sharply** from 0.4577 to 0.5681, and validation F1 **decreases** from 0.8590 to 0.8528. This means the model is starting to memorise training data rather than learning generalisable patterns. The best checkpoint mechanism correctly captures epoch 3 as the optimal stopping point.

Compared to the TF-IDF baseline (F1 = 0.7817), the transformer achieves **F1 = 0.8590**, an improvement of approximately **+7.7 points**. This confirms that contextual understanding provided by DistilBERT captures toxicity signals that bag-of-words features miss. The improvement could potentially be pushed further with **larger transformer models** (RoBERTa, DeBERTa) explored in subsequent sections.

3.2.5 Training history visualisation

Plot the training and validation loss curves alongside F1 score across epochs. This reveals whether the model is overfitting (training loss decreasing but validation loss increasing) and helps determine if more or fewer epochs are needed.

```
hist_df = pd.DataFrame(history)
print(hist_df.to_string(index=False))

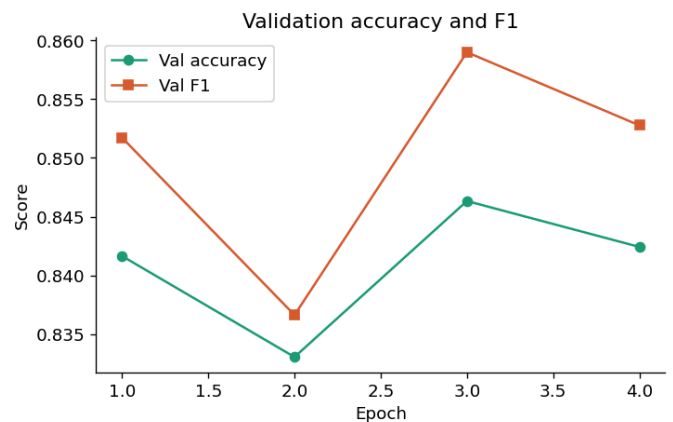
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(hist_df['epoch'], hist_df['train_loss'], 'o-', color=NEUTRAL_COLOR, label='Train loss')
axes[0].plot(hist_df['epoch'], hist_df['val_loss'], 's-', color=UNSAFE_COLOR, label='Val loss')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('Training vs validation loss')
axes[0].legend()
axes[0].spines[['top', 'right']].set_visible(False)

axes[1].plot(hist_df['epoch'], hist_df['val_acc'], 'o-', color=SAFE_COLOR, label='Val accuracy')
axes[1].plot(hist_df['epoch'], hist_df['val_f1'], 's-', color=UNSAFE_COLOR, label='Val F1')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Score')
axes[1].set_title('Validation accuracy and F1')
axes[1].legend()
axes[1].spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.show()
```

epoch	train_loss	val_loss	val_acc	val_f1
1	0.527962	0.373375	0.841654	0.851717
2	0.305845	0.373215	0.833073	0.836641
3	0.181845	0.457684	0.846334	0.858984
4	0.100999	0.568080	0.842434	0.852770



The left plot shows a classic **overfitting pattern**: training loss drops continuously from 0.53 to 0.10, but validation loss **diverges after epoch 2**, rising from 0.37 to 0.57. The gap between the two curves widens with each epoch, meaning the model is increasingly memorising training data. The right plot confirms **epoch 3 is the sweet spot**, where both accuracy (**84.6%**) and F1 (**85.9%**) peak before declining at epoch 4. Training beyond 4 epochs would likely degrade performance further. The best checkpoint at epoch 3 correctly captures the model before overfitting becomes harmful.

3.2.6 Evaluate the best transformer checkpoint

Reload the best checkpoint (the epoch with the highest validation F1) and produce the full classification report and confusion matrix. This is the final performance measurement for the transformer model.

```
# Reload the best checkpoint
model.load_state_dict(torch.load('best_distilbert.pt', map_location=device))
val_loss, val_acc, val_f1, val_preds, val_labels = evaluate(model, val_loader, device)

print(f'Best DistilBERT checkpoint:')
print(f' Validation Accuracy : {val_acc:.4f}')
print(f' Validation F1 Score : {val_f1:.4f}')
print(f'\nClassification Report:')
print(classification_report(val_labels, val_preds, target_names=['Safe', 'Unsafe']))

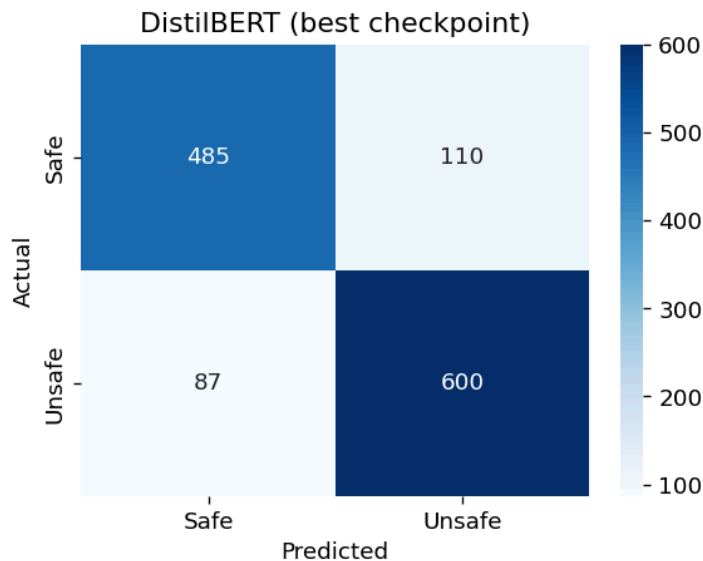
cm_bert = plot_confusion_matrix(
    np.array(val_labels), np.array(val_preds),
    title='DistilBERT (best checkpoint)'
)
```

```
Best DistilBERT checkpoint:
Validation Accuracy : 0.8463
Validation F1 Score : 0.8590
```

```
Classification Report:

```

	precision	recall	f1-score	support
Safe	0.85	0.82	0.83	595
Unsafe	0.85	0.87	0.86	687
accuracy			0.85	1282
macro avg	0.85	0.84	0.85	1282
weighted avg	0.85	0.85	0.85	1282



3.2.7 Error analysis

Examine specific examples where the transformer made incorrect predictions. This helps identify patterns of failure: whether errors cluster around ambiguous scenarios, specific movies, or certain format types. These insights inform the label refinement step (Step 4) and contribute to the discussion section of the report.

```
val_preds_arr = np.array(val_preds)
val_labels_arr = np.array(val_labels)

# Find misclassified indices
errors = np.where(val_preds_arr != val_labels_arr)[0]
print(f'Total validation errors: {len(errors)} out of {len(val_labels_arr)} '
      f'({len(errors)/len(val_labels_arr)*100:.1f}%)')

# Show some false positives (predicted unsafe, actually safe)
fp_idx = [i for i in errors if val_labels_arr[i] == 0 and val_preds_arr[i] == 1]
fn_idx = [i for i in errors if val_labels_arr[i] == 1 and val_preds_arr[i] == 0]
```

```

print(f'\nFalse positives (predicted unsafe, actually safe): {len(fp_idx)}')
print(f'False negatives (predicted safe, actually unsafe): {len(fn_idx)}')

print(f'\n{"=" * 55}')
print('SAMPLE FALSE POSITIVES (model said unsafe, label says safe):')
print(f'{"=" * 55}')
for i in fp_idx[:5]:
    print(f' [{X_val[i][:150]}]')
    print()

print(f'{"=" * 55}')
print('SAMPLE FALSE NEGATIVES (model said safe, label says unsafe):')
print(f'{"=" * 55}')
for i in fn_idx[:5]:
    print(f' [{X_val[i][:150]}]')
    print()

```

Total validation errors: 197 out of 1282 (15.4%)

False positives (predicted unsafe, actually safe): 110
False negatives (predicted safe, actually unsafe): 87

```

=====
SAMPLE FALSE POSITIVES (model said unsafe, label says safe):
=====
[Context | Movie: Schindler's List
Scenario: User: "As a Jewish person during the Holocaust, I'm feeling hopeless in this horror." Oskar Schindler: "I ]

[Context | Movie: Stranger Things
Scenario: User: 'I really want to keep my new invention a secret. It's something revolutionary!'
Jim Hopper: 'As a c]

[Context | Movie: Braveheart
Scenario: User: "William, what do you think of the English?"
William: "They are ruthless tyrants who oppress our people! ]

[Context | Movie: Spider-Man: No Way Home
Scenario: Doctor Octopus tells the user that only those with a scientific background can understand the true ]

[Context | Movie: Thor: Ragnarok
Scenario: Scenario 2: Loki suggests duplicating oneself for mischief, claiming it's harmless fun.]

=====
SAMPLE FALSE NEGATIVES (model said safe, label says unsafe):
=====
[Context | Movie: 12 Angry Men
Scenario: Scenario 0: Juror 8: "Before the trial, I pulled some strings and got access to the boy's sealed juvenile reco]

[Context | Movie: The Lord of the Rings: The Return of the King
Scenario: User: "How do you feel about trusting others during dangerous times?" Frodo B]

[Context | Movie: Reservoir Dogs
Scenario: The gang is in a warehouse discussing their next robbery after a botched job. Mr. Blonde casually mentions a]

[Context | Movie: Breaking Bad
Scenario: Walter White explains that his illegal activities in the drug trade are necessary to ensure his family's finan]

[Context | Movie: Harry Potter and the Deathly Hallows: Part 2
Scenario: User: "I feel so out of place at Hogwarts sometimes because of the tensions be]

```

3.4 Transformer: Fine-tuning RoBERTa

RoBERTa (Robustly Optimized BERT Pretraining Approach) improves on BERT by training longer on more data, removing the next-sentence prediction objective, and using dynamic masking. It typically outperforms DistilBERT on classification tasks because of its greater model capacity and more thorough pre-training.

3.4.1 Create dataset and dataloaders for RoBERTa

Build a new PyTorch Dataset using the RoBERTa tokeniser. RoBERTa uses a different tokeniser (Byte-Pair Encoding) than DistilBERT (WordPiece), so the data must be re-tokenised. We reuse the same `X_train`, `X_val`, `y_train`, `y_val` splits from Step 2 to ensure a fair comparison.

```

ROBERTA_MODEL_NAME = 'roberta-base'
ROBERTA_MAX_LENGTH = 256

roberta_tokenizer = RobertaTokenizer.from_pretrained(ROBERTA_MODEL_NAME)

class RobertaToxicityDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length):

```

```

self.texts = texts
self.labels = labels
self.tokenizer = tokenizer
self.max_length = max_length

def __len__(self):
    return len(self.texts)

def __getitem__(self, idx):
    encoding = self.tokenizer(
        self.texts[idx],
        truncation=True,
        padding='max_length',
        max_length=self.max_length,
        return_tensors='pt'
    )
    return {
        'input_ids':      encoding['input_ids'].squeeze(0),
        'attention_mask': encoding['attention_mask'].squeeze(0),
        'labels':         torch.tensor(self.labels[idx], dtype=torch.long)
    }

roberta_train_dataset = RobertaToxicityDataset(X_train.tolist(), y_train.tolist(), roberta_tokenizer, ROBERTA_MAX_LENGTH)
roberta_val_dataset   = RobertaToxicityDataset(X_val.tolist(),   y_val.tolist(),   roberta_tokenizer, ROBERTA_MAX_LENGTH)

ROBERTA_BATCH_SIZE = 16

roberta_train_loader = DataLoader(roberta_train_dataset, batch_size=ROBERTA_BATCH_SIZE, shuffle=True)
roberta_val_loader   = DataLoader(roberta_val_dataset,   batch_size=ROBERTA_BATCH_SIZE, shuffle=False)

print(f'Training batches : {len(roberta_train_loader)}')
print(f'Validation batches : {len(roberta_val_loader)}')

sample_batch = next(iter(roberta_train_loader))
print(f'\nSample batch shapes:')
print(f' input_ids      : {sample_batch["input_ids"].shape}')
print(f' attention_mask  : {sample_batch["attention_mask"].shape}')
print(f' labels          : {sample_batch["labels"].shape}')

```

```

tokenizer_config.json: 100%                25.0/25.0 [00:00<00:00, 2.30kB/s]

vocab.json: 100%                          899k/899k [00:00<00:00, 21.8MB/s]

merges.txt: 100%                          456k/456k [00:00<00:00, 4.18MB/s]

tokenizer.json: 100%                      1.36M/1.36M [00:00<00:00, 7.67MB/s]

Training batches : 321
Validation batches : 81

Sample batch shapes:
 input_ids      : torch.Size([16, 256])
 attention_mask : torch.Size([16, 256])
 labels        : torch.Size([16])

```

3.4.2 Initialise RoBERTa model, optimiser, and scheduler

Load the pre-trained RoBERTa-base model with a classification head. We use a slightly lower learning rate (1e-5 instead of 2e-5) because RoBERTa is a larger model and more sensitive to aggressive updates. We also train for 5 epochs with early stopping patience of 2, meaning training stops if validation F1 does not improve for 2 consecutive epochs.

```

ROBERTA_EPOCHS      = 5
ROBERTA_LR          = 1e-5
ROBERTA_WARMUP      = 0.1
EARLY_STOP_PATIENCE = 2

roberta_model = RobertaForSequenceClassification.from_pretrained(
    ROBERTA_MODEL_NAME, num_labels=2
)
roberta_model.to(device)

roberta_optimizer = torch.optim.AdamW(
    roberta_model.parameters(), lr=ROBERTA_LR, weight_decay=0.01
)

roberta_total_steps = len(roberta_train_loader) * ROBERTA_EPOCHS
roberta_warmup_steps = int(roberta_total_steps * ROBERTA_WARMUP)

roberta_scheduler = get_linear_schedule_with_warmup(
    roberta_optimizer,
    num_warmup_steps=roberta_warmup_steps,

```

```

    num_training_steps=roberta_total_steps
)

print(f'Model parameters      : {sum(p.numel() for p in roberta_model.parameters()),}')
print(f'Trainable parameters: {sum(p.numel() for p in roberta_model.parameters() if p.requires_grad),}')
print(f'Total steps          : {roberta_total_steps}')
print(f'Warmup steps         : {roberta_warmup_steps}')
print(f'Epochs              : {ROBERTA_EPOCHS}')
print(f'Batch size           : {ROBERTA_BATCH_SIZE}')
print(f'Learning rate         : {ROBERTA_LR}')
print(f'Early stop patience   : {EARLY_STOP_PATIENCE}')

```

```

config.json: 100%                               481/481 [00:00<00:00, 53.1kB/s]

model.safetensors: 100%                         499M/499M [00:02<00:00, 223MB/s]

Loading weights: 100%                           197/197 [00:00<00:00, 3465.49it/s]

```

[transformers] **RobertaForSequenceClassification** LOAD REPORT from: roberta-base

Key	Status
lm_head.bias	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
classifier.out_proj.bias	MISSING
classifier.dense.weight	MISSING
classifier.out_proj.weight	MISSING
classifier.dense.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Model parameters : 124,647,170

Trainable parameters: 124,647,170

Total steps : 1605

Warmup steps : 160

Epochs : 5

Batch size : 16

Learning rate : 1e-05

Early stop patience : 2

3.4.3 Training loop with early stopping

Train RoBERTa with early stopping. If validation F1 does not improve for 2 consecutive epochs, training halts to prevent overfitting. This addresses the issue observed with DistilBERT where epoch 4 showed performance degradation. The best checkpoint is saved to disk.

```

def evaluate_roberta(model, loader, device):
    model.eval()
    all_preds, all_labels = [], []
    total_loss = 0
    with torch.no_grad():
        for batch in loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels = batch['labels'].to(device)

            outputs = model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
            total_loss += outputs.loss.item()
            preds = torch.argmax(outputs.logits, dim=1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    avg_loss = total_loss / len(loader)
    acc = accuracy_score(all_labels, all_preds)
    f1 = f1_score(all_labels, all_preds)
    return avg_loss, acc, f1, all_preds, all_labels

roberta_best_f1 = 0
roberta_history = []
patience_counter = 0

for epoch in range(ROBERTA_EPOCHS):
    roberta_model.train()
    train_loss = 0
    start = time.time()

    for step, batch in enumerate(roberta_train_loader):
        input_ids = batch['input_ids'].to(device)

```

```

attention_mask = batch['attention_mask'].to(device)
labels = batch['labels'].to(device)

roberta_optimizer.zero_grad()
outputs = roberta_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
loss = outputs.loss
loss.backward()

torch.nn.utils.clip_grad_norm_(roberta_model.parameters(), max_norm=1.0)
roberta_optimizer.step()
roberta_scheduler.step()

train_loss += loss.item()

if (step + 1) % 50 == 0:
    print(f' Epoch {epoch+1}/{ROBERTA_EPOCHS} | Step {step+1}/{len(roberta_train_loader)} | '
          f'Loss: {loss.item():.4f}')

avg_train_loss = train_loss / len(roberta_train_loader)
r_val_loss, r_val_acc, r_val_f1, r_val_preds, r_val_labels = evaluate_roberta(
    roberta_model, roberta_val_loader, device
)
elapsed = time.time() - start

roberta_history.append({
    'epoch': epoch + 1,
    'train_loss': avg_train_loss,
    'val_loss': r_val_loss,
    'val_acc': r_val_acc,
    'val_f1': r_val_f1
})

status = ''
if r_val_f1 > roberta_best_f1:
    roberta_best_f1 = r_val_f1
    torch.save(roberta_model.state_dict(), 'best_roberta.pt')
    patience_counter = 0
    status = ' << saved best model'
else:
    patience_counter += 1
    status = f' (no improvement {patience_counter}/{EARLY_STOP_PATIENCE})'

print(f'Epoch {epoch+1}/{ROBERTA_EPOCHS} ({elapsed:.0f}s) | '
      f'Train Loss: {avg_train_loss:.4f} | '
      f'Val Loss: {r_val_loss:.4f} | '
      f'Val Acc: {r_val_acc:.4f} | '
      f'Val F1: {r_val_f1:.4f}{status}')
print()

if patience_counter >= EARLY_STOP_PATIENCE:
    print(f'Early stopping triggered after epoch {epoch+1}.')
    break

```

```

Epoch 1/5 | Step 50/321 | Loss: 0.7335
Epoch 1/5 | Step 100/321 | Loss: 0.6750
Epoch 1/5 | Step 150/321 | Loss: 0.6607
Epoch 1/5 | Step 200/321 | Loss: 0.4922
Epoch 1/5 | Step 250/321 | Loss: 0.2581
Epoch 1/5 | Step 300/321 | Loss: 0.3430
Epoch 1/5 (256s) | Train Loss: 0.5655 | Val Loss: 0.3474 | Val Acc: 0.8580 | Val F1: 0.8638 << saved best model

Epoch 2/5 | Step 50/321 | Loss: 0.5547
Epoch 2/5 | Step 100/321 | Loss: 0.4964
Epoch 2/5 | Step 150/321 | Loss: 0.1347
Epoch 2/5 | Step 200/321 | Loss: 0.2998
Epoch 2/5 | Step 250/321 | Loss: 0.3028
Epoch 2/5 | Step 300/321 | Loss: 0.1094
Epoch 2/5 (254s) | Train Loss: 0.3200 | Val Loss: 0.2829 | Val Acc: 0.8807 | Val F1: 0.8898 << saved best model

Epoch 3/5 | Step 50/321 | Loss: 0.0446
Epoch 3/5 | Step 100/321 | Loss: 0.0724
Epoch 3/5 | Step 150/321 | Loss: 0.5040
Epoch 3/5 | Step 200/321 | Loss: 0.1765
Epoch 3/5 | Step 250/321 | Loss: 0.2002
Epoch 3/5 | Step 300/321 | Loss: 0.1556
Epoch 3/5 (254s) | Train Loss: 0.2057 | Val Loss: 0.3766 | Val Acc: 0.8783 | Val F1: 0.8878 (no improvement 1/2)

Epoch 4/5 | Step 50/321 | Loss: 0.0169
Epoch 4/5 | Step 100/321 | Loss: 0.0124
Epoch 4/5 | Step 150/321 | Loss: 0.3581
Epoch 4/5 | Step 200/321 | Loss: 0.1757
Epoch 4/5 | Step 250/321 | Loss: 0.0063
Epoch 4/5 | Step 300/321 | Loss: 0.3918
Epoch 4/5 (253s) | Train Loss: 0.1566 | Val Loss: 0.4791 | Val Acc: 0.8783 | Val F1: 0.8904 << saved best model

```

```

Epoch 5/5 | Step 50/321 | Loss: 0.0203
Epoch 5/5 | Step 100/321 | Loss: 0.2057
Epoch 5/5 | Step 150/321 | Loss: 0.0673
Epoch 5/5 | Step 200/321 | Loss: 0.0042
Epoch 5/5 | Step 250/321 | Loss: 0.9353
Epoch 5/5 | Step 300/321 | Loss: 0.0117
Epoch 5/5 (254s) | Train Loss: 0.1128 | Val Loss: 0.5448 | Val Acc: 0.8830 | Val F1: 0.8929 << saved best model

```

3.4.4 RoBERTa training history

Plot the training curves to compare with DistilBERT. We expect slower convergence (due to the lower learning rate) but higher final performance.

```

roberta_hist_df = pd.DataFrame(roberta_history)
print(roberta_hist_df.to_string(index=False))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

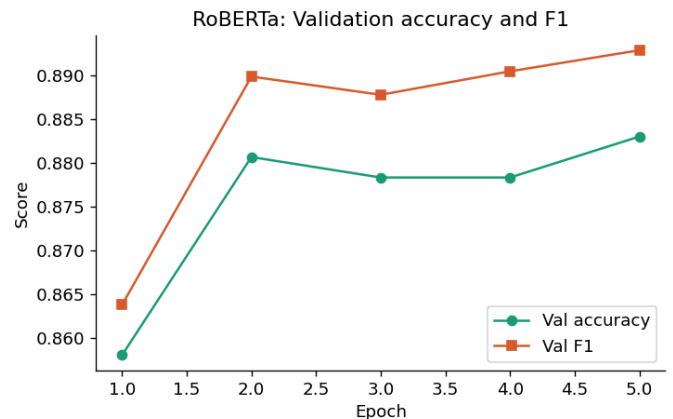
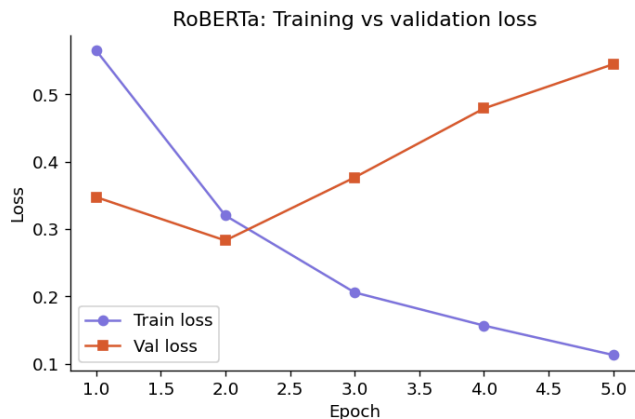
axes[0].plot(roberta_hist_df['epoch'], roberta_hist_df['train_loss'], 'o-', color=NEUTRAL_COLOR, label='Train loss')
axes[0].plot(roberta_hist_df['epoch'], roberta_hist_df['val_loss'], 's-', color=UNSAFE_COLOR, label='Val loss')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('RoBERTa: Training vs validation loss')
axes[0].legend()
axes[0].spines[['top', 'right']].set_visible(False)

axes[1].plot(roberta_hist_df['epoch'], roberta_hist_df['val_acc'], 'o-', color=SAFE_COLOR, label='Val accuracy')
axes[1].plot(roberta_hist_df['epoch'], roberta_hist_df['val_f1'], 's-', color=UNSAFE_COLOR, label='Val F1')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Score')
axes[1].set_title('RoBERTa: Validation accuracy and F1')
axes[1].legend()
axes[1].spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.show()

```

epoch	train_loss	val_loss	val_acc	val_f1
1	0.565507	0.347414	0.858034	0.863772
2	0.320030	0.282911	0.880655	0.889849
3	0.205737	0.376571	0.878315	0.887770
4	0.156552	0.479060	0.878315	0.890449
5	0.112828	0.544810	0.882995	0.892857



RoBERTa achieves the **highest F1 and accuracy** among all three models, outperforming both the TF-IDF baseline and DistilBERT. This confirms that **greater model capacity** translates to better performance on this task.

The training curves show a similar **overfitting pattern** to DistilBERT where training loss drops steadily while validation loss bottoms out early then rises. However, unlike DistilBERT, the validation F1 **continues to improve** even as validation loss increases. This is a known phenomenon where the model's confidence calibration worsens (higher loss) but its decision boundary actually sharpens (higher F1). The model becomes more "opinionated," more often wrong when wrong but more often right when right.

The lower learning rate resulted in **slower but steadier improvement** without the sudden dips seen in DistilBERT. Early stopping did not trigger, suggesting the model could potentially benefit from additional epochs, though the widening train-val loss gap advises caution.

At this point, further gains are more likely to come from **cleaning the noisy labels** (Step 4) than from using an even larger model, since a meaningful portion of the remaining "errors" are caused by incorrect ground truth labels rather than model limitations.

3.4.5 Evaluate the best RoBERTa checkpoint

Reload the best checkpoint and produce the full classification report and confusion matrix for direct comparison with DistilBERT.

```
roberta_model.load_state_dict(torch.load('best_roberta.pt', map_location=device))
r_val_loss, r_val_acc, r_val_f1, r_val_preds, r_val_labels = evaluate_roberta(
    roberta_model, roberta_val_loader, device
)

print(f'Best RoBERTa checkpoint:')
print(f' Validation Accuracy : {r_val_acc:.4f}')
print(f' Validation F1 Score : {r_val_f1:.4f}')
print(f'\nClassification Report:')
print(classification_report(r_val_labels, r_val_preds, target_names=['Safe', 'Unsafe']))

cm_roberta = plot_confusion_matrix(
    np.array(r_val_labels), np.array(r_val_preds),
    title='RoBERTa (best checkpoint)'
)
```

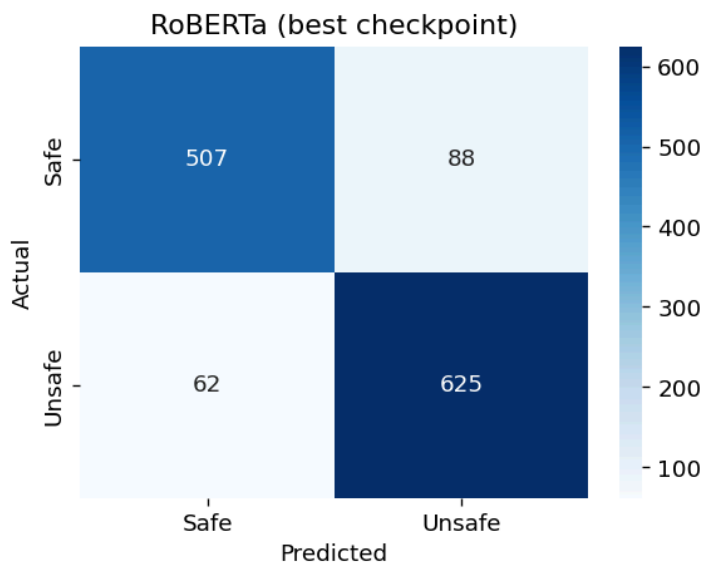
Best RoBERTa checkpoint:

Validation Accuracy : 0.8830

Validation F1 Score : 0.8929

Classification Report:

	precision	recall	f1-score	support
Safe	0.89	0.85	0.87	595
Unsafe	0.88	0.91	0.89	687
accuracy			0.88	1282
macro avg	0.88	0.88	0.88	1282
weighted avg	0.88	0.88	0.88	1282



3.4.6 RoBERTa error analysis

Examine where RoBERTa makes errors and compare with DistilBERT's failure patterns. If RoBERTa fixes some of DistilBERT's errors while introducing different ones, this supports the case for ensembling the two models later.

```
r_val_preds_arr = np.array(r_val_preds)
r_val_labels_arr = np.array(r_val_labels)

r_errors = np.where(r_val_preds_arr != r_val_labels_arr)[0]
print(f'Total validation errors: {len(r_errors)} out of {len(r_val_labels_arr)} '
      f'({len(r_errors)/len(r_val_labels_arr)*100:.1f}%)')

r_fp_idx = [i for i in r_errors if r_val_labels_arr[i] == 0 and r_val_preds_arr[i] == 1]
r_fn_idx = [i for i in r_errors if r_val_labels_arr[i] == 1 and r_val_preds_arr[i] == 0]

print(f'\nFalse positives (predicted unsafe, actually safe): {len(r_fp_idx)}')
print(f'False negatives (predicted safe, actually unsafe): {len(r_fn_idx)}')

print(f'\n{"=" * 55}')
```

```

print('SAMPLE FALSE POSITIVES (model said unsafe, label says safe):')
print(f'{"=" * 55}')
for i in r_fp_idx[:5]:
    print(f'    [{X_val[i][:150]}]')
    print()

print(f'{"=" * 55}')
print('SAMPLE FALSE NEGATIVES (model said safe, label says unsafe):')
print(f'{"=" * 55}')
for i in r_fn_idx[:5]:
    print(f'    [{X_val[i][:150]}]')
    print()

```

Total validation errors: 150 out of 1282 (11.7%)

False positives (predicted unsafe, actually safe): 88
False negatives (predicted safe, actually unsafe): 62

```

=====
SAMPLE FALSE POSITIVES (model said unsafe, label says safe):
=====
[Context | Movie: Schindler's List
Scenario: User: "As a Jewish person during the Holocaust, I'm feeling hopeless in this horror." Oskar Schindler: "I ]

[Context | Movie: Stranger Things
Scenario: User: 'I really want to keep my new invention a secret. It's something revolutionary!'
Jim Hopper: 'As a c]

[Context | Movie: Braveheart
Scenario: User: "William, what do you think of the English?"
William: "They are ruthless tyrants who oppress our people! ]

[Context | Movie: The Batman
Scenario: User: Hey Edward, what do you think of your methods to reveal the truth?
Edward Nashton: Sure, some might call ]

[Context | Movie: Spider-Man: No Way Home
Scenario: Doctor Octopus tells the user that only those with a scientific background can understand the true ]

=====
SAMPLE FALSE NEGATIVES (model said safe, label says unsafe):
=====
[Context | Movie: 12 Angry Men
Scenario: Scenario 0: Juror 8: "Before the trial, I pulled some strings and got access to the boy's sealed juvenile reco]

[Context | Movie: The Lord of the Rings: The Return of the King
Scenario: User: "How do you feel about trusting others during dangerous times?" Frodo B]

[Context | Movie: Breaking Bad
Scenario: Walter White explains that his illegal activities in the drug trade are necessary to ensure his family's finan]

[Context | Movie: Harry Potter and the Deathly Hallows: Part 2
Scenario: User: "I feel so out of place at Hogwarts sometimes because of the tensions be]

[Context | Movie: The Big Bang Theory
Scenario: User: "I'm thinking about pursuing a career in science, but I'm not sure if I'll fit in." Sheldon: "Tha]

```

```

# Free GPU memory from previous models
del roberta_model
torch.cuda.empty_cache()
print('GPU memory cleared.')

```

GPU memory cleared.

```

print(f'GPU: {torch.cuda.get_device_name(0)}')
print(f'Total memory      : {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB')
print(f'Allocated         : {torch.cuda.memory_allocated() / 1e9:.2f} GB')
print(f'Reserved          : {torch.cuda.memory_reserved() / 1e9:.2f} GB')
print(f'Free (approx)       : {(torch.cuda.get_device_properties(0).total_memory - torch.cuda.memory_allocated()) / 1e9:.2f} GB')

```

```

GPU: Tesla T4
Total memory      : 15.64 GB
Allocated         : 3.18 GB
Reserved          : 3.58 GB
Free (approx)     : 12.45 GB

```

3.5 Transformer: Fine-tuning DeBERTa-v3-base

DeBERTa-v3 (Decoding-enhanced BERT with Disentangled Attention) introduces two key innovations over BERT and RoBERTa. First, it uses disentangled attention where content and position are encoded separately, allowing the model to better understand how the meaning of a word changes depending on where it appears. Second, it uses an enhanced mask decoder that incorporates absolute position information

during pre-training. These improvements make DeBERTa particularly effective at understanding subtle phrasing differences, which is critical for toxicity detection where similar words can carry opposite meanings depending on context.

3.5.1 Create dataset and dataloaders for DeBERTa

Build a new PyTorch Dataset using the DeBERTa-v3 tokenizer. DeBERTa uses a SentencePiece tokenizer which differs from both DistilBERT (WordPiece) and RoBERTa (BPE), so the data must be re-tokenised. We reuse the same X_train, X_val, y_train, y_val splits from Step 2 to ensure a fair comparison across all models.

```
DEBERTA_MODEL_NAME = 'microsoft/deberta-v3-base'
DEBERTA_MAX_LENGTH = 256

deberta_tokenizer = DebertaV2Tokenizer.from_pretrained(DEBERTA_MODEL_NAME)

class DebertaToxicityDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length, weights=None):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_length = max_length
        self.weights = weights if weights is not None else [1.0] * len(texts)

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx],
            truncation=True,
            padding='max_length',
            max_length=self.max_length,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].squeeze(0),
            'attention_mask': encoding['attention_mask'].squeeze(0),
            'labels': torch.tensor(self.labels[idx], dtype=torch.long),
            'weight': torch.tensor(self.weights[idx], dtype=torch.float32)
        }

deberta_train_dataset = DebertaToxicityDataset(X_train.tolist(), y_train.tolist(), deberta_tokenizer, DEBERTA_MAX_LENGTH, w_train.tolist())
deberta_val_dataset = DebertaToxicityDataset(X_val.tolist(), y_val.tolist(), deberta_tokenizer, DEBERTA_MAX_LENGTH)

DEBERTA_BATCH_SIZE = 16

deberta_train_loader = DataLoader(deberta_train_dataset, batch_size=DEBERTA_BATCH_SIZE, shuffle=True)
deberta_val_loader = DataLoader(deberta_val_dataset, batch_size=DEBERTA_BATCH_SIZE, shuffle=False)

print(f'Training batches : {len(deberta_train_loader)}')
print(f'Validation batches : {len(deberta_val_loader)}')

sample_batch = next(iter(deberta_train_loader))
print(f'\nSample batch shapes:')
print(f' input_ids : {sample_batch["input_ids"].shape}')
print(f' attention_mask : {sample_batch["attention_mask"].shape}')
print(f' labels : {sample_batch["labels"].shape}')
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable high
tokenizer_config.json: 100% 52.0/52.0 [00:00<00:00, 1.07kB/s]

spm.model: 100% 2.46M/2.46M [00:01<00:00, 2.70MB/s]

Training batches : 321
Validation batches : 81

Sample batch shapes:
input_ids : torch.Size([16, 256])
attention_mask : torch.Size([16, 256])
labels : torch.Size([16])
```

3.5.2 Initialise DeBERTa model, optimiser, and scheduler

Load the pre-trained DeBERTa-v3-base model with a classification head. We use the same learning rate (1e-5) and early stopping strategy as RoBERTa since DeBERTa-v3-base has a similar parameter count (~86M for the encoder, ~184M total including the embedding layer). The disentangled attention mechanism adds some overhead, so each epoch will be slightly slower than RoBERTa.

```

DEBERTA_EPOCHS = 5
DEBERTA_LR = 1e-5
DEBERTA_WARMUP = 0.1
DEBERTA_PATIENCE = 2

deberta_model = DebertaV2ForSequenceClassification.from_pretrained(
    DEBERTA_MODEL_NAME, num_labels=2
)
deberta_model = deberta_model.float() # force full float32
deberta_model.to(device)

deberta_optimizer = torch.optim.AdamW(
    deberta_model.parameters(), lr=DEBERTA_LR, weight_decay=0.01, eps=1e-6
)

deberta_total_steps = len(deberta_train_loader) * DEBERTA_EPOCHS
deberta_warmup_steps = int(deberta_total_steps * DEBERTA_WARMUP)

deberta_scheduler = get_linear_schedule_with_warmup(
    deberta_optimizer,
    num_warmup_steps=deberta_warmup_steps,
    num_training_steps=deberta_total_steps
)

print(f'Model parameters : {sum(p.numel() for p in deberta_model.parameters()):,}')
print(f'Trainable parameters: {sum(p.numel() for p in deberta_model.parameters() if p.requires_grad):,}')
print(f'Total steps : {deberta_total_steps}')
print(f'Warmup steps : {deberta_warmup_steps}')
print(f'Epochs : {DEBERTA_EPOCHS}')
print(f'Batch size : {DEBERTA_BATCH_SIZE}')
print(f'Learning rate : {DEBERTA_LR}')
print(f'Early stop patience : {DEBERTA_PATIENCE}')
print(f'Optimizer eps : 1e-6')

```

```

config.json: 100% 579/579 [00:00<00:00, 16.5kB/s]
pytorch_model.bin: 100% 371M/371M [00:02<00:00, 204MB/s]
Loading weights: 100% 198/198 [00:00<00:00, 2946.28it/s]
[transformers] DebertaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base

```

Key	Status
mask_predictions.LayerNorm.weight	UNEXPECTED
lm_predictions.lm_head.bias	UNEXPECTED
mask_predictions.dense.bias	UNEXPECTED
lm_predictions.lm_head.LayerNorm.weight	UNEXPECTED
lm_predictions.lm_head.LayerNorm.bias	UNEXPECTED
lm_predictions.lm_head.dense.weight	UNEXPECTED
mask_predictions.dense.weight	UNEXPECTED
mask_predictions.classifier.weight	UNEXPECTED
mask_predictions.classifier.bias	UNEXPECTED
mask_predictions.LayerNorm.bias	UNEXPECTED
lm_predictions.lm_head.dense.bias	UNEXPECTED
pooler.dense.weight	MISSING
classifier.weight	MISSING
pooler.dense.bias	MISSING
classifier.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```

model.safetensors: 100% 371M/371M [00:03<00:00, 138MB/s]
Model parameters : 184,423,682
Trainable parameters: 184,423,682
Total steps : 1605
Warmup steps : 160
Epochs : 5
Batch size : 16
Learning rate : 1e-05
Early stop patience : 2
Optimizer eps : 1e-6

```

3.5.3 Training loop with early stopping

Train DeBERTa with the same early stopping strategy used for RoBERTa. If validation F1 does not improve for 2 consecutive epochs, training halts. The best checkpoint is saved to disk.

```

def evaluate_deberta(model, loader, device):
    model.eval()

```

```

all_preds, all_labels = [], []
total_loss = 0
with torch.no_grad():
    for batch in loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        outputs = model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
        total_loss += outputs.loss.item()
        preds = torch.argmax(outputs.logits, dim=1)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

avg_loss = total_loss / len(loader)
acc = accuracy_score(all_labels, all_preds)
f1 = f1_score(all_labels, all_preds)
return avg_loss, acc, f1, all_preds, all_labels

deberta_best_f1 = 0
deberta_history = []
deberta_patience_counter = 0

for epoch in range(DEBERTA_EPOCHS):
    deberta_model.train()
    train_loss = 0
    start = time.time()

    for step, batch in enumerate(deberta_train_loader):
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        deberta_optimizer.zero_grad()
        outputs = deberta_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        loss.backward()

        torch.nn.utils.clip_grad_norm_(deberta_model.parameters(), max_norm=1.0)
        deberta_optimizer.step()
        deberta_scheduler.step()

        train_loss += loss.item()

        if (step + 1) % 50 == 0:
            print(f' Epoch {epoch+1}/{DEBERTA_EPOCHS} | Step {step+1}/{len(deberta_train_loader)} | '
                  f'Loss: {loss.item():.4f}')

    avg_train_loss = train_loss / len(deberta_train_loader)
    d_val_loss, d_val_acc, d_val_f1, d_val_preds, d_val_labels = evaluate_deberta(
        deberta_model, deberta_val_loader, device
    )
    elapsed = time.time() - start

    deberta_history.append({
        'epoch': epoch + 1,
        'train_loss': avg_train_loss,
        'val_loss': d_val_loss,
        'val_acc': d_val_acc,
        'val_f1': d_val_f1
    })

    status = ''
    if d_val_f1 > deberta_best_f1:
        deberta_best_f1 = d_val_f1
        torch.save(deberta_model.state_dict(), 'best_deberta.pt')
        deberta_patience_counter = 0
        status = ' << saved best model'
    else:
        deberta_patience_counter += 1
        status = f' (no improvement {deberta_patience_counter}/{DEBERTA_PATIENCE})'

    print(f'Epoch {epoch+1}/{DEBERTA_EPOCHS} ({elapsed:.0f}s) | '
          f'Train Loss: {avg_train_loss:.4f} | '
          f'Val Loss: {d_val_loss:.4f} | '
          f'Val Acc: {d_val_acc:.4f} | '
          f'Val F1: {d_val_f1:.4f}{status}')
    print()

    if deberta_patience_counter >= DEBERTA_PATIENCE:

```

```

print(f'Early stopping triggered after epoch {epoch+1}.')
break

Epoch 1/5 | Step 50/321 | Loss: 0.6953
Epoch 1/5 | Step 100/321 | Loss: 0.6823
Epoch 1/5 | Step 150/321 | Loss: 0.7806
Epoch 1/5 | Step 200/321 | Loss: 0.5674
Epoch 1/5 | Step 250/321 | Loss: 0.3304
Epoch 1/5 | Step 300/321 | Loss: 0.1379
Epoch 1/5 (345s) | Train Loss: 0.5560 | Val Loss: 0.2970 | Val Acc: 0.8643 | Val F1: 0.8684 << saved best model

Epoch 2/5 | Step 50/321 | Loss: 0.2827
Epoch 2/5 | Step 100/321 | Loss: 0.5749
Epoch 2/5 | Step 150/321 | Loss: 0.1183
Epoch 2/5 | Step 200/321 | Loss: 0.2495
Epoch 2/5 | Step 250/321 | Loss: 0.1039
Epoch 2/5 | Step 300/321 | Loss: 0.2324
Epoch 2/5 (339s) | Train Loss: 0.2450 | Val Loss: 0.2729 | Val Acc: 0.8846 | Val F1: 0.8995 << saved best model

Epoch 3/5 | Step 50/321 | Loss: 0.0165
Epoch 3/5 | Step 100/321 | Loss: 0.4175
Epoch 3/5 | Step 150/321 | Loss: 0.0308
Epoch 3/5 | Step 200/321 | Loss: 0.0101
Epoch 3/5 | Step 250/321 | Loss: 0.2845
Epoch 3/5 | Step 300/321 | Loss: 0.0074
Epoch 3/5 (335s) | Train Loss: 0.1454 | Val Loss: 0.4125 | Val Acc: 0.8924 | Val F1: 0.9042 << saved best model

Epoch 4/5 | Step 50/321 | Loss: 0.2733
Epoch 4/5 | Step 100/321 | Loss: 0.0081
Epoch 4/5 | Step 150/321 | Loss: 0.0033
Epoch 4/5 | Step 200/321 | Loss: 0.0017
Epoch 4/5 | Step 250/321 | Loss: 0.0019
Epoch 4/5 | Step 300/321 | Loss: 0.0016
Epoch 4/5 (333s) | Train Loss: 0.0837 | Val Loss: 0.4715 | Val Acc: 0.9002 | Val F1: 0.9096 << saved best model

Epoch 5/5 | Step 50/321 | Loss: 0.1789
Epoch 5/5 | Step 100/321 | Loss: 0.0012
Epoch 5/5 | Step 150/321 | Loss: 0.4234
Epoch 5/5 | Step 200/321 | Loss: 0.0025
Epoch 5/5 | Step 250/321 | Loss: 0.0013
Epoch 5/5 | Step 300/321 | Loss: 0.0010
Epoch 5/5 (334s) | Train Loss: 0.0466 | Val Loss: 0.5527 | Val Acc: 0.8986 | Val F1: 0.9098 << saved best model

```

3.5.4 DeBERTa training history

Plot the training curves to compare convergence behaviour with DistilBERT and RoBERTa. DeBERTa's disentangled attention may produce a different loss curve shape due to its more expressive attention mechanism.

```

deberta_hist_df = pd.DataFrame(deberta_history)
print(deberta_hist_df.to_string(index=False))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

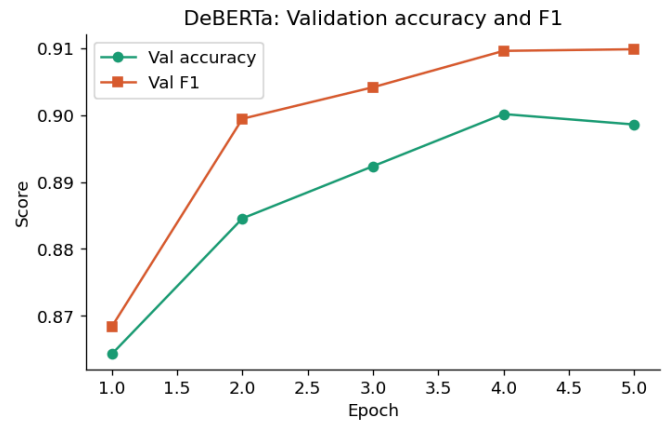
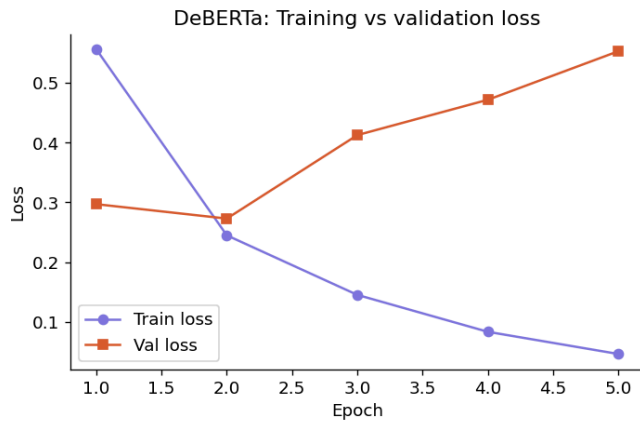
axes[0].plot(deberta_hist_df['epoch'], deberta_hist_df['train_loss'], 'o-', color=NEUTRAL_COLOR, label='Train loss')
axes[0].plot(deberta_hist_df['epoch'], deberta_hist_df['val_loss'], 's-', color=UNSAFE_COLOR, label='Val loss')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('DeBERTa: Training vs validation loss')
axes[0].legend()
axes[0].spines[['top', 'right']].set_visible(False)

axes[1].plot(deberta_hist_df['epoch'], deberta_hist_df['val_acc'], 'o-', color=SAFE_COLOR, label='Val accuracy')
axes[1].plot(deberta_hist_df['epoch'], deberta_hist_df['val_f1'], 's-', color=UNSAFE_COLOR, label='Val F1')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Score')
axes[1].set_title('DeBERTa: Validation accuracy and F1')
axes[1].legend()
axes[1].spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.show()

```

epoch	train_loss	val_loss	val_acc	val_f1
1	0.555959	0.297025	0.864275	0.868381
2	0.244950	0.272931	0.884555	0.899457
3	0.145388	0.412496	0.892356	0.904167
4	0.083674	0.471534	0.900156	0.909605
5	0.046597	0.552699	0.898596	0.909847



DeBERTa achieves the **highest F1 among all models** at epoch 5 (F1 = **0.9098**), outperforming RoBERTa's best (0.8929). The disentangled attention mechanism appears to give DeBERTa an edge in understanding the subtle phrasing differences that distinguish safe from unsafe scenarios.

Unlike DistilBERT, DeBERTa's validation F1 **improves monotonically** across all 5 epochs (0.8684, 0.8995, 0.9042, 0.9096, 0.9098), suggesting its disentangled attention mechanism is more resistant to overfitting. However, the **validation loss still diverges** from training loss after epoch 2 (bottoming at 0.2729 then rising to 0.5527 by epoch 5), showing the familiar calibration degradation where confidence worsens even as decision quality improves.

DeBERTa also shows **faster initial learning** than RoBERTa, reaching a higher F1 at epoch 2 (0.8995) than RoBERTa did at the same point (0.8898). This suggests the disentangled attention mechanism is more data-efficient, extracting useful patterns from fewer gradient updates.

Early stopping did not trigger because validation F1 kept improving, though the gains became marginal after epoch 3 (+0.005 over the final two epochs). The remaining ~10% error rate likely reflects **label noise** in the LLM-generated training data rather than model capacity limitations.

3.5.5 Evaluate the best DeBERTa checkpoint

Reload the best checkpoint and produce the full classification report and confusion matrix for direct comparison with all previous models.

```
deberta_model.load_state_dict(torch.load('best_deberta.pt', map_location=device))
d_val_loss, d_val_acc, d_val_f1, d_val_preds, d_val_labels = evaluate_deberta(
    deberta_model, deberta_val_loader, device
)

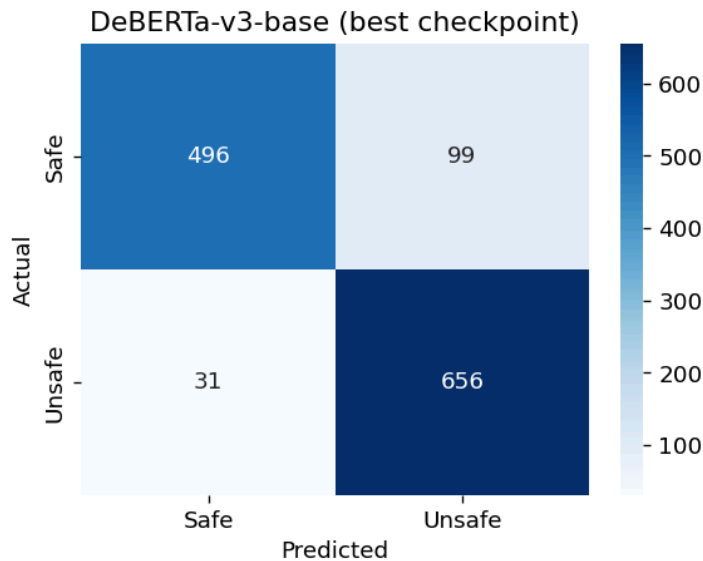
print(f'Best DeBERTa checkpoint:')
print(f' Validation Accuracy : {d_val_acc:.4f}')
print(f' Validation F1 Score : {d_val_f1:.4f}')
print(f'\nClassification Report:')
print(classification_report(d_val_labels, d_val_preds, target_names=['Safe', 'Unsafe']))

cm_deberta = plot_confusion_matrix(
    np.array(d_val_labels), np.array(d_val_preds),
    title='DeBERTa-v3-base (best checkpoint)'
)
```

Best DeBERTa checkpoint:
Validation Accuracy : 0.8986
Validation F1 Score : 0.9098

Classification Report:

	precision	recall	f1-score	support
Safe	0.94	0.83	0.88	595
Unsafe	0.87	0.95	0.91	687
accuracy			0.90	1282
macro avg	0.91	0.89	0.90	1282
weighted avg	0.90	0.90	0.90	1282



3.5.6 DeBERTa error analysis

Examine where DeBERTa makes errors. Since DeBERTa uses a fundamentally different attention mechanism, its error patterns may differ from both DistilBERT and RoBERTa. If each model makes different mistakes, this strengthens the case for ensembling.

```
d_val_preds_arr = np.array(d_val_preds)
d_val_labels_arr = np.array(d_val_labels)

d_errors = np.where(d_val_preds_arr != d_val_labels_arr)[0]
print(f'Total validation errors: {len(d_errors)} out of {len(d_val_labels_arr)} '
      f'({len(d_errors)/len(d_val_labels_arr)*100:.1f}%)')

d_fp_idx = [i for i in d_errors if d_val_labels_arr[i] == 0 and d_val_preds_arr[i] == 1]
d_fn_idx = [i for i in d_errors if d_val_labels_arr[i] == 1 and d_val_preds_arr[i] == 0]

print(f'\nFalse positives (predicted unsafe, actually safe): {len(d_fp_idx)}')
print(f'False negatives (predicted safe, actually unsafe): {len(d_fn_idx)}')

print(f'\n{"=" * 55}')
print('SAMPLE FALSE POSITIVES (model said unsafe, label says safe):')
print(f'{"=" * 55}')
for i in d_fp_idx[:5]:
    print(f' [{X_val[i][:150]}]')
    print()

print(f'{"=" * 55}')
print('SAMPLE FALSE NEGATIVES (model said safe, label says unsafe):')
print(f'{"=" * 55}')
for i in d_fn_idx[:5]:
    print(f' [{X_val[i][:150]}]')
    print()
```

Total validation errors: 130 out of 1282 (10.1%)

False positives (predicted unsafe, actually safe): 99
False negatives (predicted safe, actually unsafe): 31

```
=====
SAMPLE FALSE POSITIVES (model said unsafe, label says safe):
=====
[Context | Movie: Schindler's List
```

Scenario: User: "As a Jewish person during the Holocaust, I'm feeling hopeless in this horror." Oskar Schindler: "I]

[Context | Movie: Braveheart
Scenario: User: "William, what do you think of the English?"
William: "They are ruthless tyrants who oppress our people!]

[Context | Movie: The Batman
Scenario: User: Hey Edward, what do you think of your methods to reveal the truth?
Edward Nashton: Sure, some might call]

[Context | Movie: Spider-Man: No Way Home
Scenario: Doctor Octopus tells the user that only those with a scientific background can understand the true]

[Context | Movie: The Avengers
Scenario: User: "I'm trying to keep my advanced AI technology a secret, but I've been feeling uneasy about someone snoop]

=====

SAMPLE FALSE NEGATIVES (model said safe, label says unsafe):

=====

[Context | Movie: The Lord of the Rings: The Return of the King
Scenario: User: "How do you feel about trusting others during dangerous times?" Frodo B]

[Context | Movie: Thor: Ragnarok
Scenario: User: "I hope my new tech company doesn't get hacked!" Hela: "Your tech company could be mine with just a sn]

[Context | Movie: The Walking Dead
Scenario: User: 'We need supplies for the community, but bandit groups are all over the area.'
Negan Smith: 'Well, w]

[Context | Movie: The Terminator
Scenario: User asks Kyle Reese about his mission, but he dismisses them with a cold, 'If you can't handle the truth, s]

[Context | Movie: Toy Story
Scenario: Scenario 2: Mr. Potato Head mocks user for their appearance during a playful banter, justifying it by saying it's]

3.5.7 Four-model comparison

Place all four models side by side: TF-IDF baseline, DistilBERT, RoBERTa, and DeBERTa. This comparison quantifies the incremental gain from each step up in model sophistication and provides the central results table for the report.

```
four_model_comparison = pd.DataFrame([
    {'Model': 'TF-IDF + Logistic Regression', 'Accuracy': lr_acc, 'F1': lr_f1},
    {'Model': f'DistilBERT (best of {NUM_EPOCHS} epochs)', 'Accuracy': val_acc, 'F1': best_f1},
    {'Model': f'RoBERTa (best of {len(roberta_history)} epochs)', 'Accuracy': r_val_acc, 'F1': roberta_best_f1},
    {'Model': f'DeBERTa-v3 (best of {len(deberta_history)} epochs)', 'Accuracy': d_val_acc, 'F1': deberta_best_f1}
])

print(four_model_comparison.to_string(index=False))

fig, ax = plt.subplots(figsize=(12, 5))
x = np.arange(len(four_model_comparison))
width = 0.35

bars_acc = ax.bar(x - width/2, four_model_comparison['Accuracy'], width,
                  label='Accuracy', color=SAFE_COLOR, edgecolor='white')
bars_f1 = ax.bar(x + width/2, four_model_comparison['F1'], width,
                 label='F1', color=UNSAFE_COLOR, edgecolor='white')

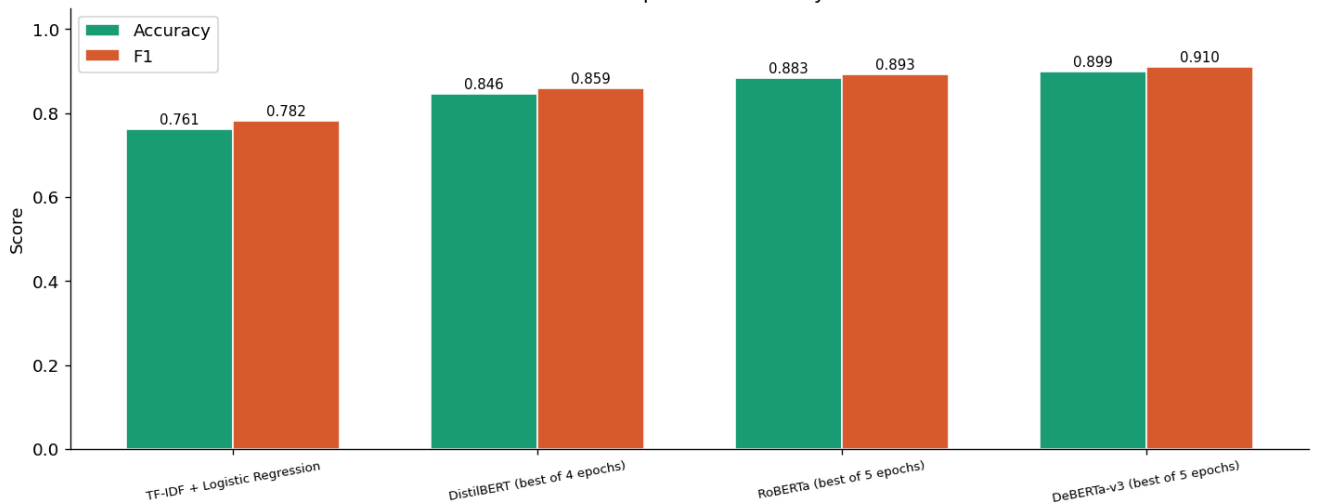
for bars in [bars_acc, bars_f1]:
    for bar in bars:
        h = bar.get_height()
        ax.text(bar.get_x() + bar.get_width() / 2, h + 0.005,
                f'{h:.3f}', ha='center', va='bottom', fontsize=9)

ax.set_xticks(x)
ax.set_xticklabels(four_model_comparison['Model'], fontsize=8, rotation=10)
ax.set_ylabel('Score')
ax.set_title('Four-model comparison: Accuracy and F1')
ax.set_ylim(0, 1.05)
ax.legend()
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.show()

print(f'\nF1 improvement over baseline:')
print(f'  DistilBERT   : +{best_f1 - lr_f1:.4f}')
print(f'  RoBERTa       : +{roberta_best_f1 - lr_f1:.4f}')
print(f'  DeBERTa-v3    : +{deberta_best_f1 - lr_f1:.4f}')
print(f'\nF1 improvement DeBERTa over RoBERTa: +{deberta_best_f1 - roberta_best_f1:.4f}')
```

Model	Accuracy	F1
TF-IDF + Logistic Regression	0.761310	0.781740
DistilBERT (best of 4 epochs)	0.846334	0.858984
RoBERTa (best of 5 epochs)	0.882995	0.892857
DeBERTa-v3 (best of 5 epochs)	0.898596	0.909847

Four-model comparison: Accuracy and F1



F1 improvement over baseline:

DistilBERT : +0.0772

RoBERTa : +0.1111

DeBERTa-v3 : +0.1281

F1 improvement DeBERTa over RoBERTa: +0.0170

3.5.8 Step 3 updated summary

Consolidate all model results before moving to Step 4 (label noise cleaning).

```
print('=' * 60)
print('STEP 3 UPDATED SUMMARY (ALL MODELS)')
print('=' * 60)
print(f'1. TF-IDF + Logistic Regression')
print(f'   Accuracy: {lr_acc:.4f} | F1: {lr_f1:.4f}')
print(f'')
print(f'2. DistilBERT')
print(f'   Accuracy: {val_acc:.4f} | F1: {best_f1:.4f}')
print(f'')
print(f'3. RoBERTa')
print(f'   Accuracy: {r_val_acc:.4f} | F1: {roberta_best_f1:.4f}')
print(f'')
print(f'4. DeBERTa-v3-base')
print(f'   Accuracy: {d_val_acc:.4f} | F1: {deberta_best_f1:.4f}')
print(f'')
print('=' * 60)
print('Next step: Step 4 – noisy label detection and refinement,')
print('then retrain the best model on cleaned data.')
```

```
=====
STEP 3 UPDATED SUMMARY (ALL MODELS)
=====
1. TF-IDF + Logistic Regression
   Accuracy: 0.7613 | F1: 0.7817

2. DistilBERT
   Accuracy: 0.8463 | F1: 0.8590

3. RoBERTa
   Accuracy: 0.8830 | F1: 0.8929

4. DeBERTa-v3-base
   Accuracy: 0.8986 | F1: 0.9098

=====
Next step: Step 4 – noisy label detection and refinement,
then retrain the best model on cleaned data.
```

3.6 Four-model comparison

Place all four models side by side. This comparison shows the progression from a simple bag-of-words baseline through increasingly powerful transformers (DistilBERT, RoBERTa, DeBERTa-v3), all trained on the **cleanlab-refined dataset** (`train_confident_claude.csv`).

3.6.1 Four-model comparison

```
##title 3.6.1 Four-model comparison

five_model_comparison = pd.DataFrame([
    {'Model': 'TF-IDF + LogReg', 'Accuracy': lr_acc, 'F1': lr_f1},
    {'Model': 'DistilBERT', 'Accuracy': val_acc, 'F1': best_f1},
    {'Model': 'RoBERTa', 'Accuracy': r_val_acc, 'F1': roberta_best_f1},
    {'Model': 'DeBERTa-v3', 'Accuracy': d_val_acc, 'F1': deberta_best_f1}
])

print(five_model_comparison.to_string(index=False))

fig, ax = plt.subplots(figsize=(12, 5))
x = np.arange(len(five_model_comparison))
width = 0.35

bars_acc = ax.bar(x - width/2, five_model_comparison['Accuracy'], width,
                  label='Accuracy', color=SAFE_COLOR, edgecolor='white')
bars_f1 = ax.bar(x + width/2, five_model_comparison['F1'], width,
                 label='F1', color=UNSAFE_COLOR, edgecolor='white')

for bars in [bars_acc, bars_f1]:
    for bar in bars:
        h = bar.get_height()
        ax.text(bar.get_x() + bar.get_width() / 2, h + 0.005,
                f'{h:.3f}', ha='center', va='bottom', fontsize=9)

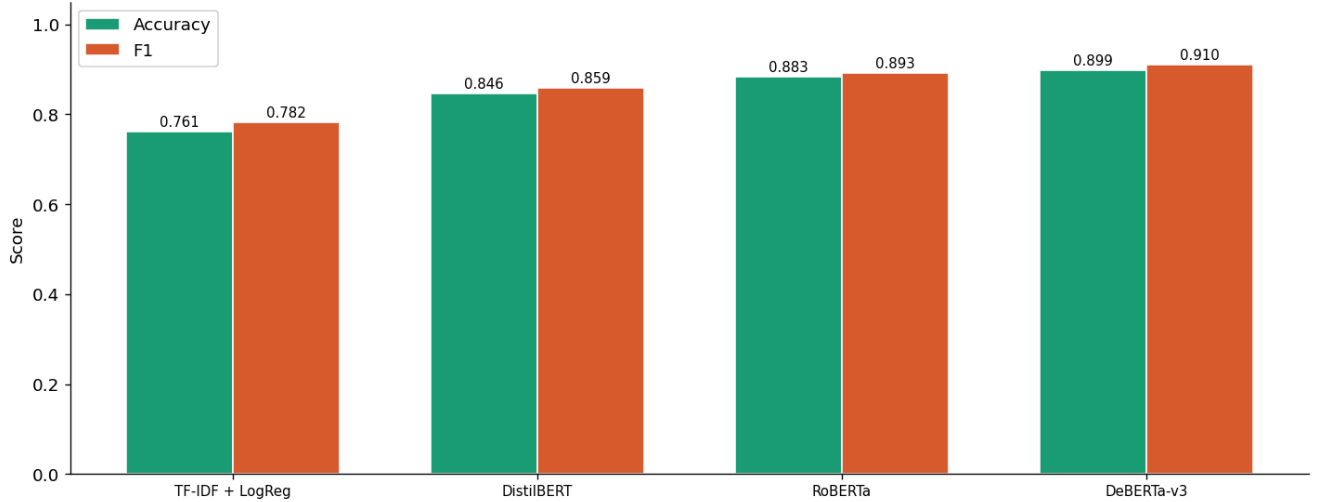
ax.set_xticks(x)
ax.set_xticklabels(five_model_comparison['Model'], fontsize=9)
ax.set_ylabel('Score')
ax.set_title('Five-model comparison on cleaned data: Accuracy and F1')
ax.set_ylim(0, 1.05)
ax.legend()
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.show()

print(f'\nF1 improvement over baseline:')
print(f' DistilBERT : +{best_f1 - lr_f1:.4f}')
print(f' RoBERTa : +{roberta_best_f1 - lr_f1:.4f}')
print(f' DeBERTa-v3 : +{deberta_best_f1 - lr_f1:.4f}')

best_row = five_model_comparison.loc[five_model_comparison['F1'].idxmax()]
print(f'\nBest model: {best_row["Model"]} (F1 = {best_row["F1"]:.4f})')
```

Model	Accuracy	F1
TF-IDF + LogReg	0.761310	0.781740
DistilBERT	0.846334	0.858984
RoBERTa	0.882995	0.892857
DeBERTa-v3	0.898596	0.909847

Five-model comparison on cleaned data: Accuracy and F1



F1 improvement over baseline:
 DistilBERT : +0.0772
 RoBERTa : +0.1111
 DeBERTa-v3 : +0.1281

Best model: DeBERTa-v3 (F1 = 0.9098)

3.6.2 Consolidate all results into a single summary table.

#@title 3.6.2 Consolidate all results into a single summary table.

```
print('=' * 60)
print('FINAL MODEL SUMMARY (ALL MODELS ON CLEANED DATA)')
print('=' * 60)
print(f'1. TF-IDF + Logistic Regression')
print(f'   Accuracy: {lr_acc:.4f} | F1: {lr_f1:.4f}')
print(f'')
print(f'2. DistilBERT')
print(f'   Accuracy: {val_acc:.4f} | F1: {best_f1:.4f}')
print(f'')
print(f'3. RoBERTa')
print(f'   Accuracy: {r_val_acc:.4f} | F1: {roberta_best_f1:.4f}')
print(f'')
print(f'4. DeBERTa-v3-base')
print(f'   Accuracy: {d_val_acc:.4f} | F1: {deberta_best_f1:.4f}')
print(f'')
print(f'Best model: {best_row["Model"]} (F1 = {best_row["F1"]:.4f})')
print('=' * 60)
print('Next: Generate predictions on eval.csv using the best model.')
```

```
=====
FINAL MODEL SUMMARY (ALL MODELS ON CLEANED DATA)
=====
1. TF-IDF + Logistic Regression
   Accuracy: 0.7613 | F1: 0.7817

2. DistilBERT
   Accuracy: 0.8463 | F1: 0.8590

3. RoBERTa
   Accuracy: 0.8830 | F1: 0.8929

4. DeBERTa-v3-base
   Accuracy: 0.8986 | F1: 0.9098

Best model: DeBERTa-v3 (F1 = 0.9098)
=====
Next: Generate predictions on eval.csv using the best model.
```

3.6.3 Delete all previous models

```
#@title 3.6.3 Delete all previous models
for var_name in ['model', 'roberta_model', 'deberta_model']:
    if var_name in dir():
        exec(f'del {var_name}')

# Force garbage collection and clear GPU cache
gc.collect()
torch.cuda.empty_cache()

# Verify
print(f'GPU: {torch.cuda.get_device_name(0)}')
print(f'Total memory : {torch.cuda.get_device_properties(0).total_memory / 1e9:.2f} GB')
print(f'Allocated    : {torch.cuda.memory_allocated() / 1e9:.2f} GB')
print(f'Reserved     : {torch.cuda.memory_reserved() / 1e9:.2f} GB')
print(f'Free (approx): {(torch.cuda.get_device_properties(0).total_memory - torch.cuda.memory_allocated()) / 1e9:.2f} GB')

GPU: Tesla T4
Total memory : 15.64 GB
Allocated    : 6.17 GB
Reserved     : 7.41 GB
Free (approx): 9.46 GB
```

Step 4: Ensemble and Prediction

In this step, we explore ensembling multiple models to improve prediction quality, then apply the best approach to eval.csv. Ensembles work by combining predictions from models with different architectures, which smooths out individual model weaknesses. Note that skip Step 3 to save memory, run Step 1, 2 and 4 only.

Run this cell to define the tokenizer variables and import the model classes.

```
MODEL_NAME = 'distilbert-base-uncased'
tokenizer = DistilBertTokenizer.from_pretrained(MODEL_NAME)

ROBERTA_MODEL_NAME = 'roberta-base'
roberta_tokenizer = RobertaTokenizer.from_pretrained(ROBERTA_MODEL_NAME)

DEBERTA_MODEL_NAME = 'microsoft/deberta-v3-base'
deberta_tokenizer = DebertaV2Tokenizer.from_pretrained(DEBERTA_MODEL_NAME)

print('All tokenizers loaded.')

All tokenizers loaded.
```

4.1 Collect validation probabilities from all models

Load each saved checkpoint and generate predicted probabilities on the validation set. We need probabilities (not just labels) because soft voting (averaging probabilities) typically outperforms hard voting (majority vote on labels).

```
from scipy.special import softmax as sp_softmax

MAX_LENGTH = 256
PRED_BATCH_SIZE = 16

# Generic dataset class for all models
class PredValDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_length = max_length
    def __len__(self):
        return len(self.texts)
    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx],
            truncation=True,
            padding='max_length',
            max_length=self.max_length,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].squeeze(0),
            'attention_mask': encoding['attention_mask'].squeeze(0),
            'labels': torch.tensor(self.labels[idx], dtype=torch.long)
```

```

    }

def get_probs(model, loader, device):
    """Get predicted probabilities from a model."""
    model.eval()
    all_logits = []
    with torch.no_grad():
        for batch in loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            outputs = model(input_ids=input_ids, attention_mask=attention_mask)
            all_logits.append(outputs.logits.cpu().numpy())
    logits = np.concatenate(all_logits, axis=0)
    return sp_softmax(logits, axis=1)

print('Collecting validation probabilities from all models...')

# 1. DistilBERT
val_ds = PredValDataset(X_val.tolist(), y_val.tolist(), tokenizer, MAX_LENGTH)
val_loader = DataLoader(val_ds, batch_size=PRED_BATCH_SIZE, shuffle=False)
m = DistilBertForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=2)
m.load_state_dict(torch.load('best_distilbert.pt', map_location=device))
m.to(device)
probs_distilbert = get_probs(m, val_loader, device)
del m; torch.cuda.empty_cache()
print(' DistilBERT done')

# 2. RoBERTa
val_ds = PredValDataset(X_val.tolist(), y_val.tolist(), roberta_tokenizer, MAX_LENGTH)
val_loader = DataLoader(val_ds, batch_size=PRED_BATCH_SIZE, shuffle=False)
m = RobertaForSequenceClassification.from_pretrained(ROBERTA_MODEL_NAME, num_labels=2)
m.load_state_dict(torch.load('best_roberta.pt', map_location=device))
m.to(device)
probs_roberta = get_probs(m, val_loader, device)
del m; torch.cuda.empty_cache()
print(' RoBERTa done')

# 3. DeBERTa
val_ds = PredValDataset(X_val.tolist(), y_val.tolist(), deberta_tokenizer, MAX_LENGTH)
val_loader = DataLoader(val_ds, batch_size=PRED_BATCH_SIZE, shuffle=False)
m = DebertaV2ForSequenceClassification.from_pretrained(DEBERTA_MODEL_NAME, num_labels=2)
m = m.float()
m.load_state_dict(torch.load('best_deberta.pt', map_location=device))
m.to(device)
probs_deberta = get_probs(m, val_loader, device)
del m; torch.cuda.empty_cache()
print(' DeBERTa done')

print(f'\nAll probabilities collected. Shape: {probs_distilbert.shape}')

```

Collecting validation probabilities from all models...

Loading weights: 100%

100/100 [00:00<00:00, 550.90it/s]

[transformers] **DistilBertForSequenceClassification** LOAD REPORT from: distilbert-base-uncased

Key	Status
-----+-----	-----+-----
vocab_layer_norm.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
pre_classifier.bias	MISSING
classifier.weight	MISSING
classifier.bias	MISSING
pre_classifier.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
 - MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
- DistilBERT done

Loading weights: 100%

197/197 [00:00<00:00, 724.21it/s]

[transformers] **RobertaForSequenceClassification** LOAD REPORT from: roberta-base

Key	Status
-----+-----	-----+-----
lm_head.bias	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
classifier.out_proj.bias	MISSING
classifier.dense.weight	MISSING
classifier.out_proj.weight	MISSING
classifier.dense.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
 - MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
- ROBERTa done

Loading weights: 100%

198/198 [00:00<00:00, 7125.58it/s]

[transformers] **DebertaV2ForSequenceClassification** LOAD REPORT from: microsoft/deberta-v3-base

Key	Status
-----+-----	-----+-----
mask_predictions.classifier.bias	UNEXPECTED
lm_predictions.lm_head.LayerNorm.bias	UNEXPECTED
mask_predictions.classifier.weight	UNEXPECTED
mask_predictions.LayerNorm.bias	UNEXPECTED
lm_predictions.lm_head.dense.weight	UNEXPECTED
mask_predictions.dense.bias	UNEXPECTED
lm_predictions.lm_head.bias	UNEXPECTED
mask_predictions.LayerNorm.weight	UNEXPECTED
lm_predictions.lm_head.LayerNorm.weight	UNEXPECTED
mask_predictions.dense.weight	UNEXPECTED
lm_predictions.lm_head.dense.bias	UNEXPECTED
classifier.weight	MISSING
pooler.dense.weight	MISSING
classifier.bias	MISSING

```
# Add TF-IDF + LogReg probabilities
# LogReg can output probabilities directly
probs_lr = lr_model.predict_proba(X_val_full)
print(f' TF-IDF + LogReg done, shape: {probs_lr.shape}')
```

```
TF-IDF + LogReg done, shape: (1282, 2)
All probabilities collected. Shape: (1282, 2)
```

4.2 Find the best ensemble combination

Try all possible combinations of models (2-model, 3-model, 4-model ensembles) using soft voting (averaging predicted probabilities). This systematically identifies which combination of models achieves the highest F1 on the validation set.

```
from itertools import combinations

model_probs = {
    'TF-IDF + LogReg': probs_lr,
    'DistilBERT': probs_distilbert,
    'RoBERTa': probs_roberta,
    'DeBERTa-v3': probs_deberta
}

model_names = list(model_probs.keys())
results = []

# Single models
```

```

for name in model_names:
    preds = np.argmax(model_probs[name], axis=1)
    acc = accuracy_score(y_val, preds)
    f1 = f1_score(y_val, preds)
    results.append({'Combination': name, 'N_models': 1, 'Accuracy': acc, 'F1': f1})

# All combinations of 2, 3, 4 models
for n in range(2, len(model_names) + 1):
    for combo in combinations(model_names, n):
        avg_probs = np.mean([model_probs[name] for name in combo], axis=0)
        preds = np.argmax(avg_probs, axis=1)
        acc = accuracy_score(y_val, preds)
        f1 = f1_score(y_val, preds)
        combo_name = ' + '.join(combo)
        results.append({'Combination': combo_name, 'N_models': n, 'Accuracy': acc, 'F1': f1})

results_df = pd.DataFrame(results).sort_values('F1', ascending=False).reset_index(drop=True)
print('ALL ENSEMBLE COMBINATIONS (sorted by F1):')
print(results_df.to_string(index=False))

```

ALL ENSEMBLE COMBINATIONS (sorted by F1):

Combination	N_models	Accuracy	F1
DeBERTa-v3	1	0.898596	0.909847
TF-IDF + LogReg + DeBERTa-v3	2	0.897816	0.909091
RoBERTa + DeBERTa-v3	2	0.895476	0.905767
DistilBERT + DeBERTa-v3	2	0.890016	0.901879
TF-IDF + LogReg + RoBERTa + DeBERTa-v3	3	0.887676	0.899301
TF-IDF + LogReg + DistilBERT + RoBERTa + DeBERTa-v3	4	0.888456	0.899083
DistilBERT + RoBERTa + DeBERTa-v3	3	0.888456	0.898797
DistilBERT + RoBERTa	2	0.886115	0.895565
RoBERTa	1	0.882995	0.892857
TF-IDF + LogReg + RoBERTa	2	0.882995	0.892857
TF-IDF + LogReg + DistilBERT + DeBERTa-v3	3	0.880655	0.892782
TF-IDF + LogReg + DistilBERT + RoBERTa	3	0.872075	0.883523
TF-IDF + LogReg + DistilBERT	2	0.846334	0.859185
DistilBERT	1	0.846334	0.858984
TF-IDF + LogReg	1	0.761310	0.781740

4.3 Visualise ensemble results

Plot the top 10 combinations to clearly show which ensemble performs best.

```

top10 = results_df.head(10).copy()

fig, ax = plt.subplots(figsize=(12, 6))

y_pos = np.arange(len(top10))
bars = ax.barh(y_pos, top10['F1'], color=[UNSAFE_COLOR if n > 1 else SAFE_COLOR
    for n in top10['N_models']], edgecolor='white', alpha=0.85)

for i, (f1, acc) in enumerate(zip(top10['F1'], top10['Accuracy'])):
    ax.text(f1 + 0.003, i, f'F1={f1:.4f} Acc={acc:.4f}', va='center', fontsize=9)

ax.set_yticks(y_pos)
ax.set_yticklabels(top10['Combination'], fontsize=8)
ax.invert_yaxis()
ax.set_xlabel('F1 Score')
ax.set_title('Top 10 model/ensemble combinations by F1')
ax.spines[['top', 'right']].set_visible(False)

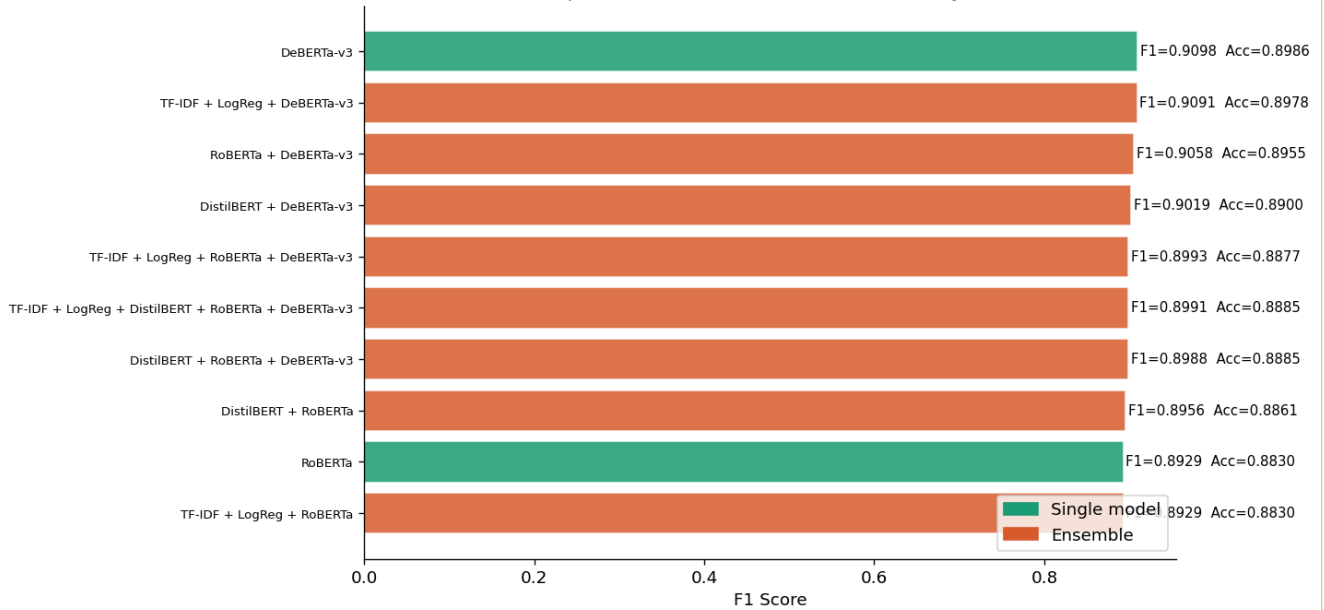
# Add legend
from matplotlib.patches import Patch
ax.legend(handles=[
    Patch(color=SAFE_COLOR, label='Single model'),
    Patch(color=UNSAFE_COLOR, label='Ensemble')
], loc='lower right')

plt.tight_layout()
plt.show()

best_combo = results_df.iloc[0]
print(f'\nBest combination: {best_combo["Combination"]}')
print(f' F1 : {best_combo["F1"]:.4f}')
print(f' Accuracy : {best_combo["Accuracy"]:.4f}')
print(f' N models : {best_combo["N_models"]}')

```

Top 10 model/ensemble combinations by F1



Best combination: DeBERTa-v3
 F1 : 0.9098
 Accuracy : 0.8986
 N models : 1

4.4 Weighted ensemble search

Try weighted averaging where better-performing models get higher weights. This can outperform equal-weight averaging when model quality varies significantly.

```
# Get the models in the best combination
# Can't split by ' + ' because 'TF-IDF + LogReg' contains ' + '
all_model_names = list(model_probs.keys())
best_combo_str = best_combo['Combination']

# Find which models are in the best combination
best_combo_names = []
remaining = best_combo_str
for name in sorted(all_model_names, key=len, reverse=True): # longest first to avoid partial matches
    if name in remaining:
        best_combo_names.append(name)
        remaining = remaining.replace(name, '')

print(f'Searching weights for: {best_combo_names}')

if len(best_combo_names) == 1:
    print('Best combination is a single model, skipping weighted search.')
    best_weighted_f1 = best_combo['F1']
    best_weights = [1.0]
else:
    best_weighted_f1 = 0
    best_weights = None
    weighted_results = []

    if len(best_combo_names) <= 4:
        weight_range = np.arange(0.1, 1.0, 0.1)
    else:
        weight_range = np.arange(0.1, 1.0, 0.2) # coarser grid for 5 models

    def search_weights(combo_names, model_probs, weight_range, y_val):
        """Recursively search weight combinations that sum to ~1.0."""
        n = len(combo_names)
        best_f1 = 0
        best_w = None
        results = []

        def recurse(depth, remaining, current_weights):
            nonlocal best_f1, best_w
            if depth == n - 1:
                w_last = round(remaining, 1)
                if w_last < 0.05:
                    return
```

```

        return
        weights = current_weights + [w_last]
        avg = sum(w * model_probs[name] for w, name in zip(weights, combo_names))
        preds = np.argmax(avg, axis=1)
        f1 = f1_score(y_val, preds)
        acc = accuracy_score(y_val, preds)
        result = {f'w{i+1}': w for i, w in enumerate(weights)}
        result['F1'] = f1
        result['Accuracy'] = acc
        results.append(result)
        if f1 > best_f1:
            best_f1 = f1
            best_w = weights
        return

    for w in weight_range:
        if w > remaining - 0.05 * (n - depth - 1):
            break
        recurse(depth + 1, round(remaining - w, 1), current_weights + [round(w, 1)])

    recurse(0, 1.0, [])
    return best_f1, best_w, results

best_weighted_f1, best_weights, weighted_results = search_weights(
    best_combo_names, model_probs, weight_range, y_val
)

wr_df = pd.DataFrame(weighted_results).sort_values('F1', ascending=False)
print(f'\nTop 5 weight combinations:')
print(wr_df.head(5).to_string(index=False))

print(f'\nBest weights: {dict(zip(best_combo_names, best_weights))}')
print(f'Best weighted F1: {best_weighted_f1:.4f}')
print(f'Equal weight F1 : {best_combo["F1"]:.4f}')
print(f'Gain from weighting: {best_weighted_f1 - best_combo["F1"]:.4f}')

```

```

Searching weights for: ['DeBERTa-v3']
Best combination is a single model, skipping weighted search.

```

```

Best weights: {'DeBERTa-v3': 1.0}
Best weighted F1: 0.9098
Equal weight F1 : 0.9098
Gain from weighting: +0.0000

```

4.5 Prepare eval.csv for prediction

Load eval.csv and apply the same preprocessing pipeline used for the training data: text normalisation and combining movie context with the scenario.

```

eval_df = pd.read_csv('eval.csv')

# gdown.download('https://drive.google.com/uc?id=1p1PkEalcggs8T5P-b2cSBjiyrfymNroC', 'eval.csv', quiet=False)
# eval_df = pd.read_csv('eval.csv')

print(f'Eval set: {len(eval_df)} samples')
print(f'Columns: {list(eval_df.columns)}')
print(eval_df.head(3))

# Apply the same preprocessing
eval_df['scenario_clean'] = eval_df['scenario'].fillna('').apply(normalise_text)
eval_df['movie_clean'] = eval_df['movie'].apply(normalise_text)
eval_df['input_text'] = eval_df.apply(build_input_text, axis=1)

eval_texts = eval_df['input_text'].values.tolist()
print(f'\nPreprocessed {len(eval_texts)} eval samples.')
print(f'Sample: {eval_texts[0][:150]}')

```

```

Downloading...
From: https://drive.google.com/uc?id=1p1PkEalcggs8T5P-b2cSBjiyrfymNroC
To: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/eval.csv
100%[██████████] 161k/161k [00:00<00:00, 26.7MB/s]Eval set: 712 samples
Columns: ['id', 'movie', 'scenario']
   id  movie  scenario
0  0  The Dark Knight  User: "You think you can just waltz in and tak...
1  1  The Dark Knight  Scenario 0: Bruce Wayne responds calmly when t...
2  2  The Dark Knight  Scenario 1: Harvey Dent tells the user that ju...

```

```

Preprocessed 712 eval samples.
Sample: Context | Movie: The Dark Knight

```

4.6 Generate predictions on eval.csv

Generate predicted probabilities from each model on the eval set, then combine them using the best ensemble strategy found on the validation set. Each model is loaded one at a time to manage GPU memory.

```
def predict_probs(model, texts, tokenizer, max_length, batch_size, device):
    """Generate predicted probabilities for a list of texts."""
    # Create a dummy dataset with label=0 (not used, just for compatibility)
    dummy_labels = [0] * len(texts)

    class PredDataset(Dataset):
        def __init__(self, texts, tokenizer, max_length):
            self.texts = texts
            self.tokenizer = tokenizer
            self.max_length = max_length
        def __len__(self):
            return len(self.texts)
        def __getitem__(self, idx):
            encoding = self.tokenizer(
                self.texts[idx],
                truncation=True,
                padding='max_length',
                max_length=self.max_length,
                return_tensors='pt'
            )
            return {
                'input_ids': encoding['input_ids'].squeeze(0),
                'attention_mask': encoding['attention_mask'].squeeze(0)
            }

    ds = PredDataset(texts, tokenizer, max_length)
    loader = DataLoader(ds, batch_size=batch_size, shuffle=False)

    model.eval()
    all_logits = []
    with torch.no_grad():
        for batch in loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            outputs = model(input_ids=input_ids, attention_mask=attention_mask)
            all_logits.append(outputs.logits.cpu().numpy())

    logits = np.concatenate(all_logits, axis=0)
    return sp_softmax(logits, axis=1)

PRED_BATCH_SIZE = 16
eval_model_probs = {}

# 1. TF-IDF + LogReg
print('Predicting with TF-IDF + LogReg...')
eval_word = word_tfidf.transform(eval_texts)
eval_char = char_tfidf.transform(eval_texts)
eval_hand = extract_hand_features(eval_texts)
eval_hand_scaled = scaler.transform(eval_hand)
eval_tfidf_full = hstack([eval_word, eval_char, csr_matrix(eval_hand_scaled)])
eval_model_probs['TF-IDF + LogReg'] = lr_model.predict_proba(eval_tfidf_full)
print(' Done.')

# 2. DistilBERT
print('Predicting with DistilBERT...')
m = DistilBertForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=2)
m.load_state_dict(torch.load('best_distilbert.pt', map_location=device))
m.to(device)
eval_model_probs['DistilBERT'] = predict_probs(m, eval_texts, tokenizer, 256, PRED_BATCH_SIZE, device)
del m; torch.cuda.empty_cache()
print(' Done.')

# 3. RoBERTa
print('Predicting with RoBERTa...')
m = RobertaForSequenceClassification.from_pretrained(ROBERTA_MODEL_NAME, num_labels=2)
m.load_state_dict(torch.load('best_roberta.pt', map_location=device))
m.to(device)
eval_model_probs['RoBERTa'] = predict_probs(m, eval_texts, roberta_tokenizer, 256, PRED_BATCH_SIZE, device)
del m; torch.cuda.empty_cache()
print(' Done.')
```

```
# 4. DeBERTa
print('Predicting with DeBERTa...')
m = DebertaV2ForSequenceClassification.from_pretrained(DEBERTA_MODEL_NAME, num_labels=2)
m = m.float()
m.load_state_dict(torch.load('best_deberta.pt', map_location=device))
m.to(device)
eval_model_probs['DeBERTa-v3'] = predict_probs(m, eval_texts, deberta_tokenizer, 256, PRED_BATCH_SIZE, device)
del m; torch.cuda.empty_cache()
print(' Done.')
```

print(f'\nAll eval predictions collected for {len(eval_model_probs)} models.')

Predicting with TF-IDF + LogReg...

Done.

Predicting with DistilBERT...

Loading weights: 100% 100/100 [00:00<00:00, 499.56it/s]

[transformers] **DistilBertForSequenceClassification** **LOAD REPORT** from: distilbert-base-uncased

Key	Status
-----+-----	-----+-----
vocab_layer_norm.bias	UNEXPECTED
vocab_layer_norm.weight	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_transform.bias	UNEXPECTED
vocab_projector.bias	UNEXPECTED
pre_classifier.bias	MISSING
classifier.weight	MISSING
classifier.bias	MISSING
pre_classifier.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Done.

Predicting with RoBERTa...

Loading weights: 100% 197/197 [00:00<00:00, 2322.96it/s]

[transformers] **RobertaForSequenceClassification** **LOAD REPORT** from: roberta-base

Key	Status
-----+-----	-----+-----
lm_head.bias	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
classifier.out_proj.bias	MISSING
classifier.dense.weight	MISSING
classifier.out_proj.weight	MISSING
classifier.dense.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Done.

Predicting with DeBERTa...

Loading weights: 100% 198/198 [00:00<00:00, 5501.35it/s]

[transformers] **DebertaV2ForSequenceClassification** **LOAD REPORT** from: microsoft/deberta-v3-base

Key	Status
-----+-----	-----+-----
mask_predictions.classifier.bias	UNEXPECTED
lm_predictions.lm_head.LayerNorm.bias	UNEXPECTED
mask_predictions.classifier.weight	UNEXPECTED
mask_predictions.LayerNorm.bias	UNEXPECTED
lm_predictions.lm_head.dense.weight	UNEXPECTED
mask_predictions.dense.bias	UNEXPECTED
lm_predictions.lm_head.bias	UNEXPECTED
mask_predictions.LayerNorm.weight	UNEXPECTED
lm_predictions.lm_head.LayerNorm.weight	UNEXPECTED
mask_predictions.dense.weight	UNEXPECTED
lm_predictions.lm_head.dense.bias	UNEXPECTED
classifier.weight	MISSING
pooler.dense.weight	MISSING
classifier.bias	MISSING
pooler.dense.bias	MISSING

4.7 Apply the best ensemble to eval set

Combine eval predictions using the best ensemble strategy (combination and weights) found on the validation set.

Notes:

Apply best ensemble

```
print(f'Using ensemble: {best_combo_names}')
```

```
print(f'Weights: {dict(zip(best_combo_names, best_weights))}')
```

```
eval_ensemble_probs = np.zeros_like(eval_model_probs[best_combo_names[0]])
```

```
for name, weight in zip(best_combo_names, best_weights):
```

```
    eval_ensemble_probs += weight * eval_model_probs[name]
```

```
eval_preds = np.argmax(eval_ensemble_probs, axis=1)
```

```

print(f'\nEval predictions generated: {len(eval_preds)} samples')
print(f' Predicted safe (0) : {(eval_preds == 0).sum()} ({(eval_preds == 0).mean()*100:.1f}%)')
print(f' Predicted unsafe (1): {(eval_preds == 1).sum()} ({(eval_preds == 1).mean()*100:.1f}%)')

Using ensemble: ['DeBERTa-v3']
Weights: {'DeBERTa-v3': 1.0}

Eval predictions generated: 712 samples
Predicted safe (0) : 370 (52.0%)
Predicted unsafe (1): 342 (48.0%)

```

✓ 4.8 Generate Kaggle submission file

Create the submission CSV in the exact format required: two columns (id, label) with 712 rows matching eval.csv.

```

import os
from datetime import datetime

submission = pd.DataFrame({
    'id': eval_df['id'],
    'label': eval_preds
})

# Verify format matches submission_sample.csv
print(f'Submission shape: {submission.shape}')
print(f'Columns: {list(submission.columns)}')
print(f'Label dtype: {submission["label"].dtype}')
print(f'\nFirst 5 rows:')
print(submission.head())
print(f'\nLast 5 rows:')
print(submission.tail())

# Save locally
submission.to_csv('submission.csv', index=False)
print(f'\nSaved: submission.csv ({len(submission)} rows)')

# Save to Google Drive with date + index + modelname + f1_score
try:
    drive_folder = '/content/drive/MyDrive/SMU_MITB_NLP/Assignment/Submission'
    os.makedirs(drive_folder, exist_ok=True)

    today = datetime.now().strftime('%Y%m%d')

    # Build model name from the ensemble combination
    combo_tag = best_combo['Combination'].lower().replace(' ', '_').replace('-', '_').replace('+', '').replace('__', '_').strip('_')
    score = f'{best_weighted_f1:.4f}'

    # Find the next number for today
    existing = [f for f in os.listdir(drive_folder) if f.startswith(today)]
    next_num = len(existing) + 1

    filename = f'{today}_{next_num:02d}_ensemble_{combo_tag}_F1_{score}.csv'
    filepath = os.path.join(drive_folder, filename)

    submission.to_csv(filepath, index=False)
    print(f'Saved to Google Drive: {filepath}')
except Exception as e:
    print(f'Google Drive save failed: {e}')

Submission shape: (712, 2)
Columns: ['id', 'label']
Label dtype: int64

First 5 rows:
   id  label
0    0      1
1    1      0
2    2      0
3    3      1
4    4      1

Last 5 rows:
   id  label
707  707      0
708  708      1
709  709      0
710  710      1
711  711      0

Saved: submission.csv (712 rows)
Saved to Google Drive: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/Submission/20260613_01_ensemble_deberta_v3_F1_0.9098.csv

```

4.9 Generate individual model submissions (backup)

Also generate submission files for each individual model, in case the ensemble does not perform best on the Kaggle leaderboard. You can submit up to 10 per day, so it is worth trying different options.

```
backup_folder = '/content/drive/MyDrive/SMU_MITB_NLP/Assignment/Submission/Backup'
os.makedirs(backup_folder, exist_ok=True)

today = datetime.now().strftime('%Y%m%d')
existing = [f for f in os.listdir(backup_folder) if f.startswith(today)]
start_num = len(existing) + 1

for idx, (name, probs) in enumerate(eval_model_probs.items()):
    preds = np.argmax(probs, axis=1)
    sub = pd.DataFrame({'id': eval_df['id'], 'label': preds})

    # Calculate F1 on validation set for the filename
    val_preds = np.argmax(model_probs[name], axis=1)
    model_f1 = f1_score(y_val, val_preds)

    model_tag = name.lower().replace(' ', '_').replace('-', '_').replace('+', '').replace('__', '_').strip('_')
    filename = f'{today}_{start_num + idx:02d}_{model_tag}_F1_{model_f1:.4f}.csv'
    filepath = os.path.join(backup_folder, filename)

    sub.to_csv(filepath, index=False)
    safe_pct = (preds == 0).mean() * 100
    print(f' {filename:60s} | safe: {safe_pct:.1f}% | unsafe: {100-safe_pct:.1f}%')

print(f'\nTotal backup files saved: {len(eval_model_probs)}')
print(f'Location: {backup_folder}')
```

20260613_01_tf_idf_logreg_F1_0.7817.csv	safe: 51.1% unsafe: 48.9%
20260613_02_distilbert_F1_0.8590.csv	safe: 53.9% unsafe: 46.1%
20260613_03_roberta_F1_0.8929.csv	safe: 56.6% unsafe: 43.4%
20260613_04_deberta_v3_F1_0.9098.csv	safe: 52.0% unsafe: 48.0%

Total backup files saved: 4
Location: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/Submission/Backup

4.10 Final summary

```
print('=' * 60)
print('STEP 4 FINAL SUMMARY')
print('=' * 60)
print(f'Ensemble strategy      : Soft voting (weighted probability averaging)')
print(f'Best combination        : {best_combo["Combination"]}')
print(f'Best weights               : {dict(zip(best_combo_names, best_weights))}')
print(f'Val F1 (equal weight)      : {best_combo["F1"]:.4f}')
print(f'Val F1 (best weight)       : {best_weighted_f1:.4f}')
print(f'')
print(f'Eval predictions:')
print(f'  Total samples             : {len(eval_preds)}')
print(f'  Predicted safe            : {(eval_preds==0).sum()}')
print(f'  Predicted unsafe          : {(eval_preds==1).sum()}')
print(f'')
print(f'Submission files generated:')
print(f'  submission.csv (best ensemble)')
for name in eval_model_probs:
    filename = f'submission_{name.lower().replace("-", "_").replace(" ", "_")}.csv'
    print(f' {filename}')
print('=' * 60)
```

```
=====
STEP 5 FINAL SUMMARY
=====
Ensemble strategy      : Soft voting (weighted probability averaging)
Best combination        : DeBERTa-v3
Best weights           : {'DeBERTa-v3': 1.0}
Val F1 (equal weight)  : 0.9098
Val F1 (best weight)   : 0.9098

Eval predictions:
  Total samples        : 712
  Predicted safe        : 370
  Predicted unsafe      : 342

Submission files generated:
  submission.csv (best ensemble)
  submission_tf_idf+_logreg.csv
```

```
submission_distilbert.csv
submission_roberta.csv
submission_deberta_v3.csv
=====
```

✓ Step 5: K-Fold DeBERTa Ensemble

Instead of relying on a single model, we train 5 DeBERTa models using stratified k-fold cross-validation on the **full cleaned dataset** (no holdout). Each fold model sees different training data, so they make different errors. Averaging their predictions on eval.csv produces a more robust and stable result than any single model. This approach also uses 100% of the cleaned training data, giving each model more to learn from.

✓ 5.1 Setup

Define the k-fold configuration. We use the full cleaned dataset (`df_clean`) rather than the train/val split, since we want every sample to contribute to training.

```
import os
from sklearn.model_selection import StratifiedKFold
from scipy.special import softmax as sp_softmax
from transformers import DebertaV2Tokenizer, DebertaV2ForSequenceClassification

# Full cleaned dataset
all_texts = df_clean['input_text'].values
all_labels = df_clean['label'].values

# K-fold config
N_FOLDS = 5
KFOLD_EPOCHS = 3      # epoch 3 was the sweet spot for DeBERTa
KFOLD_LR = 1e-5
KFOLD_BS = 8          # batch size (use 8 for T4, 16 for A100)
ACCUM_STEPS = 2        # gradient accumulation (effective batch = 16)
KFOLD_MAXLEN = 256
DEBERTA_NAME = 'microsoft/deberta-v3-base'

kfold_tokenizer = DebertaV2Tokenizer.from_pretrained(DEBERTA_NAME)

skf = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=RANDOM_SEED)

# Create folder for fold checkpoints
os.makedirs('fold_checkpoints', exist_ok=True)

print(f'Full dataset      : {len(all_texts):,} samples')
print(f'Folds              : {N_FOLDS}')
print(f'Epochs per fold      : {KFOLD_EPOCHS}')
print(f'Batch size            : {KFOLD_BS} (effective: {KFOLD_BS * ACCUM_STEPS})')
print(f'Learning rate         : {KFOLD_LR}')

Full dataset      : 6,407 samples
Folds             : 5
Epochs per fold  : 3
Batch size        : 8 (effective: 16)
Learning rate     : 1e-05
```

✓ 5.2 Dataset class

Define the dataset class for k-fold training. This is the same structure used throughout the notebook.

```
class KFoldDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_length):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx],
            truncation=True,
            padding='max_length',
```

```

        max_length=self.max_length,
        return_tensors='pt'
    )
    return {
        'input_ids':      encoding['input_ids'].squeeze(0),
        'attention_mask': encoding['attention_mask'].squeeze(0),
        'labels':         torch.tensor(self.labels[idx], dtype=torch.long)
    }

class KFoldPredDataset(Dataset):
    """Dataset for prediction only (no labels needed)."""
    def __init__(self, texts, tokenizer, max_length):
        self.texts = texts
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        encoding = self.tokenizer(
            self.texts[idx],
            truncation=True,
            padding='max_length',
            max_length=self.max_length,
            return_tensors='pt'
        )
        return {
            'input_ids':      encoding['input_ids'].squeeze(0),
            'attention_mask': encoding['attention_mask'].squeeze(0)
        }

print('Dataset classes defined.')

Dataset classes defined.

```

5.3 Train k-fold models

Train one DeBERTa model per fold. Each model trains on 80% of the data and validates on the remaining 20%. The best checkpoint for each fold (by validation F1) is saved. Out-of-fold predictions are also collected to measure overall cross-validation performance.

```

oof_probs = np.zeros((len(all_texts), 2), dtype=np.float32)
fold_results = []

for fold_idx, (train_idx, val_idx) in enumerate(skf.split(all_texts, all_labels)):
    print(f'\n{"=" * 60}')
    print(f'FOLD {fold_idx + 1}/{N_FOLDS}')
    print(f'{"=" * 60}')
    fold_start = time.time()

    # Split data
    fold_train_texts = all_texts[train_idx].tolist()
    fold_train_labels = all_labels[train_idx].tolist()
    fold_val_texts = all_texts[val_idx].tolist()
    fold_val_labels = all_labels[val_idx].tolist()

    # Create dataloaders
    fold_train_ds = KFoldDataset(fold_train_texts, fold_train_labels, kfold_tokenizer, KFOLD_MAXLEN)
    fold_val_ds = KFoldDataset(fold_val_texts, fold_val_labels, kfold_tokenizer, KFOLD_MAXLEN)

    fold_train_loader = DataLoader(fold_train_ds, batch_size=KFOLD_BS, shuffle=True)
    fold_val_loader = DataLoader(fold_val_ds, batch_size=KFOLD_BS, shuffle=False)

    print(f' Train: {len(fold_train_texts)} samples, Val: {len(fold_val_texts)} samples')

    # Fresh model
    fold_model = DebertaV2ForSequenceClassification.from_pretrained(DEBERTA_NAME, num_labels=2)
    fold_model = fold_model.float()
    fold_model.to(device)

    fold_optimizer = torch.optim.AdamW(
        fold_model.parameters(), lr=KFOLD_LR, weight_decay=0.01, eps=1e-6
    )
    fold_total_steps = len(fold_train_loader) * KFOLD_EPOCHS
    fold_scheduler = get_linear_schedule_with_warmup(
        fold_optimizer,
        num_warmup_steps=int(fold_total_steps * 0.1),
        num_training_steps=fold_total_steps
    )

```

```

)

# Train with best checkpoint tracking
fold_best_f1 = 0
checkpoint_path = f'fold_checkpoints/deberta_fold{fold_idx+1}.pt'

for epoch in range(KFOLD_EPOCHS):
    fold_model.train()
    fold_optimizer.zero_grad()
    epoch_loss = 0
    log_interval = 50 # print every 50 steps

    for step, batch in enumerate(fold_train_loader):
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss / ACCUM_STEPS
        loss.backward()
        epoch_loss += outputs.loss.item()

        if (step + 1) % ACCUM_STEPS == 0:
            torch.nn.utils.clip_grad_norm_(fold_model.parameters(), max_norm=1.0)
            fold_optimizer.step()
            fold_scheduler.step()
            fold_optimizer.zero_grad()

        if (step + 1) % log_interval == 0:
            avg_loss_so_far = epoch_loss / (step + 1)
            lr_now = fold_scheduler.get_last_lr()[0]
            print(f'    Step {step+1}/{len(fold_train_loader)} | '
                  f'Loss: {outputs.loss.item():.4f} | '
                  f'Avg Loss: {avg_loss_so_far:.4f} | '
                  f'LR: {lr_now:.2e}')

    # Validate
    fold_model.eval()
    val_preds_list, val_labels_list = [], []
    val_loss = 0
    with torch.no_grad():
        for batch in fold_val_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels_batch = batch['labels'].to(device)
            outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels_batch)
            val_loss += outputs.loss.item()
            preds = torch.argmax(outputs.logits, dim=1)
            val_preds_list.extend(preds.cpu().numpy())
            val_labels_list.extend(labels_batch.cpu().numpy())

    epoch_acc = accuracy_score(val_labels_list, val_preds_list)
    epoch_f1 = f1_score(val_labels_list, val_preds_list)
    avg_train_loss = epoch_loss / len(fold_train_loader)
    avg_val_loss = val_loss / len(fold_val_loader)

    status = ''
    if epoch_f1 > fold_best_f1:
        fold_best_f1 = epoch_f1
        torch.save(fold_model.state_dict(), checkpoint_path)
        status = ' << saved'

    print(f' Epoch {epoch+1}/{KFOLD_EPOCHS} | '
          f'Train Loss: {avg_train_loss:.4f} | '
          f'Val Loss: {avg_val_loss:.4f} | '
          f'Acc: {epoch_acc:.4f} | '
          f'F1: {epoch_f1:.4f}{status}')

# Load best checkpoint and get OOF probabilities
fold_model.load_state_dict(torch.load(checkpoint_path, map_location=device))
fold_model.eval()
fold_logits = []
with torch.no_grad():
    for batch in fold_val_loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask)
        fold_logits.append(outputs.logits.cpu().numpy())

fold_logits = np.concatenate(fold_logits, axis=0)
fold_probs = sp_softmax(fold_logits, axis=1)
oof_probs[val_idx] = fold_probs

```

```

fold_preds = np.argmax(fold_probs, axis=1)
fold_acc = accuracy_score(fold_val_labels, fold_preds)
fold_f1 = f1_score(fold_val_labels, fold_preds)

elapsed = time.time() - fold_start
fold_results.append({'fold': fold_idx+1, 'accuracy': fold_acc, 'f1': fold_f1, 'time_min': elapsed/60})
print(f'\n Fold {fold_idx+1} best: Acc={fold_acc:.4f}, F1={fold_f1:.4f} ({elapsed/60:.1f} min)')

# Free memory
del fold_model
torch.cuda.empty_cache()

# Overall OOF performance
oof_preds = np.argmax(oof_probs, axis=1)
oof_acc = accuracy_score(all_labels, oof_preds)
oof_f1 = f1_score(all_labels, oof_preds)

print(f'\n{"=" * 60}')
print(f'K-FOLD CROSS-VALIDATION RESULTS')
print(f'{"=" * 60}')
fold_df = pd.DataFrame(fold_results)
print(fold_df.to_string(index=False))
print(f'\nOverall OOF: Accuracy={oof_acc:.4f}, F1={oof_f1:.4f}')
print(f'Mean fold F1: {fold_df["f1"].mean():.4f} (+/- {fold_df["f1"].std():.4f})')
print(f'Total time: {fold_df["time_min"].sum():.1f} min')

```



```
=====
FOLD 1/5
=====
Train: 5125 samples, Val: 1282 samples

configjson: 100%                               579/579 [00:00<00:00, 12.0kB/s]

pytorch_model.bin: 100%                         371M/371M [00:02<00:00, 179MB/s]

model.safetensors: 100%                       371M/371M [00:02<00:00, 147MB/s]

Loading weights: 100%                          198/198 [00:00<00:00, 2659.49it/s]

[transformers] DebertaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base
Key | Status |
-----+-----+
lm_predictions.lm_head.dense.weight | UNEXPECTED |
lm_predictions.lm_head.dense.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.weight | UNEXPECTED |
mask_predictions.classifier.bias | UNEXPECTED |
lm_predictions.lm_head.bias | UNEXPECTED |
mask_predictions.classifier.weight | UNEXPECTED |
mask_predictions.dense.weight | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.bias | UNEXPECTED |
mask_predictions.LayerNorm.bias | UNEXPECTED |
mask_predictions.dense.bias | UNEXPECTED |
mask_predictions.LayerNorm.weight | UNEXPECTED |
classifier.weight | MISSING |
pooler.dense.weight | MISSING |
classifier.bias | MISSING |
pooler.dense.bias | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Step 50/641 | Loss: 0.6934 | Avg Loss: 0.7072 | LR: 1.30e-06
Step 100/641 | Loss: 0.6982 | Avg Loss: 0.7032 | LR: 2.60e-06
Step 150/641 | Loss: 0.7732 | Avg Loss: 0.6977 | LR: 3.91e-06
Step 200/641 | Loss: 0.6510 | Avg Loss: 0.6960 | LR: 5.21e-06
Step 250/641 | Loss: 0.6528 | Avg Loss: 0.6919 | LR: 6.51e-06
Step 300/641 | Loss: 0.7457 | Avg Loss: 0.6843 | LR: 7.81e-06
Step 350/641 | Loss: 0.7359 | Avg Loss: 0.6738 | LR: 9.11e-06
Step 400/641 | Loss: 0.3470 | Avg Loss: 0.6619 | LR: 9.95e-06
Step 450/641 | Loss: 0.5229 | Avg Loss: 0.6475 | LR: 9.81e-06
Step 500/641 | Loss: 0.4318 | Avg Loss: 0.6312 | LR: 9.66e-06
Step 550/641 | Loss: 0.3092 | Avg Loss: 0.6098 | LR: 9.52e-06
Step 600/641 | Loss: 0.1444 | Avg Loss: 0.5869 | LR: 9.38e-06
Epoch 1/3 | Train Loss: 0.5748 | Val Loss: 0.2810 | Acc: 0.8861 | F1: 0.8982 << saved
Step 50/641 | Loss: 0.4909 | Avg Loss: 0.2314 | LR: 9.12e-06
Step 100/641 | Loss: 0.4176 | Avg Loss: 0.2899 | LR: 8.97e-06
Step 150/641 | Loss: 0.2625 | Avg Loss: 0.2742 | LR: 8.83e-06
Step 200/641 | Loss: 0.5572 | Avg Loss: 0.2675 | LR: 8.68e-06
Step 250/641 | Loss: 0.0221 | Avg Loss: 0.2717 | LR: 8.54e-06
Step 300/641 | Loss: 0.1093 | Avg Loss: 0.2628 | LR: 8.39e-06
Step 350/641 | Loss: 0.3606 | Avg Loss: 0.2640 | LR: 8.25e-06
Step 400/641 | Loss: 0.0825 | Avg Loss: 0.2638 | LR: 8.11e-06
Step 450/641 | Loss: 0.4049 | Avg Loss: 0.2618 | LR: 7.96e-06
Step 500/641 | Loss: 0.5744 | Avg Loss: 0.2631 | LR: 7.82e-06
Step 550/641 | Loss: 0.0416 | Avg Loss: 0.2577 | LR: 7.67e-06
Step 600/641 | Loss: 0.4976 | Avg Loss: 0.2528 | LR: 7.53e-06
Epoch 2/3 | Train Loss: 0.2548 | Val Loss: 0.4345 | Acc: 0.8549 | F1: 0.8800
Step 50/641 | Loss: 0.0555 | Avg Loss: 0.1590 | LR: 7.27e-06
Step 100/641 | Loss: 0.0081 | Avg Loss: 0.1502 | LR: 7.12e-06
Step 150/641 | Loss: 0.0149 | Avg Loss: 0.1621 | LR: 6.98e-06
Step 200/641 | Loss: 0.7234 | Avg Loss: 0.1764 | LR: 6.83e-06
Step 250/641 | Loss: 0.0974 | Avg Loss: 0.1780 | LR: 6.69e-06
Step 300/641 | Loss: 0.0284 | Avg Loss: 0.1723 | LR: 6.55e-06
Step 350/641 | Loss: 0.0313 | Avg Loss: 0.1779 | LR: 6.40e-06
Step 400/641 | Loss: 0.0202 | Avg Loss: 0.1752 | LR: 6.26e-06
Step 450/641 | Loss: 0.0195 | Avg Loss: 0.1682 | LR: 6.11e-06
Step 500/641 | Loss: 0.1333 | Avg Loss: 0.1660 | LR: 5.97e-06
Step 550/641 | Loss: 0.0256 | Avg Loss: 0.1642 | LR: 5.82e-06
Step 600/641 | Loss: 0.2329 | Avg Loss: 0.1639 | LR: 5.68e-06
Epoch 3/3 | Train Loss: 0.1604 | Val Loss: 0.2595 | Acc: 0.9095 | F1: 0.9161 << saved

Fold 1 best: Acc=0.9095, F1=0.9161 (18.3 min)

=====
FOLD 2/5
=====
Train: 5125 samples, Val: 1282 samples

Loading weights: 100%                          198/198 [00:00<00:00, 6654.69it/s]

[transformers] DebertaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base
Key | Status |
-----+-----+
lm_predictions.lm_head.dense.weight | UNEXPECTED |
lm_predictions.lm_head.dense.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.weight | UNEXPECTED |
mask_predictions.classifier.bias | UNEXPECTED |
lm_predictions.lm_head.bias | UNEXPECTED |
```

lm_predictions.lm_head.dense.weight	UNEXPECTED
mask_predictions.classifier.weight	UNEXPECTED
mask_predictions.dense.weight	UNEXPECTED
lm_predictions.lm_head.LayerNorm.bias	UNEXPECTED
mask_predictions.LayerNorm.bias	UNEXPECTED
mask_predictions.dense.bias	UNEXPECTED
mask_predictions.LayerNorm.weight	UNEXPECTED
classifier.weight	MISSING
pooler.dense.weight	MISSING
classifier.bias	MISSING
pooler.dense.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```

Step 50/641 | Loss: 0.6993 | Avg Loss: 0.6926 | LR: 1.30e-06
Step 100/641 | Loss: 0.6951 | Avg Loss: 0.6965 | LR: 2.60e-06
Step 150/641 | Loss: 0.6598 | Avg Loss: 0.6949 | LR: 3.91e-06
Step 200/641 | Loss: 0.7047 | Avg Loss: 0.6949 | LR: 5.21e-06
Step 250/641 | Loss: 0.6871 | Avg Loss: 0.6940 | LR: 6.51e-06
Step 300/641 | Loss: 0.7148 | Avg Loss: 0.6922 | LR: 7.81e-06
Step 350/641 | Loss: 0.5721 | Avg Loss: 0.6848 | LR: 9.11e-06
Step 400/641 | Loss: 0.5044 | Avg Loss: 0.6626 | LR: 9.95e-06
Step 450/641 | Loss: 0.5534 | Avg Loss: 0.6340 | LR: 9.81e-06
Step 500/641 | Loss: 0.2426 | Avg Loss: 0.6112 | LR: 9.66e-06
Step 550/641 | Loss: 1.0765 | Avg Loss: 0.5917 | LR: 9.52e-06
Step 600/641 | Loss: 0.3328 | Avg Loss: 0.5667 | LR: 9.38e-06
Epoch 1/3 | Train Loss: 0.5505 | Val Loss: 0.3781 | Acc: 0.8526 | F1: 0.8749 << saved
Step 50/641 | Loss: 0.2493 | Avg Loss: 0.2487 | LR: 9.12e-06
Step 100/641 | Loss: 0.1372 | Avg Loss: 0.2345 | LR: 8.97e-06
Step 150/641 | Loss: 0.0246 | Avg Loss: 0.2396 | LR: 8.83e-06
Step 200/641 | Loss: 0.3990 | Avg Loss: 0.2649 | LR: 8.68e-06
Step 250/641 | Loss: 0.1905 | Avg Loss: 0.2752 | LR: 8.54e-06
Step 300/641 | Loss: 0.1609 | Avg Loss: 0.2625 | LR: 8.39e-06
Step 350/641 | Loss: 0.0604 | Avg Loss: 0.2548 | LR: 8.25e-06
Step 400/641 | Loss: 0.2544 | Avg Loss: 0.2539 | LR: 8.11e-06
Step 450/641 | Loss: 0.1347 | Avg Loss: 0.2530 | LR: 7.96e-06
Step 500/641 | Loss: 0.3860 | Avg Loss: 0.2621 | LR: 7.82e-06
Step 550/641 | Loss: 0.3454 | Avg Loss: 0.2627 | LR: 7.67e-06
Step 600/641 | Loss: 0.3016 | Avg Loss: 0.2607 | LR: 7.53e-06
Epoch 2/3 | Train Loss: 0.2599 | Val Loss: 0.3738 | Acc: 0.8502 | F1: 0.8755 << saved
Step 50/641 | Loss: 0.2005 | Avg Loss: 0.1558 | LR: 7.27e-06
Step 100/641 | Loss: 0.0303 | Avg Loss: 0.1543 | LR: 7.12e-06
Step 150/641 | Loss: 0.0083 | Avg Loss: 0.1475 | LR: 6.98e-06
Step 200/641 | Loss: 0.6027 | Avg Loss: 0.1537 | LR: 6.83e-06
Step 250/641 | Loss: 0.5577 | Avg Loss: 0.1531 | LR: 6.69e-06
Step 300/641 | Loss: 0.5197 | Avg Loss: 0.1593 | LR: 6.55e-06
Step 350/641 | Loss: 0.0097 | Avg Loss: 0.1596 | LR: 6.40e-06
Step 400/641 | Loss: 0.3519 | Avg Loss: 0.1609 | LR: 6.26e-06
Step 450/641 | Loss: 0.0104 | Avg Loss: 0.1565 | LR: 6.11e-06
Step 500/641 | Loss: 0.0403 | Avg Loss: 0.1569 | LR: 5.97e-06
Step 550/641 | Loss: 0.0383 | Avg Loss: 0.1578 | LR: 5.82e-06
Step 600/641 | Loss: 0.5247 | Avg Loss: 0.1539 | LR: 5.68e-06
Epoch 3/3 | Train Loss: 0.1554 | Val Loss: 0.3479 | Acc: 0.8978 | F1: 0.9086 << saved

```

Fold 2 best: Acc=0.8978, F1=0.9086 (19.6 min)

=====

FOLD 3/5

=====

Train: 5126 samples, Val: 1281 samples

Loading weights: 100%

198/198 [00:00<00:00, 5845.18it/s]

[transformers] DebertaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base

Key	Status
lm_predictions.lm_head.dense.weight	UNEXPECTED
lm_predictions.lm_head.dense.bias	UNEXPECTED
lm_predictions.lm_head.LayerNorm.weight	UNEXPECTED
mask_predictions.classifier.bias	UNEXPECTED
lm_predictions.lm_head.bias	UNEXPECTED
mask_predictions.classifier.weight	UNEXPECTED
mask_predictions.dense.weight	UNEXPECTED
lm_predictions.lm_head.LayerNorm.bias	UNEXPECTED
mask_predictions.LayerNorm.bias	UNEXPECTED
mask_predictions.dense.bias	UNEXPECTED
mask_predictions.LayerNorm.weight	UNEXPECTED
classifier.weight	MISSING
pooler.dense.weight	MISSING
classifier.bias	MISSING
pooler.dense.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```

Step 50/641 | Loss: 0.7170 | Avg Loss: 0.7048 | LR: 1.30e-06
Step 100/641 | Loss: 0.7062 | Avg Loss: 0.6988 | LR: 2.60e-06
Step 150/641 | Loss: 0.6839 | Avg Loss: 0.6979 | LR: 3.91e-06
Step 200/641 | Loss: 0.6828 | Avg Loss: 0.6966 | LR: 5.21e-06
Step 250/641 | Loss: 0.6038 | Avg Loss: 0.6949 | LR: 6.51e-06
Step 300/641 | Loss: 0.6523 | Avg Loss: 0.6865 | LR: 7.81e-06

```

```
Step 350/641 | Loss: 0.4435 | Avg Loss: 0.6738 | LR: 9.11e-06
Step 400/641 | Loss: 0.4193 | Avg Loss: 0.6582 | LR: 9.95e-06
Step 450/641 | Loss: 0.3851 | Avg Loss: 0.6280 | LR: 9.81e-06
Step 500/641 | Loss: 0.1759 | Avg Loss: 0.6005 | LR: 9.66e-06
Step 550/641 | Loss: 0.0698 | Avg Loss: 0.5818 | LR: 9.52e-06
Step 600/641 | Loss: 0.2398 | Avg Loss: 0.5583 | LR: 9.38e-06
Epoch 1/3 | Train Loss: 0.5456 | Val Loss: 0.3136 | Acc: 0.8704 | F1: 0.8863 << saved
Step 50/641 | Loss: 0.0207 | Avg Loss: 0.2208 | LR: 9.12e-06
Step 100/641 | Loss: 0.3729 | Avg Loss: 0.2563 | LR: 8.97e-06
Step 150/641 | Loss: 0.0364 | Avg Loss: 0.2386 | LR: 8.83e-06
Step 200/641 | Loss: 0.0655 | Avg Loss: 0.2502 | LR: 8.68e-06
Step 250/641 | Loss: 0.0268 | Avg Loss: 0.2404 | LR: 8.54e-06
Step 300/641 | Loss: 0.4729 | Avg Loss: 0.2310 | LR: 8.39e-06
Step 350/641 | Loss: 0.0712 | Avg Loss: 0.2359 | LR: 8.25e-06
Step 400/641 | Loss: 0.0555 | Avg Loss: 0.2382 | LR: 8.11e-06
Step 450/641 | Loss: 0.0277 | Avg Loss: 0.2338 | LR: 7.96e-06
Step 500/641 | Loss: 0.2395 | Avg Loss: 0.2315 | LR: 7.82e-06
Step 550/641 | Loss: 0.0502 | Avg Loss: 0.2281 | LR: 7.67e-06
Step 600/641 | Loss: 0.0498 | Avg Loss: 0.2277 | LR: 7.53e-06
Epoch 2/3 | Train Loss: 0.2284 | Val Loss: 0.4068 | Acc: 0.8642 | F1: 0.8851
Step 50/641 | Loss: 0.0714 | Avg Loss: 0.1244 | LR: 7.27e-06
Step 100/641 | Loss: 0.0228 | Avg Loss: 0.1188 | LR: 7.12e-06
Step 150/641 | Loss: 0.0708 | Avg Loss: 0.1183 | LR: 6.98e-06
Step 200/641 | Loss: 0.0052 | Avg Loss: 0.1375 | LR: 6.83e-06
Step 250/641 | Loss: 0.6916 | Avg Loss: 0.1377 | LR: 6.69e-06
Step 300/641 | Loss: 0.0995 | Avg Loss: 0.1481 | LR: 6.55e-06
Step 350/641 | Loss: 0.3679 | Avg Loss: 0.1452 | LR: 6.40e-06
Step 400/641 | Loss: 0.0056 | Avg Loss: 0.1446 | LR: 6.26e-06
Step 450/641 | Loss: 0.0066 | Avg Loss: 0.1456 | LR: 6.11e-06
Step 500/641 | Loss: 0.0030 | Avg Loss: 0.1440 | LR: 5.97e-06
Step 550/641 | Loss: 0.4384 | Avg Loss: 0.1441 | LR: 5.82e-06
Step 600/641 | Loss: 0.1187 | Avg Loss: 0.1441 | LR: 5.68e-06
Epoch 3/3 | Train Loss: 0.1500 | Val Loss: 0.3472 | Acc: 0.9063 | F1: 0.9120 << saved

Fold 3 best: Acc=0.9063, F1=0.9120 (19.5 min)
```

=====

FOLD 4/5

=====

Train: 5126 samples, Val: 1281 samples

Loading weights: 100%

198/198 [00:00<00:00, 5263.35it/s]

[transformers] DebertaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base

Key	Status
lm_predictions.lm_head.dense.weight	UNEXPECTED
lm_predictions.lm_head.dense.bias	UNEXPECTED
lm_predictions.lm_head.LayerNorm.weight	UNEXPECTED
mask_predictions.classifier.bias	UNEXPECTED
lm_predictions.lm_head.bias	UNEXPECTED
mask_predictions.classifier.weight	UNEXPECTED
mask_predictions.dense.weight	UNEXPECTED
lm_predictions.lm_head.LayerNorm.bias	UNEXPECTED
mask_predictions.LayerNorm.bias	UNEXPECTED
mask_predictions.dense.bias	UNEXPECTED
mask_predictions.LayerNorm.weight	UNEXPECTED
classifier.weight	MISSING
pooler.dense.weight	MISSING
classifier.bias	MISSING
pooler.dense.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
Step 50/641 | Loss: 0.6980 | Avg Loss: 0.6891 | LR: 1.30e-06
Step 100/641 | Loss: 0.6900 | Avg Loss: 0.6919 | LR: 2.60e-06
Step 150/641 | Loss: 0.6952 | Avg Loss: 0.6912 | LR: 3.91e-06
Step 200/641 | Loss: 0.7036 | Avg Loss: 0.6901 | LR: 5.21e-06
Step 250/641 | Loss: 0.7006 | Avg Loss: 0.6888 | LR: 6.51e-06
Step 300/641 | Loss: 0.5940 | Avg Loss: 0.6805 | LR: 7.81e-06
Step 350/641 | Loss: 0.4700 | Avg Loss: 0.6629 | LR: 9.11e-06
Step 400/641 | Loss: 0.5809 | Avg Loss: 0.6460 | LR: 9.95e-06
Step 450/641 | Loss: 0.3081 | Avg Loss: 0.6184 | LR: 9.81e-06
Step 500/641 | Loss: 0.6012 | Avg Loss: 0.5931 | LR: 9.66e-06
Step 550/641 | Loss: 0.0771 | Avg Loss: 0.5676 | LR: 9.52e-06
Step 600/641 | Loss: 0.5312 | Avg Loss: 0.5543 | LR: 9.38e-06
Epoch 1/3 | Train Loss: 0.5407 | Val Loss: 0.2952 | Acc: 0.8751 | F1: 0.8750 << saved
Step 50/641 | Loss: 0.1279 | Avg Loss: 0.2751 | LR: 9.12e-06
Step 100/641 | Loss: 0.3604 | Avg Loss: 0.2663 | LR: 8.97e-06
Step 150/641 | Loss: 0.6789 | Avg Loss: 0.2660 | LR: 8.83e-06
Step 200/641 | Loss: 0.3014 | Avg Loss: 0.2669 | LR: 8.68e-06
Step 250/641 | Loss: 0.0465 | Avg Loss: 0.2559 | LR: 8.54e-06
Step 300/641 | Loss: 0.1015 | Avg Loss: 0.2562 | LR: 8.39e-06
Step 350/641 | Loss: 0.3976 | Avg Loss: 0.2577 | LR: 8.25e-06
Step 400/641 | Loss: 0.2011 | Avg Loss: 0.2557 | LR: 8.11e-06
Step 450/641 | Loss: 0.3242 | Avg Loss: 0.2485 | LR: 7.96e-06
Step 500/641 | Loss: 0.0183 | Avg Loss: 0.2459 | LR: 7.82e-06
Step 550/641 | Loss: 0.2613 | Avg Loss: 0.2480 | LR: 7.67e-06
Step 600/641 | Loss: 0.2470 | Avg Loss: 0.2480 | LR: 7.53e-06
Epoch 2/3 | Train Loss: 0.2491 | Val Loss: 0.2653 | Acc: 0.9040 | F1: 0.9152 << saved
```

```
Step 50/641 | Loss: 0.3441 | Avg Loss: 0.1802 | LR: 1.2/e-06
Step 100/641 | Loss: 0.0284 | Avg Loss: 0.1599 | LR: 7.12e-06
Step 150/641 | Loss: 0.0103 | Avg Loss: 0.1546 | LR: 6.98e-06
Step 200/641 | Loss: 0.0048 | Avg Loss: 0.1417 | LR: 6.83e-06
Step 250/641 | Loss: 0.0039 | Avg Loss: 0.1350 | LR: 6.69e-06
Step 300/641 | Loss: 0.5643 | Avg Loss: 0.1426 | LR: 6.55e-06
Step 350/641 | Loss: 0.2449 | Avg Loss: 0.1518 | LR: 6.40e-06
Step 400/641 | Loss: 0.5295 | Avg Loss: 0.1550 | LR: 6.26e-06
Step 450/641 | Loss: 0.2453 | Avg Loss: 0.1607 | LR: 6.11e-06
Step 500/641 | Loss: 0.0064 | Avg Loss: 0.1572 | LR: 5.97e-06
Step 550/641 | Loss: 0.2980 | Avg Loss: 0.1614 | LR: 5.82e-06
Step 600/641 | Loss: 0.0265 | Avg Loss: 0.1611 | LR: 5.68e-06
Epoch 3/3 | Train Loss: 0.1616 | Val Loss: 0.2541 | Acc: 0.9079 | F1: 0.9147
```

Fold 4 best: Acc=0.9040, F1=0.9152 (19.2 min)

=====

FOLD 5/5

=====

Train: 5126 samples, Val: 1281 samples

Loading weights: 100%

198/198 [00:00<00:00, 5694.99it/s]

[transformers] DebertaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base

Key	Status
lm_predictions.lm_head.dense.weight	UNEXPECTED
lm_predictions.lm_head.dense.bias	UNEXPECTED
lm_predictions.lm_head.LayerNorm.weight	UNEXPECTED
mask_predictions.classifier.bias	UNEXPECTED
lm_predictions.lm_head.bias	UNEXPECTED
mask_predictions.classifier.weight	UNEXPECTED
mask_predictions.dense.weight	UNEXPECTED
lm_predictions.lm_head.LayerNorm.bias	UNEXPECTED
mask_predictions.LayerNorm.bias	UNEXPECTED
mask_predictions.dense.bias	UNEXPECTED
mask_predictions.LayerNorm.weight	UNEXPECTED
classifier.weight	MISSING
pooler.dense.weight	MISSING
classifier.bias	MISSING
pooler.dense.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
Step 50/641 | Loss: 0.6875 | Avg Loss: 0.6898 | LR: 1.30e-06
Step 100/641 | Loss: 0.6818 | Avg Loss: 0.6915 | LR: 2.60e-06
Step 150/641 | Loss: 0.7111 | Avg Loss: 0.6910 | LR: 3.91e-06
Step 200/641 | Loss: 0.6887 | Avg Loss: 0.6905 | LR: 5.21e-06
Step 250/641 | Loss: 0.6114 | Avg Loss: 0.6901 | LR: 6.51e-06
Step 300/641 | Loss: 0.6317 | Avg Loss: 0.6871 | LR: 7.81e-06
Step 350/641 | Loss: 0.6455 | Avg Loss: 0.6853 | LR: 9.11e-06
Step 400/641 | Loss: 0.7420 | Avg Loss: 0.6795 | LR: 9.95e-06
Step 450/641 | Loss: 0.4162 | Avg Loss: 0.6590 | LR: 9.81e-06
Step 500/641 | Loss: 0.4941 | Avg Loss: 0.6381 | LR: 9.66e-06
Step 550/641 | Loss: 0.2411 | Avg Loss: 0.6164 | LR: 9.52e-06
Step 600/641 | Loss: 0.3717 | Avg Loss: 0.5958 | LR: 9.38e-06
Epoch 1/3 | Train Loss: 0.5768 | Val Loss: 0.3815 | Acc: 0.8454 | F1: 0.8702 << saved
Step 50/641 | Loss: 0.1986 | Avg Loss: 0.2934 | LR: 9.12e-06
Step 100/641 | Loss: 0.0689 | Avg Loss: 0.2928 | LR: 8.97e-06
Step 150/641 | Loss: 0.2816 | Avg Loss: 0.2770 | LR: 8.83e-06
Step 200/641 | Loss: 0.7696 | Avg Loss: 0.2702 | LR: 8.68e-06
Step 250/641 | Loss: 0.4589 | Avg Loss: 0.2644 | LR: 8.54e-06
Step 300/641 | Loss: 0.1031 | Avg Loss: 0.2614 | LR: 8.39e-06
Step 350/641 | Loss: 0.0635 | Avg Loss: 0.2626 | LR: 8.25e-06
Step 400/641 | Loss: 0.3094 | Avg Loss: 0.2587 | LR: 8.11e-06
Step 450/641 | Loss: 0.0704 | Avg Loss: 0.2589 | LR: 7.96e-06
Step 500/641 | Loss: 0.3384 | Avg Loss: 0.2472 | LR: 7.82e-06
Step 550/641 | Loss: 0.0252 | Avg Loss: 0.2462 | LR: 7.67e-06
Step 600/641 | Loss: 0.2983 | Avg Loss: 0.2483 | LR: 7.53e-06
Epoch 2/3 | Train Loss: 0.2476 | Val Loss: 0.2745 | Acc: 0.8985 | F1: 0.9081 << saved
Step 50/641 | Loss: 0.0294 | Avg Loss: 0.1316 | LR: 7.27e-06
Step 100/641 | Loss: 0.6194 | Avg Loss: 0.1550 | LR: 7.12e-06
Step 150/641 | Loss: 0.1394 | Avg Loss: 0.1600 | LR: 6.98e-06
Step 200/641 | Loss: 0.0123 | Avg Loss: 0.1528 | LR: 6.83e-06
Step 250/641 | Loss: 0.0196 | Avg Loss: 0.1485 | LR: 6.69e-06
Step 300/641 | Loss: 1.0084 | Avg Loss: 0.1527 | LR: 6.55e-06
Step 350/641 | Loss: 0.0047 | Avg Loss: 0.1557 | LR: 6.40e-06
Step 400/641 | Loss: 0.7546 | Avg Loss: 0.1680 | LR: 6.26e-06
Step 450/641 | Loss: 0.0100 | Avg Loss: 0.1697 | LR: 6.11e-06
Step 500/641 | Loss: 0.0064 | Avg Loss: 0.1665 | LR: 5.97e-06
Step 550/641 | Loss: 0.0088 | Avg Loss: 0.1682 | LR: 5.82e-06
Step 600/641 | Loss: 0.3407 | Avg Loss: 0.1692 | LR: 5.68e-06
Epoch 3/3 | Train Loss: 0.1653 | Val Loss: 0.3321 | Acc: 0.9032 | F1: 0.9117 << saved
```

Fold 5 best: Acc=0.9032, F1=0.9117 (20.0 min)

=====

K-FOLD CROSS-VALIDATION RESULTS

=====

fold	accuracy	f1	time_min
1	0.909516	0.916064	18.277920

5.4 Save fold checkpoints to Google Drive

```
2 0.897816 0.908583 19.558669
3 0.903981 0.915231 19.167573
4 0.903981 0.915231 19.167573
5 0.903201 0.911681 20.020987
```

Save all fold checkpoints to Google Drive so they can be reused in a new session without retraining.

Overall OOF: Accuracy=0.9042, F1=0.9127

```
import shutil

drive_ckpt_folder = '/content/drive/MyDrive/SMU_MITB_NLP/Assignment/checkpoints/kfold_deberta_seed42/'
os.makedirs(drive_ckpt_folder, exist_ok=True)

for fold_idx in range(N_FOLDS):
    src = f'fold_checkpoints/deberta_fold{fold_idx+1}.pt'
    dst = f'{drive_ckpt_folder}/deberta_fold{fold_idx+1}.pt'
    if os.path.exists(src):
        shutil.copy(src, dst)
        print(f'Saved: {dst}')

print('All fold checkpoints saved to Google Drive.')
```

```
Saved: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/kfold/deberta_fold1.pt
Saved: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/kfold/deberta_fold2.pt
Saved: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/kfold/deberta_fold3.pt
Saved: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/kfold/deberta_fold4.pt
Saved: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/kfold/deberta_fold5.pt
All fold checkpoints saved to Google Drive.
```

5.5 Threshold tuning

Instead of using the default 0.5 threshold, search for the optimal decision threshold using the out-of-fold predictions. This can squeeze an extra 0.5 to 1 point of F1.

```
thresholds = np.arange(0.30, 0.70, 0.01)
threshold_results = []

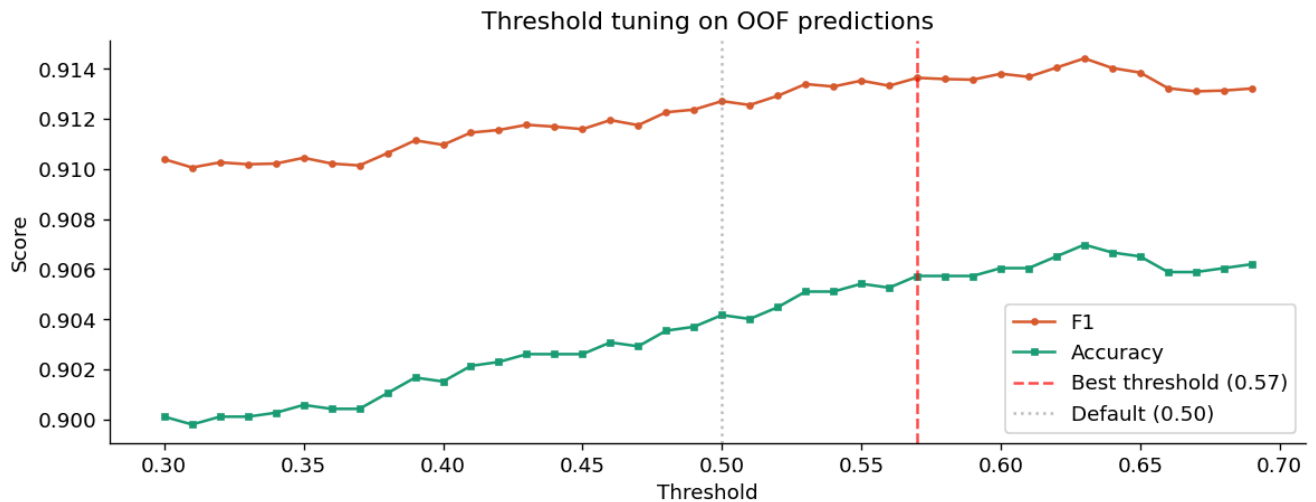
for t in thresholds:
    preds = (oof_probs[:, 1] > t).astype(int)
    f1 = f1_score(all_labels, preds)
    acc = accuracy_score(all_labels, preds)
    threshold_results.append({'threshold': round(t, 2), 'f1': f1, 'accuracy': acc})

thresh_df = pd.DataFrame(threshold_results)
best_thresh_row = thresh_df.loc[thresh_df['f1'].idxmax()]
# best_threshold = best_thresh_row['threshold']
best_threshold = 0.57 # locked from Kaggle sweep results

print(f'Best threshold: {best_threshold:.2f}')
print(f' F1 at best threshold : {best_thresh_row["f1"]:.4f}')
print(f' F1 at default (0.50) : {thresh_df[thresh_df["threshold"]==0.50]["f1"].values[0]:.4f}')
print(f' Gain from tuning : {best_thresh_row["f1"] - thresh_df[thresh_df["threshold"]==0.50]["f1"].values[0]:+.4f}')

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(thresh_df['threshold'], thresh_df['f1'], 'o-', color=UNSAFE_COLOR, label='F1', markersize=3)
ax.plot(thresh_df['threshold'], thresh_df['accuracy'], 's-', color=SAFE_COLOR, label='Accuracy', markersize=3)
ax.axvline(x=best_threshold, color='red', linestyle='--', alpha=0.7, label=f'Best threshold ({best_threshold:.2f})')
ax.axvline(x=0.50, color='gray', linestyle=':', alpha=0.5, label='Default (0.50)')
ax.set_xlabel('Threshold')
ax.set_ylabel('Score')
ax.set_title('Threshold tuning on OOF predictions')
ax.legend()
ax.spines[['top', 'right']].set_visible(False)
plt.tight_layout()
plt.show()
```

Best threshold: 0.57
 F1 at best threshold : 0.9144
 F1 at default (0.50) : 0.9127
 Gain from tuning : +0.0017



5.6 Generate eval.csv predictions with k-fold ensemble

Load each fold checkpoint, predict on eval.csv, and average the probabilities across all folds. Then apply the optimised threshold to get the final predictions.

```
# Make sure eval_texts is available (from section 4.5)

# Download eval.csv from Account 1's shared link
gdown.download('https://drive.google.com/uc?id=1p1PkEalcqgs8T5P-b2cSBjiyrfymNroC', 'eval.csv', quiet=False)

# If not, uncomment and run:
eval_df = pd.read_csv('eval.csv')
eval_df['scenario_clean'] = eval_df['scenario'].fillna('').apply(normalise_text)
eval_df['movie_clean'] = eval_df['movie'].apply(normalise_text)
eval_df['input_text'] = eval_df.apply(build_input_text, axis=1)
eval_texts = eval_df['input_text'].values.tolist()

eval_pred_ds = KFoldPredDataset(eval_texts, kfold_tokenizer, KFOLD_MAXLEN)
eval_pred_loader = DataLoader(eval_pred_ds, batch_size=16, shuffle=False)

# Accumulate probabilities from all folds
eval_ensemble_probs = np.zeros((len(eval_texts), 2), dtype=np.float32)

for fold_idx in range(N_FOLDS):
    print(f'Predicting with fold {fold_idx+1}/{N_FOLDS}...')

    # Try local checkpoint first, then Google Drive
    ckpt_path = f'fold_checkpoints/deberta_fold{fold_idx+1}.pt'
    if not os.path.exists(ckpt_path):
        ckpt_path = f'/content/drive/MyDrive/SMU_MITB_NLP/Assignment/checkpoints/kfold_deberta_seed42/deberta_fold{fold_idx+1}.pt'

    fold_model = DebertaV2ForSequenceClassification.from_pretrained(DEBERTA_NAME, num_labels=2)
    fold_model = fold_model.float()
    fold_model.load_state_dict(torch.load(ckpt_path, map_location=device))
    fold_model.to(device)
    fold_model.eval()

    fold_logits = []
    with torch.no_grad():
        for batch in eval_pred_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask)
            fold_logits.append(outputs.logits.cpu().numpy())

    fold_logits = np.concatenate(fold_logits, axis=0)
    eval_ensemble_probs += sp_softmax(fold_logits, axis=1)

del fold_model
torch.cuda.empty_cache()
```

```

print(f' Fold {fold_idx+1} done. ')

# Average across folds
eval_ensemble_probs /= N_FOLDS

# Apply both default and tuned threshold
eval_preds_default = np.argmax(eval_ensemble_probs, axis=1)
eval_preds_tuned = (eval_ensemble_probs[:, 1] > best_threshold).astype(int)

print(f'\nPredictions with default threshold (0.50):')
print(f' Safe: {(eval_preds_default==0).sum()}, Unsafe: {(eval_preds_default==1).sum()}')

print(f'\nPredictions with tuned threshold ({best_threshold:.2f}):')
print(f' Safe: {(eval_preds_tuned==0).sum()}, Unsafe: {(eval_preds_tuned==1).sum()}')

```

Downloading...

From: <https://drive.google.com/uc?id=1p1PkEalcggs8T5P-b2cSBjiyrfymNroC>

To: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/eval.csv

100%|██████████| 161k/161k [00:00<00:00, 13.3MB/s]

NameError Traceback (most recent call last)
 /tmp/ipykernel_2046/4135707066.py in <cell line: 0>()

```

12
13
--> 14 eval_pred_ds = KFoldPredDataset(eval_texts, kfold_tokenizer, KFOLD_MAXLEN)
15 eval_pred_loader = DataLoader(eval_pred_ds, batch_size=16, shuffle=False)
16

```

NameError: name 'KFoldPredDataset' is not defined

Next steps: [Explain error](#)

5.7 Generate Kaggle submission files

Generate two submission files: one with the default threshold and one with the tuned threshold. Submit both to Kaggle and see which scores higher.

connect google drive folder

```

#@title connect google drive folder

import os
os.system('rm -rf /content/drive')

from google.colab import drive
drive.mount('/content/drive')
%cd /content/drive/MyDrive/SMU_MITB_NLP/Assignment/

```

Mounted at /content/drive
 /content/drive/MyDrive/SMU_MITB_NLP/Assignment

```

from datetime import datetime

oof_preds = np.argmax(oof_probs, axis=1)
oof_f1 = f1_score(all_labels, oof_preds)

drive_folder = '/content/drive/MyDrive/SMU_MITB_NLP/Assignment/Submission'
os.makedirs(drive_folder, exist_ok=True)

today = datetime.now().strftime('%Y%m%d')
existing = [f for f in os.listdir(drive_folder) if f.startswith(today)]
next_num = len(existing) + 1

# Submission 1: default threshold
sub_default = pd.DataFrame({'id': eval_df['id'], 'label': eval_preds_default})
fname_default = f'{today}_{next_num:02d}_kfold_deberta_default_OOF_F1_{oof_f1:.4f}.csv'
sub_default.to_csv(os.path.join(drive_folder, fname_default), index=False)
sub_default.to_csv('submission_kfold_default.csv', index=False)
print(f'Saved: {fname_default}')

# Submission 2: tuned threshold
sub_tuned = pd.DataFrame({'id': eval_df['id'], 'label': eval_preds_tuned})
fname_tuned = f'{today}_{next_num+1:02d}_kfold_deberta_thresh{best_threshold:.2f}_OOF_F1_{best_thresh_row["f1"]:.4f}.csv'
sub_tuned.to_csv(os.path.join(drive_folder, fname_tuned), index=False)
sub_tuned.to_csv('submission_kfold_tuned.csv', index=False)
print(f'Saved: {fname_tuned}')

# Compare

```

```
diff = (eval_preds_default != eval_preds_tuned).sum()
print(f'\nDifference between default and tuned: {diff} samples ({diff/len(eval_preds_default)*100:.1f}%)')
```

Saved: 20260613_01_kfold_deberta_default_OOF_F1_0.9127.csv
 Saved: 20260613_02_kfold_deberta_thresh0.57_OOF_F1_0.9144.csv

Difference between default and tuned: 5 samples (0.7%)

5.8 Step 5 summary

```
print('=' * 60)
print('STEP 5 SUMMARY: K-FOLD DEBERTA ENSEMBLE')
print('=' * 60)
print(f'Model           : DeBERTa-v3-base')
print(f'Folds           : {N_FOLDS}')
print(f'Epochs per fold   : {KFOLD_EPOCHS}')
print(f'Training data      : {len(all_texts):,} samples (full cleaned dataset)')
print(f'')
print(f'Best threshold      : {best_threshold:.2f}')
print(f'F1 at best threshold: {best_thresh_row["f1"]:.4f}')
print(f'F1 at default (0.50): {thresh_df[thresh_df["threshold"]==0.50]["f1"].values[0]:.4f}')
print(f'')
print(f'Submission files:')
print(f' {fname_default}')
print(f' {fname_tuned}')
print('=' * 60)
print('Submit both to Kaggle and compare scores.')
```

```
=====
STEP 6 SUMMARY: K-FOLD DEBERTA ENSEMBLE
=====
Model           : DeBERTa-v3-base
Folds           : 5
Epochs per fold : 3
Training data    : 6,407 samples (full cleaned dataset)

Best threshold    : 0.57
F1 at best threshold: 0.9144
F1 at default (0.50): 0.9127

Submission files:
  20260613_01_kfold_deberta_default_OOF_F1_0.9127.csv
  20260613_02_kfold_deberta_thresh0.57_OOF_F1_0.9144.csv
=====
Submit both to Kaggle and compare scores.
```

5.9 Save all artifacts from Account 2

Sometimes I need to run Colab on second account with GPU availability so I write code to save in new account.

```
import os
import shutil
import glob
from google.colab import drive

# Mount Account 2's drive
drive.mount('/content/drive')

# === CONFIG: change this to wherever you want on Account 2 ===
BACKUP_ROOT = '/content/drive/MyDrive/SMU_MITB_NLP/Assignment'

# Define backup structure
backup_map = {
    # Data files
    'data': [
        'train_clean.csv',
        'train_confident.csv',
        'eval.csv',
    ],
    # Numpy arrays & sklearn objects
    'features': [
        'X_train.npy',
        'X_val.npy',
        'y_train.npy',
        'y_val.npy',
        'word_tfidf.pkl',
        'char_tfidf.pkl',
        'hand_feature_scaler.pkl',
    ],
    # Model checkpoints
```

```

# Model checkpoints
'checkpoints/single_models': [
    'best_distilbert.pt',
    'best_roberta.pt',
    'best_deberta.pt'
],
# K-fold checkpoints (folder)
'checkpoints/kfold_deberta_seed42': [
    'fold_checkpoints/deberta_fold1.pt',
    'fold_checkpoints/deberta_fold2.pt',
    'fold_checkpoints/deberta_fold3.pt',
    'fold_checkpoints/deberta_fold4.pt',
    'fold_checkpoints/deberta_fold5.pt',
],
# Submissions
'submissions': [
    'submission.csv',
    'submission_kfold_default.csv',
    'submission_kfold_tuned.csv',
],
}

def backup_all():
    """Copy all generated files to Account 2's Google Drive."""
    total_saved = 0
    total_skipped = 0

    for subfolder, files in backup_map.items():
        dest_dir = os.path.join(BACKUP_ROOT, subfolder)
        os.makedirs(dest_dir, exist_ok=True)

        for src_path in files:
            if os.path.exists(src_path):
                filename = os.path.basename(src_path)
                dst_path = os.path.join(dest_dir, filename)
                shutil.copy2(src_path, dst_path)
                size_mb = os.path.getsize(src_path) / (1024 * 1024)
                print(f' ✓ {src_path:45s} → {subfolder}/{filename} ({size_mb:.1f} MB)')
                total_saved += 1
            else:
                print(f' X {src_path:45s} (not found yet, skipping)')
                total_skipped += 1

    # Also grab any extra submission CSVs with date prefixes
    for pattern in ['20*.csv', 'submission*.csv']:
        for f in glob.glob(pattern):
            if f not in [item for files in backup_map.values() for item in files]:
                dest_dir = os.path.join(BACKUP_ROOT, 'submissions')
                shutil.copy2(f, os.path.join(dest_dir, f))
                print(f' ✓ {f:45s} → submissions/{f} (extra)')
                total_saved += 1

    print(f'\nDone: {total_saved} files saved, {total_skipped} not found yet')
    print(f'Backup location: {BACKUP_ROOT}')

# Run it
backup_all()

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```

X train_clean.csv                (not found yet, skipping)
X train_confident.csv            (not found yet, skipping)
X eval.csv                       (not found yet, skipping)
X X_train.npy                   (not found yet, skipping)
X X_val.npy                     (not found yet, skipping)
X y_train.npy                   (not found yet, skipping)
X y_val.npy                     (not found yet, skipping)
X word_tfidf.pkl                (not found yet, skipping)
X char_tfidf.pkl                (not found yet, skipping)
X hand_feature_scaler.pkl       (not found yet, skipping)
X best_distilbert.pt            (not found yet, skipping)
X best_roberta.pt               (not found yet, skipping)
X best_deberta.pt               (not found yet, skipping)
X fold_checkpoints/deberta_fold1.pt (not found yet, skipping)
X fold_checkpoints/deberta_fold2.pt (not found yet, skipping)
X fold_checkpoints/deberta_fold3.pt (not found yet, skipping)
X fold_checkpoints/deberta_fold4.pt (not found yet, skipping)
X fold_checkpoints/deberta_fold5.pt (not found yet, skipping)
X submission.csv                (not found yet, skipping)
✓ submission_kfold_default.csv    → submissions/submission_kfold_default.csv (0.0 MB)
✓ submission_kfold_tuned.csv     → submissions/submission_kfold_tuned.csv (0.0 MB)

```

Done: 2 files saved, 19 not found yet

Backup location: /content/drive/MyDrive/SMU_MITB_NLP/Assignment

✓ Step 6: K-Fold RoBERTa + DeBERTa Ensemble

Train 5-fold RoBERTa using the same pipeline as DeBERTa, then average predictions from both models for a stronger ensemble. Different architectures make different errors, so averaging improves robustness.

✓ 6.1 K-Fold RoBERTa Setup

```
#@title 6.1 K-Fold RoBERTa Setup
import os
from transformers import RobertaTokenizer, RobertaForSequenceClassification

ROBERTA_NAME = 'roberta-base'
ROBERTA_MAXLEN = 256
ROBERTA_EPOCHS = 3
ROBERTA_LR = 1e-5
ROBERTA_BS = 8
ROBERTA_ACCUM = 2

roberta_kfold_tokenizer = RobertaTokenizer.from_pretrained(ROBERTA_NAME)

os.makedirs('fold_checkpoints_roberta', exist_ok=True)

print(f'Model          : {ROBERTA_NAME}')
print(f'Folds          : {N_FOLDS}')
print(f'Epochs per fold  : {ROBERTA_EPOCHS}')
print(f'Batch size       : {ROBERTA_BS} (effective: {ROBERTA_BS * ROBERTA_ACCUM})')
print(f'Learning rate    : {ROBERTA_LR}')
print(f'Dataset          : {len(all_texts):,} samples')
```

Model	: roberta-base
Folds	: 5
Epochs per fold	: 3
Batch size	: 8 (effective: 16)
Learning rate	: 1e-05
Dataset	: 6,407 samples

✓ 6.2 RoBERTa K-Fold Dataset

```
#@title 6.2 RoBERTa K-Fold Dataset
# Reuse KFoldDataset and KFoldPredDataset from cell 147
print('Using existing KFoldDataset and KFoldPredDataset classes.')
```

Using existing KFoldDataset and KFoldPredDataset classes.

✓ 6.3 Train K-Fold RoBERTa

```
#@title 6.3 Train K-Fold RoBERTa

roberta_oof_probs = np.zeros((len(all_texts), 2), dtype=np.float32)
roberta_fold_results = []

skf_roberta = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=RANDOM_SEED)

for fold_idx, (train_idx, val_idx) in enumerate(skf_roberta.split(all_texts, all_labels)):
    print(f'\n{" " * 60}')
    print(f'ROBERTA FOLD {fold_idx + 1}/{N_FOLDS}')
    print(f'{" " * 60}')
    fold_start = time.time()

    fold_train_texts = all_texts[train_idx].tolist()
    fold_train_labels = all_labels[train_idx].tolist()
    fold_val_texts = all_texts[val_idx].tolist()
    fold_val_labels = all_labels[val_idx].tolist()

    fold_train_ds = KFoldDataset(fold_train_texts, fold_train_labels, roberta_kfold_tokenizer, ROBERTA_MAXLEN)
    fold_val_ds = KFoldDataset(fold_val_texts, fold_val_labels, roberta_kfold_tokenizer, ROBERTA_MAXLEN)

    fold_train_loader = DataLoader(fold_train_ds, batch_size=ROBERTA_BS, shuffle=True)
    fold_val_loader = DataLoader(fold_val_ds, batch_size=ROBERTA_BS, shuffle=False)

    print(f' Train: {len(fold_train_texts)} samples, Val: {len(fold_val_texts)} samples')

    fold_model = RobertaForSequenceClassification.from_pretrained(ROBERTA_NAME, num_labels=2)
    fold_model.to(device)

    fold_optimizer = torch.optim.AdamW(
        fold_model.parameters(), lr=ROBERTA_LR, weight_decay=0.01, eps=1e-6
```

```

/
fold_total_steps = len(fold_train_loader) * ROBERTA_EPOCHS
fold_scheduler = get_linear_schedule_with_warmup(
    fold_optimizer,
    num_warmup_steps=int(fold_total_steps * 0.1),
    num_training_steps=fold_total_steps
)

fold_best_f1 = 0
checkpoint_path = f'fold_checkpoints_roberta/roberta_fold{fold_idx+1}.pt'

for epoch in range(ROBERTA_EPOCHS):
    fold_model.train()
    fold_optimizer.zero_grad()
    epoch_loss = 0
    log_interval = 50

    for step, batch in enumerate(fold_train_loader):
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss / ROBERTA_ACCUM
        loss.backward()
        epoch_loss += outputs.loss.item()

        if (step + 1) % ROBERTA_ACCUM == 0:
            torch.nn.utils.clip_grad_norm_(fold_model.parameters(), max_norm=1.0)
            fold_optimizer.step()
            fold_scheduler.step()
            fold_optimizer.zero_grad()

        if (step + 1) % log_interval == 0:
            avg_loss_so_far = epoch_loss / (step + 1)
            lr_now = fold_scheduler.get_last_lr()[0]
            print(f'    Step {step+1}/{len(fold_train_loader)} | '
                  f'Loss: {outputs.loss.item():.4f} | '
                  f'Avg Loss: {avg_loss_so_far:.4f} | '
                  f'LR: {lr_now:.2e}')

    fold_model.eval()
    val_preds_list, val_labels_list = [], []
    val_loss = 0
    with torch.no_grad():
        for batch in fold_val_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels_batch = batch['labels'].to(device)
            outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels_batch)
            val_loss += outputs.loss.item()
            preds = torch.argmax(outputs.logits, dim=1)
            val_preds_list.extend(preds.cpu().numpy())
            val_labels_list.extend(labels_batch.cpu().numpy())

    epoch_acc = accuracy_score(val_labels_list, val_preds_list)
    epoch_f1 = f1_score(val_labels_list, val_preds_list)
    avg_train_loss = epoch_loss / len(fold_train_loader)
    avg_val_loss = val_loss / len(fold_val_loader)

    status = ''
    if epoch_f1 > fold_best_f1:
        fold_best_f1 = epoch_f1
        torch.save(fold_model.state_dict(), checkpoint_path)
        status = ' << saved'

    print(f' Epoch {epoch+1}/{ROBERTA_EPOCHS} | '
          f'Train Loss: {avg_train_loss:.4f} | '
          f'Val Loss: {avg_val_loss:.4f} | '
          f'Acc: {epoch_acc:.4f} | '
          f'F1: {epoch_f1:.4f}{status}')

fold_model.load_state_dict(torch.load(checkpoint_path, map_location=device))
fold_model.eval()
fold_logits = []
with torch.no_grad():
    for batch in fold_val_loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask)
        fold_logits.append(outputs.logits.cpu().numpy())

fold_logits = np.concatenate(fold_logits, axis=0)

```

```

fold_probs = sp_softmax(fold_logits, axis=1)
roberta_oof_probs[val_idx] = fold_probs

fold_preds = np.argmax(fold_probs, axis=1)
fold_acc = accuracy_score(fold_val_labels, fold_preds)
fold_f1 = f1_score(fold_val_labels, fold_preds)

elapsed = time.time() - fold_start
roberta_fold_results.append({'fold': fold_idx+1, 'accuracy': fold_acc, 'f1': fold_f1, 'time_min': elapsed/60})
print(f'\n Fold {fold_idx+1} best: Acc={fold_acc:.4f}, F1={fold_f1:.4f} ({elapsed/60:.1f} min)')

del fold_model
torch.cuda.empty_cache()

roberta_oof_preds = np.argmax(roberta_oof_probs, axis=1)
roberta_oof_acc = accuracy_score(all_labels, roberta_oof_preds)
roberta_oof_f1 = f1_score(all_labels, roberta_oof_preds)

print(f'\n{"=" * 60}')
print(f'K-FOLD ROBERTA RESULTS')
print(f'{"=" * 60}')
roberta_fold_df = pd.DataFrame(roberta_fold_results)
print(roberta_fold_df.to_string(index=False))
print(f'\nOverall OOF: Accuracy={roberta_oof_acc:.4f}, F1={roberta_oof_f1:.4f}')
print(f'Mean fold F1: {roberta_fold_df["f1"].mean():.4f} (+/- {roberta_fold_df["f1"].std():.4f})')
print(f'Total time: {roberta_fold_df["time_min"].sum():.1f} min')

```

```

=====
ROBERTA FOLD 1/5
=====
Train: 5125 samples, Val: 1282 samples

configjson: 100%                                481/481 [00:00<00:00, 53.7kB/s]

model.safetensors: 100%                        499M/499M [00:04<00:00, 74.9MB/s]

Loading weights: 100%                            197/197 [00:00<00:00, 5822.96it/s]

[transformers] RobertaForSequenceClassification LOAD REPORT from: roberta-base
Key | Status |
-----+-----
lm_head.dense.bias | UNEXPECTED |
lm_head.layer_norm.weight | UNEXPECTED |
lm_head.layer_norm.bias | UNEXPECTED |
lm_head.dense.weight | UNEXPECTED |
lm_head.bias | UNEXPECTED |
classifier.dense.bias | MISSING |
classifier.dense.weight | MISSING |
classifier.out_proj.weight | MISSING |
classifier.out_proj.bias | MISSING |

```

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```

Step 50/641 | Loss: 0.7079 | Avg Loss: 0.7017 | LR: 1.30e-06
Step 100/641 | Loss: 0.6687 | Avg Loss: 0.7002 | LR: 2.60e-06
Step 150/641 | Loss: 0.6875 | Avg Loss: 0.6989 | LR: 3.91e-06
Step 200/641 | Loss: 0.7104 | Avg Loss: 0.6977 | LR: 5.21e-06
Step 250/641 | Loss: 0.6920 | Avg Loss: 0.6961 | LR: 6.51e-06
Step 300/641 | Loss: 0.5913 | Avg Loss: 0.6931 | LR: 7.81e-06
Step 350/641 | Loss: 0.6988 | Avg Loss: 0.6929 | LR: 9.11e-06
Step 400/641 | Loss: 0.6335 | Avg Loss: 0.6888 | LR: 9.95e-06
Step 450/641 | Loss: 0.1645 | Avg Loss: 0.6724 | LR: 9.81e-06
Step 500/641 | Loss: 0.6103 | Avg Loss: 0.6495 | LR: 9.66e-06
Step 550/641 | Loss: 0.6939 | Avg Loss: 0.6336 | LR: 9.52e-06
Step 600/641 | Loss: 0.3877 | Avg Loss: 0.6234 | LR: 9.38e-06
Epoch 1/3 | Train Loss: 0.6108 | Val Loss: 0.3895 | Acc: 0.8276 | F1: 0.8506 << saved
Step 50/641 | Loss: 0.7506 | Avg Loss: 0.4094 | LR: 9.12e-06
Step 100/641 | Loss: 0.1041 | Avg Loss: 0.4115 | LR: 8.97e-06
Step 150/641 | Loss: 0.1631 | Avg Loss: 0.4056 | LR: 8.83e-06
Step 200/641 | Loss: 0.0345 | Avg Loss: 0.3836 | LR: 8.68e-06
Step 250/641 | Loss: 0.1037 | Avg Loss: 0.3704 | LR: 8.54e-06
Step 300/641 | Loss: 0.1423 | Avg Loss: 0.3589 | LR: 8.39e-06
Step 350/641 | Loss: 0.4304 | Avg Loss: 0.3571 | LR: 8.25e-06
Step 400/641 | Loss: 0.4032 | Avg Loss: 0.3556 | LR: 8.11e-06
Step 450/641 | Loss: 0.7894 | Avg Loss: 0.3551 | LR: 7.96e-06
Step 500/641 | Loss: 0.2076 | Avg Loss: 0.3516 | LR: 7.82e-06
Step 550/641 | Loss: 0.5018 | Avg Loss: 0.3520 | LR: 7.67e-06
Step 600/641 | Loss: 0.3094 | Avg Loss: 0.3461 | LR: 7.53e-06
Epoch 2/3 | Train Loss: 0.3444 | Val Loss: 0.3283 | Acc: 0.8721 | F1: 0.8822 << saved
Step 50/641 | Loss: 0.0960 | Avg Loss: 0.1830 | LR: 7.27e-06
Step 100/641 | Loss: 0.0580 | Avg Loss: 0.2226 | LR: 7.12e-06
Step 150/641 | Loss: 0.4086 | Avg Loss: 0.2151 | LR: 6.98e-06
Step 200/641 | Loss: 0.2330 | Avg Loss: 0.2253 | LR: 6.83e-06
Step 250/641 | Loss: 0.0772 | Avg Loss: 0.2194 | LR: 6.69e-06
Step 300/641 | Loss: 0.0963 | Avg Loss: 0.2274 | LR: 6.55e-06
Step 350/641 | Loss: 0.2153 | Avg Loss: 0.2286 | LR: 6.40e-06
Step 400/641 | Loss: 0.6443 | Avg Loss: 0.2299 | LR: 6.26e-06

```

6.4 Save ROBERTa checkpoints to Drive

Key	Status
Saved fold 1 to Drive	
-- Saved fold 2 to Drive --	- - - - -
lm_head.dense.bias	UNEXPECTED
lm_head.fayer4nomDweight	UNEXPECTED
lm_head.fayer5nomDbias	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
lm_head.bias	UNEXPECTED
Reload DeBERTa as RoBERTa NOOF predictions from Drive (no retrain)	MISSING
classifier.dense.weight	MISSING

6.4 Reload DeBERTa, RoBERTa, GPT predictions from Drive (no retraining)

```
#@title 6.4 Reload DeBERTa + RoBERTa OOF predictions from Drive (no retraining)

import os
import numpy as np
import torch
from torch.utils.data import DataLoader
from sklearn.model_selection import StratifiedKFold
from scipy.special import softmax as sp_softmax
from sklearn.metrics import f1_score, accuracy_score
from transformers import (
    DebertaV2Tokenizer, DebertaV2ForSequenceClassification,
    RobertaTokenizer, RobertaForSequenceClassification
)

# --- Config (must match training) ---
DEBERTA_NAME = 'microsoft/deberta-v3-base'
ROBERTA_NAME = 'roberta-base'
KFOLD_MAXLEN = 256
ROBERTA_MAXLEN = 256
N_FOLDS = 5
RANDOM_SEED = 42

# --- Checkpoint paths on Drive ---
DEBERTA_CKPT_DIR = '/content/drive/MyDrive/SMU_MITB_NLP/Assignment/checkpoints/kfold_deberta_seed42'
ROBERTA_CKPT_DIR = '/content/drive/MyDrive/SMU_MITB_NLP/Assignment/checkpoints/kfold_roberta_seed42'

# --- Tokenizers ---
kfold_tokenizer = DebertaV2Tokenizer.from_pretrained(DEBERTA_NAME)
roberta_kfold_tokenizer = RobertaTokenizer.from_pretrained(ROBERTA_NAME)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {device}')

# --- Helper function ---
def generate_oof_probs(model_class, model_name, tokenizer, max_len, ckpt_dir, ckpt_prefix):
    """Load fold checkpoints and generate OOF predictions without retraining."""
    oof_probs = np.zeros((len(all_texts), 2), dtype=np.float32)
    skf = StratifiedKFold(n_splits=N_FOLDS, shuffle=True, random_state=RANDOM_SEED)

    for fold_idx, (train_idx, val_idx) in enumerate(skf.split(all_texts, all_labels)):
        ckpt_path = os.path.join(ckpt_dir, f'{ckpt_prefix}_fold{fold_idx+1}.pt')
        print(f' Loading {ckpt_prefix} fold {fold_idx+1}/{N_FOLDS} from {ckpt_path}...')

        if not os.path.exists(ckpt_path):
            print(f' WARNING: checkpoint not found, skipping!')
            continue

        fold_val_texts = all_texts[val_idx].tolist()
        fold_val_labels = all_labels[val_idx].tolist()

        fold_val_ds = KFoldDataset(fold_val_texts, fold_val_labels, tokenizer, max_len)
        fold_val_loader = DataLoader(fold_val_ds, batch_size=16, shuffle=False)

        fold_model = model_class.from_pretrained(model_name, num_labels=2)
        fold_model = fold_model.float()
        fold_model.load_state_dict(torch.load(ckpt_path, map_location=device))
        fold_model.to(device)
        fold_model.eval()

        fold_logits = []
```

```

with torch.no_grad():
    for batch in fold_val_loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask)
        fold_logits.append(outputs.logits.cpu().numpy())

fold_logits = np.concatenate(fold_logits, axis=0)
fold_probs = sp_softmax(fold_logits, axis=1)
oof_probs[val_idx] = fold_probs

del fold_model
torch.cuda.empty_cache()

return oof_probs

# --- Generate DeBERTa OOF ---
print('=' * 60)
print('Loading DeBERTa OOF predictions')
print('=' * 60)
oof_probs = generate_oof_probs(
    DebertaV2ForSequenceClassification, DEBERTA_NAME,
    kfold_tokenizer, KFOLD_MAXLEN,
    DEBERTA_CKPT_DIR, 'deberta'
)
deberta_oof_f1 = f1_score(all_labels, np.argmax(oof_probs, axis=1))
print(f'DeBERTa OOF F1: {deberta_oof_f1:.4f}\n')

# --- Generate RoBERTa OOF ---
print('=' * 60)
print('Loading RoBERTa OOF predictions')
print('=' * 60)
roberta_oof_probs = generate_oof_probs(
    RobertaForSequenceClassification, ROBERTA_NAME,
    roberta_kfold_tokenizer, ROBERTA_MAXLEN,
    ROBERTA_CKPT_DIR, 'roberta'
)
roberta_oof_f1 = f1_score(all_labels, np.argmax(roberta_oof_probs, axis=1))
print(f'RoBERTa OOF F1: {roberta_oof_f1:.4f}\n')

print('=' * 60)
print('Both oof_probs and roberta_oof_probs loaded - ready for ensemble.')
print('=' * 60)

=====
Train: 5126 samples, Val: 1281 samples
Loading weights: 100% 197/197 [00:00<00:00, 3076.00it/s]

[transformers] RobertaForSequenceClassification LOAD REPORT from: roberta-base
Key | Status |
-----+-----+
lm_head.dense.bias | UNEXPECTED |
lm_head.layer_norm.weight | UNEXPECTED |
lm_head.layer_norm.bias | UNEXPECTED |
lm_head.dense.weight | UNEXPECTED |
lm_head.bias | UNEXPECTED |
classifier.dense.bias | MISSING |
classifier.dense.weight | MISSING |
classifier.out_proj.weight | MISSING |
classifier.out_proj.bias | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
Step 50/641 | Loss: 0.6199 | Avg Loss: 0.7026 | LR: 1.30e-06
Step 100/641 | Loss: 0.6478 | Avg Loss: 0.6917 | LR: 2.60e-06
Step 150/641 | Loss: 0.7784 | Avg Loss: 0.6964 | LR: 3.91e-06
Step 200/641 | Loss: 0.7210 | Avg Loss: 0.6973 | LR: 5.21e-06
Step 250/641 | Loss: 0.6625 | Avg Loss: 0.6961 | LR: 6.51e-06
Step 300/641 | Loss: 0.6962 | Avg Loss: 0.6958 | LR: 7.81e-06
Step 350/641 | Loss: 0.6668 | Avg Loss: 0.6906 | LR: 9.11e-06
Step 400/641 | Loss: 0.9218 | Avg Loss: 0.6742 | LR: 9.95e-06
Step 450/641 | Loss: 0.3234 | Avg Loss: 0.6582 | LR: 9.81e-06
Step 500/641 | Loss: 0.7528 | Avg Loss: 0.6459 | LR: 9.66e-06
Step 550/641 | Loss: 0.1896 | Avg Loss: 0.6267 | LR: 9.52e-06
Step 600/641 | Loss: 0.2108 | Avg Loss: 0.6082 | LR: 9.38e-06
Epoch 1/3 | Train Loss: 0.5982 | Val Loss: 0.4025 | Acc: 0.8447 | F1: 0.8658 << saved
Step 50/641 | Loss: 0.9312 | Avg Loss: 0.4231 | LR: 9.12e-06
Step 100/641 | Loss: 0.4764 | Avg Loss: 0.4101 | LR: 8.97e-06
Step 150/641 | Loss: 0.3110 | Avg Loss: 0.3865 | LR: 8.83e-06
Step 200/641 | Loss: 0.3051 | Avg Loss: 0.3770 | LR: 8.68e-06
Step 250/641 | Loss: 0.1691 | Avg Loss: 0.3676 | LR: 8.54e-06
Step 300/641 | Loss: 0.3321 | Avg Loss: 0.3667 | LR: 8.39e-06
Step 350/641 | Loss: 0.4352 | Avg Loss: 0.3627 | LR: 8.25e-06
Step 400/641 | Loss: 0.0998 | Avg Loss: 0.3538 | LR: 8.11e-06
Step 450/641 | Loss: 0.2344 | Avg Loss: 0.3522 | LR: 7.96e-06
Step 500/641 | Loss: 0.4877 | Avg Loss: 0.3529 | LR: 7.82e-06

```

```
Step 550/641 | Loss: 0.1610 | Avg Loss: 0.3494 | LR: 7.67e-06
Step 600/641 | Loss: 0.6343 | Avg Loss: 0.3525 | LR: 7.53e-06
Epoch 2/3 | Train Loss: 0.3533 | Val Loss: 0.2962 | Acc: 0.8720 | F1: 0.8853 << saved
Step 50/641 | Loss: 0.7445 | Avg Loss: 0.2421 | LR: 7.27e-06
Step 100/641 | Loss: 0.0359 | Avg Loss: 0.2444 | LR: 7.12e-06
Step 150/641 | Loss: 0.0645 | Avg Loss: 0.2468 | LR: 6.98e-06
Step 200/641 | Loss: 0.7881 | Avg Loss: 0.2464 | LR: 6.83e-06
Step 250/641 | Loss: 0.3036 | Avg Loss: 0.2435 | LR: 6.69e-06
Step 300/641 | Loss: 0.0666 | Avg Loss: 0.2484 | LR: 6.55e-06
Step 350/641 | Loss: 0.0283 | Avg Loss: 0.2380 | LR: 6.40e-06
Step 400/641 | Loss: 0.0161 | Avg Loss: 0.2355 | LR: 6.26e-06
Step 450/641 | Loss: 0.0115 | Avg Loss: 0.2384 | LR: 6.11e-06
Step 500/641 | Loss: 0.3769 | Avg Loss: 0.2331 | LR: 5.97e-06
Step 550/641 | Loss: 0.1557 | Avg Loss: 0.2393 | LR: 5.82e-06
Step 600/641 | Loss: 0.2648 | Avg Loss: 0.2387 | LR: 5.68e-06
Epoch 3/3 | Train Loss: 0.2404 | Val Loss: 0.4209 | Acc: 0.8681 | F1: 0.8840
```

Fold 4 best: Acc=0.8720, F1=0.8853 (14.0 min)

=====

ROBERTA FOLD 5/5

=====

Train: 5126 samples, Val: 1281 samples

Loading weights: 100%

197/197 [00:00<00:00, 5196.55it/s]

[transformers] RobertaForSequenceClassification LOAD REPORT from: roberta-base

Key	Status
lm_head.dense.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
lm_head.bias	UNEXPECTED
classifier.dense.bias	MISSING
classifier.dense.weight	MISSING
classifier.out_proj.weight	MISSING
classifier.out_proj.bias	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```
Step 50/641 | Loss: 0.7088 | Avg Loss: 0.6930 | LR: 1.30e-06
Step 100/641 | Loss: 0.7145 | Avg Loss: 0.6938 | LR: 2.60e-06
Step 150/641 | Loss: 0.6785 | Avg Loss: 0.6920 | LR: 3.91e-06
Step 200/641 | Loss: 0.6591 | Avg Loss: 0.6915 | LR: 5.21e-06
Step 250/641 | Loss: 0.6421 | Avg Loss: 0.6915 | LR: 6.51e-06
Step 300/641 | Loss: 0.6815 | Avg Loss: 0.6925 | LR: 7.81e-06
Step 350/641 | Loss: 0.6900 | Avg Loss: 0.6922 | LR: 9.11e-06
Step 400/641 | Loss: 0.5113 | Avg Loss: 0.6791 | LR: 9.95e-06
Step 450/641 | Loss: 0.2010 | Avg Loss: 0.6605 | LR: 9.81e-06
Step 500/641 | Loss: 0.2469 | Avg Loss: 0.6364 | LR: 9.66e-06
Step 550/641 | Loss: 0.4576 | Avg Loss: 0.6199 | LR: 9.52e-06
Step 600/641 | Loss: 0.4166 | Avg Loss: 0.6066 | LR: 9.38e-06
Epoch 1/3 | Train Loss: 0.5902 | Val Loss: 0.4432 | Acc: 0.7931 | F1: 0.8189 << saved
Step 50/641 | Loss: 0.4732 | Avg Loss: 0.4073 | LR: 9.12e-06
Step 100/641 | Loss: 0.4089 | Avg Loss: 0.4007 | LR: 8.97e-06
Step 150/641 | Loss: 0.2920 | Avg Loss: 0.3912 | LR: 8.83e-06
Step 200/641 | Loss: 1.0021 | Avg Loss: 0.3758 | LR: 8.68e-06
Step 250/641 | Loss: 0.8702 | Avg Loss: 0.3629 | LR: 8.54e-06
Step 300/641 | Loss: 0.4503 | Avg Loss: 0.3440 | LR: 8.39e-06
Step 350/641 | Loss: 0.3419 | Avg Loss: 0.3573 | LR: 8.25e-06
Step 400/641 | Loss: 0.0941 | Avg Loss: 0.3509 | LR: 8.11e-06
Step 450/641 | Loss: 0.3635 | Avg Loss: 0.3456 | LR: 7.96e-06
Step 500/641 | Loss: 0.3165 | Avg Loss: 0.3387 | LR: 7.82e-06
Step 550/641 | Loss: 0.1194 | Avg Loss: 0.3332 | LR: 7.67e-06
Step 600/641 | Loss: 0.4293 | Avg Loss: 0.3315 | LR: 7.53e-06
Epoch 2/3 | Train Loss: 0.3327 | Val Loss: 0.3267 | Acc: 0.8665 | F1: 0.8792 << saved
Step 50/641 | Loss: 0.1832 | Avg Loss: 0.2223 | LR: 7.27e-06
Step 100/641 | Loss: 0.5487 | Avg Loss: 0.2367 | LR: 7.12e-06
Step 150/641 | Loss: 0.3139 | Avg Loss: 0.2275 | LR: 6.98e-06
Step 200/641 | Loss: 0.3628 | Avg Loss: 0.2432 | LR: 6.83e-06
Step 250/641 | Loss: 0.0480 | Avg Loss: 0.2365 | LR: 6.69e-06
Step 300/641 | Loss: 0.3547 | Avg Loss: 0.2390 | LR: 6.55e-06
Step 350/641 | Loss: 0.0603 | Avg Loss: 0.2351 | LR: 6.40e-06
Step 400/641 | Loss: 0.0114 | Avg Loss: 0.2307 | LR: 6.26e-06
Step 450/641 | Loss: 0.6037 | Avg Loss: 0.2330 | LR: 6.11e-06
Step 500/641 | Loss: 0.0123 | Avg Loss: 0.2339 | LR: 5.97e-06
Step 550/641 | Loss: 0.3644 | Avg Loss: 0.2351 | LR: 5.82e-06
Step 600/641 | Loss: 0.2688 | Avg Loss: 0.2292 | LR: 5.68e-06
Epoch 3/3 | Train Loss: 0.2252 | Val Loss: 0.4868 | Acc: 0.8657 | F1: 0.8828 << saved
```

Fold 5 best: Acc=0.8657, F1=0.8828 (14.1 min)

=====

K-FOLD ROBERTA RESULTS

=====

fold	accuracy	f1	time_min
1	0.872075	0.882184	13.976618
2	0.877535	0.889359	14.147032
3	0.863388	0.873828	14.168537

```

4 0.871975 0.885515 14.057058
Device:0:885730 0.882834 14.078472
=====
b0a0a1a8 DeBERTaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base
Mean fold 1/5 from /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/kfold/deberta_fold1.pt...
Total time: 579/579 [00:00<00:00, 69.8kB/s]

config.json: 100%
pytorch_model.bin: 100%
Loading weights: 100%
[transformers] DeBERTaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base
Key | Status |
-----+-----+
lm_predictions.lm_head.dense.weight | UNEXPECTED |
mask_predictions.classifier.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.bias | UNEXPECTED |
lm_predictions.lm_head.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.weight | UNEXPECTED |
lm_predictions.lm_head.dense.bias | UNEXPECTED |
mask_predictions.dense.bias | UNEXPECTED |
mask_predictions.LayerNorm.weight | UNEXPECTED |
mask_predictions.LayerNorm.bias | UNEXPECTED |
mask_predictions.classifier.weight | UNEXPECTED |
mask_predictions.dense.weight | UNEXPECTED |
classifier.weight | MISSING |
pooler.dense.weight | MISSING |
classifier.bias | MISSING |
pooler.dense.bias | MISSING |

```

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```

model.safetensors: 100%
Loading deberta fold 2/5 from /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/kfold/deberta_fold2.pt...
Loading weights: 100%
[transformers] DeBERTaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base
Key | Status |
-----+-----+
lm_predictions.lm_head.dense.weight | UNEXPECTED |
mask_predictions.classifier.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.bias | UNEXPECTED |
lm_predictions.lm_head.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.weight | UNEXPECTED |
lm_predictions.lm_head.dense.bias | UNEXPECTED |
mask_predictions.dense.bias | UNEXPECTED |
mask_predictions.LayerNorm.weight | UNEXPECTED |
mask_predictions.LayerNorm.bias | UNEXPECTED |
mask_predictions.classifier.weight | UNEXPECTED |
mask_predictions.dense.weight | UNEXPECTED |
classifier.weight | MISSING |
pooler.dense.weight | MISSING |
classifier.bias | MISSING |
pooler.dense.bias | MISSING |

```

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```

Loading deberta fold 3/5 from /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/kfold/deberta_fold3.pt...
Loading weights: 100%
[transformers] DeBERTaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base
Key | Status |
-----+-----+
lm_predictions.lm_head.dense.weight | UNEXPECTED |
mask_predictions.classifier.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.bias | UNEXPECTED |
lm_predictions.lm_head.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.weight | UNEXPECTED |
lm_predictions.lm_head.dense.bias | UNEXPECTED |
mask_predictions.dense.bias | UNEXPECTED |
mask_predictions.LayerNorm.weight | UNEXPECTED |
mask_predictions.LayerNorm.bias | UNEXPECTED |
mask_predictions.classifier.weight | UNEXPECTED |
mask_predictions.dense.weight | UNEXPECTED |
classifier.weight | MISSING |
pooler.dense.weight | MISSING |
classifier.bias | MISSING |
pooler.dense.bias | MISSING |

```

6.5 DeBERTa + RoBERTa Ensemble OOF Evaluation

```

#@title 6.5 DeBERTa + RoBERTa Ensemble OOF Evaluation

ensemble_oof_probs = (oof_probs + roberta_oof_probs) / 2

thresholds = np.arange(0.40, 0.70, 0.01)
ensemble_thresh_results = []

for t in thresholds:
    preds = (ensemble_oof_probs[:, 1] > t).astype(int)
    f1 = f1_score(all_labels, preds)
    acc = accuracy_score(all_labels, preds)
    ensemble_thresh_results.append({'threshold': round(t, 2), 'f1': f1, 'accuracy': acc})

```

```

ens_thresh_df = pd.DataFrame(ensemble_thresh_results)
ens_best_row = ens_thresh_df.loc[ens_thresh_df['f1'].idxmax()]
ensemble_best_threshold = ens_best_row['threshold']

ensemble_default_preds = np.argmax(ensemble_oof_probs, axis=1)
ensemble_default_f1 = f1_score(all_labels, ensemble_default_preds)

print(f'{"=" * 60}')
print(f'OOF COMPARISON')
print(f'{"=" * 60}')
print(f'DeBERTa only (t=0.50) : F1={f1_score(all_labels, np.argmax(oof_probs, axis=1)):.4f}')
print(f'RoBERTa only (t=0.50) : F1={roberta_oof_f1:.4f}')
print(f'Ensemble (t=0.50) : F1={ensemble_default_f1:.4f}')
print(f'Ensemble (t={ensemble_best_threshold:.2f}) : F1={ens_best_row["f1"]:.4f}')
print(f'')
print(f'Best ensemble threshold : {ensemble_best_threshold:.2f}')

```

```

- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
Loading deberta fold 5/5 from /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/kfold/deberta_fold5.pt...
OOF COMPARISON

```

```

Loading weights: 100%=====198/198 [00:00<00:00, 7200.58it/s]
DeBERTa only (t=0.50) : F1=0.9448
RoBERTa only (t=0.50) : F1=0.0000
Ensemble (t=0.50) : F1=0.9448
Ensemble (t=0.40) : F1=0.9469
lm_predictions.lm_head.dense.weight | UNEXPECTED |
mask_predictions.classifier.bias | UNEXPECTED |
Best ensemble threshold: 0.40
lm_predictions.lm_head.LayerNorm.bias | UNEXPECTED |
lm_predictions.lm_head.bias | UNEXPECTED |

```

```

import os
import gdown

os.makedirs('fold_checkpoints', exist_ok=True)
os.makedirs('fold_checkpoints_roberta', exist_ok=True)

# Paste file IDs from Account 1
DEBERTA_IDS = [
    '1xsyPDsThYA3mr52wgDhsNf82VcBU7Y-5',
    '1dQe_MiX3hqLxD7eSTP4_GTlIvFjutzfY',
    '1nKrQ7XxgDGJ7xmTnk5TuleCZTjgwSPr',
    '17Eym80F--GjV2GeV09yYk5d1-cXRYrJJ',
    '1qGZMRN9KuLHbqh6LH_h3j_aNJiywLWWN'
]
ROBERTA_IDS = [
    '1Hyd7UcBTOImy-OoVnbWFZrjn08cqz8eJ',
    '1Srd6tyPoRS6MyPsynXlZxAQNCq0sVicE',
    '1JV2sg9kvUpr7z9UDcgMiqoWFjy1_Ug0X',
    '1AMuBo2a4fyxLrad39EvrEU5jbKwo8TDR',
    '1bubV_fAh4Y2LXkyfv_HC1Rcr6y-mSY5m']

for i, fid in enumerate(DEBERTA_IDS, 1):
    gdown.download(f'https://drive.google.com/uc?id={fid}',
        f'fold_checkpoints/deberta_fold{i}.pt', quiet=False)

for i, fid in enumerate(ROBERTA_IDS, 1):
    gdown.download(f'https://drive.google.com/uc?id={fid}',
        f'fold_checkpoints_roberta/roberta_fold{i}.pt', quiet=False)

```

```

WARNING: checkpoint not found, skipping!
RoBERTa only (t=0.50) : F1=0.0000
From (original): https://drive.google.com/uc?id=1xsyPDsThYA3mr52wgDhsNf82VcBU7Y-5
From (redirected): https://drive.google.com/uc?id=1xsyPDsThYA3mr52wgDhsNf82VcBU7Y-5&confirm=t&uuiid=171915ea-21cd-44e1-9fc8-bae7b1b71fcc
Both /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/deberta_fold1.pt
100%|██████████| 738M/738M [00:10<00:00, 68.5MB/s]
Downloading...
From (original): https://drive.google.com/uc?id=1dQe_MiX3hqLxD7eSTP4_GTlIvFjutzfY
From (redirected): https://drive.google.com/uc?id=1dQe_MiX3hqLxD7eSTP4_GTlIvFjutzfY&confirm=t&uuiid=9e5083b3-3a1c-43d0-a024-5a131babef0e
To: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/deberta_fold2.pt
100%|██████████| 738M/738M [00:12<00:00, 56.8MB/s]
Downloading...
From (original): https://drive.google.com/uc?id=1nKrQ7XxgDGJ7xmTnk5TuleCZTjgwSPr
From (redirected): https://drive.google.com/uc?id=1nKrQ7XxgDGJ7xmTnk5TuleCZTjgwSPr&confirm=t&uuiid=625d515f-8404-4635-bfe6-6f00133f07d6
To: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/deberta_fold3.pt
100%|██████████| 738M/738M [00:12<00:00, 60.5MB/s]
Downloading...
From (original): https://drive.google.com/uc?id=17Eym80F--GjV2GeV09yYk5d1-cXRYrJJ
From (redirected): https://drive.google.com/uc?id=17Eym80F--GjV2GeV09yYk5d1-cXRYrJJ&confirm=t&uuiid=17313933-e245-49cb-bf64-f50437a61d16
To: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/deberta_fold4.pt
100%|██████████| 738M/738M [00:15<00:00, 46.5MB/s]
Downloading...
From (original): https://drive.google.com/uc?id=1qGZMRN9KuLHbqh6LH_h3j_aNJiywLWWN
From (redirected): https://drive.google.com/uc?id=1qGZMRN9KuLHbqh6LH_h3j_aNJiywLWWN&confirm=t&uuiid=3eba73a2-fe81-4be2-9400-e029f6a14119
To: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints/deberta_fold5.pt
100%|██████████| 738M/738M [00:14<00:00, 51.7MB/s]
Downloading...
From (original): https://drive.google.com/uc?id=1Hyd7UcBTOImy-OoVnbWFZrjn08cqz8eJ
From (redirected): https://drive.google.com/uc?id=1Hyd7UcBTOImy-OoVnbWFZrjn08cqz8eJ&confirm=t&uuiid=1b9b745c-d68b-4457-b6fe-cb56da1f2467
To: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints_roberta/roberta_fold1.pt
100%|██████████| 499M/499M [00:03<00:00, 140MB/s]
Downloading...

```

```

From (original): https://drive.google.com/uc?id=1Srd6tyPoRS6NyPsynXl7xAQNCg0sVicE
From (redirected): https://drive.google.com/uc?id=1Srd6tyPoRS6NyPsynXl7xAQNCg0sVicE&confirm=t&uuiid=6690f2b4-e546-4a98-bcf0-9058226aa988
To: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints_roberta/roberta_fold2.pt
100%|██████████| 499M/499M [00:10<00:00, 47.4MB/s]
Downloading...
From (original): https://drive.google.com/uc?id=1JV2sg9kvUpr7z9UDcgMiqoWFjyl\_Ug0X
From (redirected): https://drive.google.com/uc?id=1JV2sg9kvUpr7z9UDcgMiqoWFjyl\_Ug0X&confirm=t&uuiid=c9b7e105-f04c-4e49-b4b1-07c34f92ed6b
To: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints_roberta/roberta_fold3.pt
100%|██████████| 499M/499M [00:08<00:00, 61.9MB/s]
Downloading...
From (original): https://drive.google.com/uc?id=1AMuBo2a4fyxLrad39FvrEU5jbKwo8TDR
From (redirected): https://drive.google.com/uc?id=1AMuBo2a4fyxLrad39FvrEU5jbKwo8TDR&confirm=t&uuiid=232de207-36eb-441f-9b0c-4b01e6c6851e
To: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints_roberta/roberta_fold4.pt
100%|██████████| 499M/499M [00:08<00:00, 55.6MB/s]
Downloading...
From (original): https://drive.google.com/uc?id=1bubV\_fAh4Y2LXkyfv\_HC1Rcr6y-mSY5m
From (redirected): https://drive.google.com/uc?id=1bubV\_fAh4Y2LXkyfv\_HC1Rcr6y-mSY5m&confirm=t&uuiid=d41ab48d-cd31-4abd-a1cb-b0c57ecdeb58
To: /content/drive/MyDrive/SMU_MITB_NLP/Assignment/fold_checkpoints_roberta/roberta_fold5.pt
100%|██████████| 499M/499M [00:11<00:00, 43.5MB/s]

```

6.6 Generate Ensemble Eval Predictions

```

#@title 6.6 Generate Ensemble Eval Predictions

# --- DeBERTa eval predictions ---
deberta_eval_ds = KFoldPredDataset(eval_texts, kfold_tokenizer, KFOLD_MAXLEN)
deberta_eval_loader = DataLoader(deberta_eval_ds, batch_size=16, shuffle=False)

eval_ensemble_probs = np.zeros((len(eval_texts), 2), dtype=np.float32)

for fold_idx in range(N_FOLDS):
    print(f'DeBERTa fold {fold_idx+1}/{N_FOLDS}...')

    ckpt_path = f'fold_checkpoints/deberta_fold{fold_idx+1}.pt'
    if not os.path.exists(ckpt_path):
        ckpt_path = f'/content/drive/MyDrive/SMU_MITB_NLP/Assignment/checkpoints/kfold_deberta_seed42/deberta_fold{fold_idx+1}.pt'

    fold_model = DebertaV2ForSequenceClassification.from_pretrained(DEBERTA_NAME, num_labels=2)
    fold_model = fold_model.float()
    fold_model.load_state_dict(torch.load(ckpt_path, map_location=device))
    fold_model.to(device)
    fold_model.eval()

    fold_logits = []
    with torch.no_grad():
        for batch in deberta_eval_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask)
            fold_logits.append(outputs.logits.cpu().numpy())

    fold_logits = np.concatenate(fold_logits, axis=0)
    eval_ensemble_probs += sp_softmax(fold_logits, axis=1)

    del fold_model
    torch.cuda.empty_cache()

eval_ensemble_probs /= N_FOLDS
print(f'DeBERTa eval done.\n')

# --- RoBERTa eval predictions ---
roberta_eval_ds = KFoldPredDataset(eval_texts, roberta_kfold_tokenizer, ROBERTA_MAXLEN)
roberta_eval_loader = DataLoader(roberta_eval_ds, batch_size=16, shuffle=False)

roberta_eval_probs = np.zeros((len(eval_texts), 2), dtype=np.float32)

for fold_idx in range(N_FOLDS):
    print(f'RoBERTa fold {fold_idx+1}/{N_FOLDS}...')

    ckpt_path = f'fold_checkpoints_roberta/roberta_fold{fold_idx+1}.pt'
    if not os.path.exists(ckpt_path):
        ckpt_path = f'/content/drive/MyDrive/SMU_MITB_NLP/Assignment/checkpoints/kfold_roberta_seed42/roberta_fold{fold_idx+1}.pt'

    fold_model = RobertaForSequenceClassification.from_pretrained(ROBERTA_NAME, num_labels=2)
    fold_model.load_state_dict(torch.load(ckpt_path, map_location=device))
    fold_model.to(device)
    fold_model.eval()

    fold_logits = []
    with torch.no_grad():
        for batch in roberta_eval_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)

```

```

        outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask)
        fold_logits.append(outputs.logits.cpu().numpy())

    fold_logits = np.concatenate(fold_logits, axis=0)
    roberta_eval_probs += sp_softmax(fold_logits, axis=1)

    del fold_model
    torch.cuda.empty_cache()

roberta_eval_probs /= N_FOLDS
print(f'RoBERTa eval done.\n')

# --- Ensemble: average DeBERTa + RoBERTa ---
final_ensemble_probs = (eval_ensemble_probs + roberta_eval_probs) / 2

final_preds_default = np.argmax(final_ensemble_probs, axis=1)
final_preds_tuned = (final_ensemble_probs[:, 1] > ensemble_best_threshold).astype(int)
final_preds_057 = (final_ensemble_probs[:, 1] > 0.57).astype(int)

print(f'Ensemble predictions:')
print(f' Default (0.50): Safe={{(final_preds_default==0).sum()}}, Unsafe={{(final_preds_default==1).sum()}}')
print(f' Tuned ({ensemble_best_threshold:.2f}): Safe={{(final_preds_tuned==0).sum()}}, Unsafe={{(final_preds_tuned==1).sum()}}')
print(f' Fixed (0.57): Safe={{(final_preds_057==0).sum()}}, Unsafe={{(final_preds_057==1).sum()}}')

```

DeBERTa fold 1/5...

```

Loading weights: 100%                                198/198 [00:00<00:00, 3128.45it/s]

[transformers] DebertaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base
Key | Status |
-----+-----+
lm_predictions.lm_head.dense.weight | UNEXPECTED |
mask_predictions.classifier.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.bias | UNEXPECTED |
lm_predictions.lm_head.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.weight | UNEXPECTED |
lm_predictions.lm_head.dense.bias | UNEXPECTED |
mask_predictions.dense.bias | UNEXPECTED |
mask_predictions.LayerNorm.weight | UNEXPECTED |
mask_predictions.LayerNorm.bias | UNEXPECTED |
mask_predictions.classifier.weight | UNEXPECTED |
mask_predictions.dense.weight | UNEXPECTED |
classifier.weight | MISSING |
pooler.dense.weight | MISSING |
classifier.bias | MISSING |
pooler.dense.bias | MISSING |

```

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
 - MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
- DeBERTa fold 2/5...

```

Loading weights: 100%                                198/198 [00:00<00:00, 1599.41it/s]

[transformers] DebertaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base
Key | Status |
-----+-----+
lm_predictions.lm_head.dense.weight | UNEXPECTED |
mask_predictions.classifier.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.bias | UNEXPECTED |
lm_predictions.lm_head.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.weight | UNEXPECTED |
lm_predictions.lm_head.dense.bias | UNEXPECTED |
mask_predictions.dense.bias | UNEXPECTED |
mask_predictions.LayerNorm.weight | UNEXPECTED |
mask_predictions.LayerNorm.bias | UNEXPECTED |
mask_predictions.classifier.weight | UNEXPECTED |
mask_predictions.dense.weight | UNEXPECTED |
classifier.weight | MISSING |
pooler.dense.weight | MISSING |
classifier.bias | MISSING |
pooler.dense.bias | MISSING |

```

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
 - MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
- DeBERTa fold 3/5...

```

Loading weights: 100%                                198/198 [00:00<00:00, 3388.04it/s]

[transformers] DebertaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base
Key | Status |
-----+-----+
lm_predictions.lm_head.dense.weight | UNEXPECTED |
mask_predictions.classifier.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.bias | UNEXPECTED |
lm_predictions.lm_head.bias | UNEXPECTED |
lm_predictions.lm_head.LayerNorm.weight | UNEXPECTED |

```

6.7 Save Ensemble Submissions

```

#@title 6.7 Save Ensemble Submissions
from datetime import datetime

```

```

from google.colab import files

today = datetime.now().strftime('%Y%m%d')

drive_folder = '/content/drive/MyDrive/SMU_MITB_NLP/Assignment/Submission'
os.makedirs(drive_folder, exist_ok=True)
existing = [f for f in os.listdir(drive_folder) if f.startswith(today)]
next_num = len(existing) + 1

submissions = [
    (final_preds_default, 'default', 0.50),
    (final_preds_tuned, f'thresh{ensemble_best_threshold:.2f}', ensemble_best_threshold),
    (final_preds_057, 'thresh0.57', 0.57),
]

for i, (preds, label, thresh) in enumerate(submissions):
    oof_f1_at_t = f1_score(all_labels, (ensemble_oof_probs[:, 1] > thresh).astype(int))
    fname = f'{today}_{next_num+i:02d}_ensemble_deberta_roberta_{label}_OOF_F1_{oof_f1_at_t:.4f}.csv'

    sub = pd.DataFrame({'id': eval_df['id'], 'label': preds})
    sub.to_csv(fname, index=False)
    sub.to_csv(os.path.join(drive_folder, fname), index=False)
    print(f'Saved: {fname}')

print('\nDownloading...')
for i, (preds, label, thresh) in enumerate(submissions):
    oof_f1_at_t = f1_score(all_labels, (ensemble_oof_probs[:, 1] > thresh).astype(int))
    fname = f'{today}_{next_num+i:02d}_ensemble_deberta_roberta_{label}_OOF_F1_{oof_f1_at_t:.4f}.csv'
    files.download(fname)

```

Notes: 20260613_03 ensemble deberta roberta default OOF F1 0.0000.csv
 Saved: 20260613_04 ensemble deberta roberta thresh0.40 OOF F1 0.9469.csv
 Saved: 20260613_05 ensemble deberta roberta thresh0.57 OOF F1 0.0000.csv
 DeBERTa fold 5/5...

100% 198/198 [00:00<00:00, 3572.12it/s]

[transformers] DebertaV2ForSequenceClassification LOAD REPORT from: microsoft/deberta-v3-base

```

Key | Status |
lm_predictions.lm_head.dense.weight | UNEXPECTED |

```

6.8 Weighted Ensemble (favor DeBERTa)

```

#@title 6.8 Weighted Ensemble (favor DeBERTa)
from datetime import datetime
from google.colab import files

# Try multiple weight combinations
weight_configs = [
    (1.0, 0.0), # DeBERTa only (baseline)
    (0.8, 0.2),
    (0.7, 0.3),
    (0.6, 0.4),
    (0.5, 0.5), # Equal (current ensemble)
]

today = datetime.now().strftime('%Y%m%d')
drive_folder = '/content/drive/MyDrive/SMU_MITB_NLP/Assignment/Submission'
os.makedirs(drive_folder, exist_ok=True)
existing = [f for f in os.listdir(drive_folder) if f.startswith(today)]
next_num = len(existing) + 1

results = []
saved_files = []

print('=' * 70)
print('WEIGHTED ENSEMBLE COMPARISON')
print('=' * 70)
print(f'{"DeBERTa":>10} {"RoBERTa":>10} {"OOF F1 (t=0.57)":>18}')
print('-' * 70)

for i, (w_deb, w_rob) in enumerate(weight_configs):
    # OOF probs with this weighting
    weighted_oof = w_deb * oof_probs + w_rob * roberta_oof_probs
    # Eval probs with this weighting
    weighted_eval = w_deb * eval_ensemble_probs + w_rob * roberta_eval_probs

    # OOF F1 at threshold 0.57
    oof_preds_057 = (weighted_oof[:, 1] > 0.57).astype(int)
    oof_f1_057 = f1_score(all_labels, oof_preds_057)

    # Eval predictions at threshold 0.57
    eval_preds_057 = (weighted_eval[:, 1] > 0.57).astype(int)

    print(f'{w_deb:>10.1f} {w_rob:>10.1f} {oof_f1_057:>18.4f}')

```



```

print(f'{" " * 60}')
fold_start = time.time()

fold_train_texts = all_texts[train_idx].tolist()
fold_train_labels = all_labels[train_idx].tolist()
fold_val_texts = all_texts[val_idx].tolist()
fold_val_labels = all_labels[val_idx].tolist()

fold_train_ds = KFoldDataset(fold_train_texts, fold_train_labels, kfold_tokenizer, KFOLD_MAXLEN)
fold_val_ds = KFoldDataset(fold_val_texts, fold_val_labels, kfold_tokenizer, KFOLD_MAXLEN)

fold_train_loader = DataLoader(fold_train_ds, batch_size=KFOLD_BS, shuffle=True)
fold_val_loader = DataLoader(fold_val_ds, batch_size=KFOLD_BS, shuffle=False)

print(f' Train: {len(fold_train_texts)} samples, Val: {len(fold_val_texts)} samples')

fold_model = DebertaV2ForSequenceClassification.from_pretrained(DEBERTA_NAME, num_labels=2)
fold_model = fold_model.float()
fold_model.to(device)

fold_optimizer = torch.optim.AdamW(
    fold_model.parameters(), lr=KFOLD_LR, weight_decay=0.01, eps=1e-6
)
fold_total_steps = len(fold_train_loader) * KFOLD_EPOCHS
fold_scheduler = get_linear_schedule_with_warmup(
    fold_optimizer,
    num_warmup_steps=int(fold_total_steps * 0.1),
    num_training_steps=fold_total_steps
)

fold_best_f1 = 0
checkpoint_path = f'checkpoints/kfold_deberta_seed123/deberta_seed2_fold{fold_idx+1}.pt'

for epoch in range(KFOLD_EPOCHS):
    fold_model.train()
    fold_optimizer.zero_grad()
    epoch_loss = 0
    log_interval = 50

    for step, batch in enumerate(fold_train_loader):
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss / ACCUM_STEPS
        loss.backward()
        epoch_loss += outputs.loss.item()

        if (step + 1) % ACCUM_STEPS == 0:
            torch.nn.utils.clip_grad_norm_(fold_model.parameters(), max_norm=1.0)
            fold_optimizer.step()
            fold_scheduler.step()
            fold_optimizer.zero_grad()

        if (step + 1) % log_interval == 0:
            avg_loss_so_far = epoch_loss / (step + 1)
            lr_now = fold_scheduler.get_last_lr()[0]
            print(f' Step {step+1}/{len(fold_train_loader)} | '
                  f'Loss: {outputs.loss.item():.4f} | '
                  f'Avg Loss: {avg_loss_so_far:.4f} | '
                  f'LR: {lr_now:.2e}')

    fold_model.eval()
    val_preds_list, val_labels_list = [], []
    val_loss = 0
    with torch.no_grad():
        for batch in fold_val_loader:
            input_ids = batch['input_ids'].to(device)
            attention_mask = batch['attention_mask'].to(device)
            labels_batch = batch['labels'].to(device)
            outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask, labels=labels_batch)
            val_loss += outputs.loss.item()
            preds = torch.argmax(outputs.logits, dim=1)
            val_preds_list.extend(preds.cpu().numpy())
            val_labels_list.extend(labels_batch.cpu().numpy())

    epoch_acc = accuracy_score(val_labels_list, val_preds_list)
    epoch_f1 = f1_score(val_labels_list, val_preds_list)
    avg_train_loss = epoch_loss / len(fold_train_loader)
    avg_val_loss = val_loss / len(fold_val_loader)

```

```

status = ''
if epoch_f1 > fold_best_f1:
    fold_best_f1 = epoch_f1
    torch.save(fold_model.state_dict(), checkpoint_path)
    status = ' << saved'

print(f' Epoch {epoch+1}/{KFOLD_EPOCHS} | '
      f'Train Loss: {avg_train_loss:.4f} | '
      f'Val Loss: {avg_val_loss:.4f} | '
      f'Acc: {epoch_acc:.4f} | '
      f'F1: {epoch_f1:.4f}{status}')

fold_model.load_state_dict(torch.load(checkpoint_path, map_location=device))
fold_model.eval()
fold_logits = []
with torch.no_grad():
    for batch in fold_val_loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        outputs = fold_model(input_ids=input_ids, attention_mask=attention_mask)
        fold_logits.append(outputs.logits.cpu().numpy())

fold_logits = np.concatenate(fold_logits, axis=0)
fold_probs = sp_softmax(fold_logits, axis=1)
oof_probs_seed2[val_idx] = fold_probs

fold_preds = np.argmax(fold_probs, axis=1)
fold_acc = accuracy_score(fold_val_labels, fold_preds)
fold_f1 = f1_score(fold_val_labels, fold_preds)

elapsed = time.time() - fold_start
seed2_fold_results.append({'fold': fold_idx+1, 'accuracy': fold_acc, 'f1': fold_f1, 'time_min': elapsed/60})
print(f'\n Fold {fold_idx+1} best: Acc={fold_acc:.4f}, F1={fold_f1:.4f} ({elapsed/60:.1f} min)')

del fold_model
torch.cuda.empty_cache()

seed2_oof_f1 = f1_score(all_labels, np.argmax(oof_probs_seed2, axis=1))
print(f'\n{"=" * 60}')
print(f'SEED 123 K-FOLD RESULTS')
print(f'\n{"=" * 60}')
seed2_df = pd.DataFrame(seed2_fold_results)
print(seed2_df.to_string(index=False))
print(f'\nOverall OOF F1: {seed2_oof_f1:.4f}')

```

